



UNIVERSITÀ
DEGLI STUDI
DELL'AQUILA

Dipartimento di Ingegneria e Scienze dell'Informazione
e Matematica

Ph.D.'s Degree Thesis in Information and Communication Technologies
(ICT)

Architecting Data-Intensive Applications : From Data Architecture Design to Its Quality Assurance

SSD INF/01

Supervisor
Prof. Henry Muccini

Candidate
Moamin B. Abughazala
Student ID 270451

*The Sun Rises, Giving Light.
Data Rises, Giving Wisdom*

Feras M. Awaysheh

*By Having A Well Designed
Data Architecture*

Moamin Abughaxala

Declaration

I hereby declare that the thesis entitled "Architecting Data-Intensive Applications: From Data Architecture Design to Its Quality Assurance," which I am submitting as part of my doctoral program at the University of L'Aquila, is a bona fide representation of my work. This research was undertaken under the supervision of Professor Henry Muccini, in conjunction with my colleagues and co-authors. In the present thesis, the personal pronoun "I" is used instead of "We." I affirm that I have neither submitted nor will I submit this work, either in part or in its entirety, to obtain any other degree or diploma from this or any other academic institution.

Place: L'Aquila

Date: 04/01/2024

Moamin B. Abughazla

DEDICATION

To my dearest parents and beloved family members, who have been the driving force behind my success and continue to inspire me with their unwavering love and support.

To my loving and supportive wife, who has been my rock and my partner in every step of my journey. Her constant encouragement and belief in my dreams have been the fuel that keeps me going and striving toward excellence.

To my handsome and wonderful sons, I wish nothing but the brightest and most fulfilling paths for your lives.

Abstract

Context - The exponential growth of data is becoming a significant concern. Managing this data has become incredibly challenging, especially when dealing with various sources in different formats and speeds. Moreover, Ensuring data quality has become increasingly crucial for effective decision-making and operational processes. Data Architecture is crucial in describing, collecting, storing, processing, and analyzing data to meet business needs. Providing an abstract view of data-intensive applications is essential to ensure that the data is transformed into valuable information. We must take these challenges seriously to ensure we can effectively manage and use the data to our advantage.

Objective - To establish an architecture framework that enables a comprehensive description of the data architecture and effectively streamlines data quality monitoring.

Method - The architecture framework utilizes Model Driven Engineering (MDE) techniques. Its backing of data-intensive architecture descriptions empowers with an automated generation for data quality checks.

Result - The Framework offers a comprehensive solution for data-intensive applications to model their architecture efficiently and monitor the quality of their data. It automates the entire process and ensures precision and consistency in data. With DAT, architects and analysts gain access to a powerful tool that simplifies their workflow and empowers them to make informed decisions based on reliable data insights.

Conclusion - We have evaluated the DAT on more than five cases within various industry domains, demonstrating its exceptional adaptability and effectiveness.

Keywords: Data Architecture, Data Quality, Data-Intensive

Publications

This thesis is firmly grounded in the publications described below:

1. **Abughazala, M**, Muccini, H, Sharaf, M. "Architecting Data-intensive applications: from architectural design to data quality" *Journal of Systems and Software* JSS (2023) (**Submitted**)
2. Sharaf, M., **Abughazala, M**, Muccini, Mai Abusair, "Architecting IoT applications: from architectural design to realization" *Journal of Systems and Software* JSS (2023) (**Submitted**)
3. **Abughazala, Moamin**, Henry Muccini, Khitam, Qadri " *PyDaQu: Python Data Quality Code Generation Based on Data Architecture* " Proceedings of the 26th International Conference on Model Driven Engineering Languages and Systems (**MODELS 2023**), *Tool & Demo*.
4. **Abughazala, Moamin**, Henry Muccini, and M. Sharaf. " *Architecture Description Framework For Data-Intensive Applications*" International Conference on Intelligent Data Science Technologies and Applications, (**IDSTA 2023**).
5. **Abughazala, Moamin**, and Henry Muccini. " *Modeling data analytics architecture for iot applications using dat.*" 2023 IEEE 20th International Conference on Software Architecture Companion (**ICSA-C**). IEEE, 2023.
6. **Abughazala, Moamin**, Henry Muccini, and M. Sharaf. " *Dat: Data architecture modeling tool for data-driven applications.*" European Conference on Software Architecture. (**ECSA 2022**), *Tool & Demo*.
7. **Abughazala, Moamin**, and Henry Muccini. " *Modeling Data Analytics Architecture for Smart Cities Data-Driven Applications using DAT.*" **I-Cities2023**.
8. **Abughazala, Moamin.**, Moghaddam, M. T., Muccini, H., & Vaidhyathan, K. (2021, August). *Human behavior-oriented architectural design. In European Conference on Software Architecture (ECSA 2021)* (pp. 134-143).

9. **Abughazala, Moamin**, and Henry Muccini. "*Data Architecture for Digital Object Space Management Service (DOSM) using DAT.*" **I-Cities (2022)**.

Contents

List of Figures	xii
List of Tables	xiii
Acronyms	xv
1 Introduction	1
1.1 Data	2
1.2 Data Architecture	2
1.3 Need For Data Architecture	3
1.4 Data Architecture for Data-intensive	4
1.5 Limitations	6
1.6 Contributions	6
1.7 Structure of the chapters	7
2 Background	8
2.1 Data-intensive Applications	8
2.2 4Vs of Big Data	9
2.3 IEEE/ISO/IEE 42010:2022 Architecture Description	10
2.4 Big Data Architectures	11
2.5 Data Quality	12
2.6 Summary	14
3 Research Methodology	15
3.1 Research Questions	15
3.2 Qualitative Research	16
3.3 Case Companies	17
3.4 Research Methods	18
3.4.1 Case Study	19
3.4.2 Action Research	20
3.5 Research Techniques	21
3.5.1 Interviews	21
3.5.2 Observation	21
3.6 Data Analysis	22

3.6.1	Qualitative Data Analysis	22
3.7	Research Design	23
3.8	Threats to Validity	28
3.8.1	Construct Validity	28
3.8.2	Internal Validity	28
3.8.3	External Validity	29
3.9	Summary	29
4	Architecture Description Framework For Data-Intensive Applications	30
4.1	Introduction	30
4.2	Methodology	32
4.2.1	Literature Review	33
4.2.2	Meta-Model	34
4.3	Application of <i>DAF</i> Model to the Case Study	36
4.3.1	Analytical Data Warehouse	37
4.3.2	The model applied to the Case Study	38
4.4	Evaluation	38
4.5	Threats To Validity	40
4.6	Related Work	42
4.7	Conclusion and Future Work	43
5	DAT: Data Architecture Modeling Tool for Data-Driven Applications	45
5.1	Introduction	45
5.2	Background	46
5.2.1	The CAPS Modeling Framework	46
5.2.2	The Important of Data Architecture	47
5.2.3	The DAT Tool	48
5.3	Real Use Cases	50
5.4	Evaluation	51
5.4.1	Errors Data Pipeline	52
5.4.2	Hydre (Lambda Architecture)	52
5.4.3	Operational Data Warehouse	53
5.5	Conclusion and Future Work	54
5.6	Acknowledgment	54
6	Modeling Data Analytics Architecture for Data-Driven Applications using DAT	56
6.1	Introduction	56
6.2	Background	58
6.2.1	Big Data Analytics	58
6.2.2	DAT: Data Architecture Modeling Tool	59
6.3	RESEARCH METHODOLOGY	59
6.3.1	About The Company	60
6.3.2	Exploratory Case Study	60
6.3.3	Data Collection	61

6.3.4	Data Analysis	61
6.3.5	Validation Study	62
6.4	DATA Analytics Architecture Challenges	62
6.5	Application of DAT Models to the Analytics Data Warehouse Case Study	67
6.5.1	The Analytics Data Warehouse	67
6.5.2	The DAT model applied to the ADW Case Study	69
6.6	Validation Study	69
6.7	Threats to Validity	70
6.8	Related Work	71
6.9	Conclusion and Future Work	72
7	Ensuring Data Quality Throughout the Data Journey	73
7.1	Introduction	73
7.2	Background	75
7.2.1	Great Expectations	75
7.3	Research Methodology	76
7.3.1	Definition of research questions	76
7.3.2	Overview of use cases	76
7.3.3	Exploratory Case Study	77
7.4	Data Quality Challenges	78
7.5	Modeling High-Quality Data-Intensive Applications	80
7.5.1	Structural Meta-model	80
7.5.2	Behavioral Meta-model	81
7.6	Data Quality Code Generation	82
7.6.1	Modeling	83
7.6.2	Parsing	83
7.6.3	Mapping	84
7.6.4	Generate Data Quality Check Code	84
7.7	Results	85
7.8	Related Work	88
7.9	Conclusion and Future Work	89
8	PyDaQu: Python Data Quality Code Generation Based on Data Architecture	90
8.1	Introduction	90
8.2	Background	91
8.2.1	DAT Modeling Framework	92
8.3	Related Work	92
8.4	Use Cases	93
8.4.1	Case A: ISP - Internet Service Provider	93
8.4.2	Case B: Errors Data Pipeline	93
8.5	PyDaQu Framework	94
8.5.1	Approach	94
8.5.2	Architecture	94

8.6	Evaluation	98
8.7	Conclusion	99
9	Results And Evaluation	101
9.1	Comparative to the existing approaches	101
9.1.1	Supporting the engineering of data-intensive systems	102
9.1.2	Supporting the modeling of data-intensive systems	103
9.2	Supporting High-quality data-intensive applications	104
9.3	Industry Evaluation	105
9.3.1	Evaluating The Modeled Cases	105
9.3.2	Evaluating Addressed Challenges	106
10	Concluding Remarks and Future Work	108
10.1	Key contributions	109
10.2	Future Work	109
	bibliography	111
A	Appendix	124

List of Figures

2.1	Content model of an <i>AD</i> (following ISO/IEC/IEEE 42010:2022)	11
4.1	Meta-model: structural concepts	34
4.2	Meta-model: behavioral concepts	35
4.3	Modeling of the <i>ADW</i>	37
4.4	Operational Data Warehouse	41
4.5	Hydre (Lambda+ Example)	42
4.6	Errors Data Pipeline	43
4.7	NDR case study - (HLA)	44
5.1	The Data View of CAPS	48
5.2	The Hydre architecture	50
5.3	Operational Data-warehouse	51
5.4	Errors Data Pipeline	52
5.5	Hydre (Lambda+ Example)	53
5.6	Operational Data Warehouse	55
6.1	Research Methodology	60
6.2	Modeling of the Analytics Data Warehouse	68
6.3	Lambda+ Model	71
7.1	Research Methodology	76
7.2	Structural Meta-model	80
7.3	Quality Driven Meta-Model For Data Life Cycle	82
7.4	Code Generation Framework	83
7.5	Example Of Generated Code	86
7.6	Challenges Mapped to DQDs and Great Expectations	87
8.1	PyDaQu automatic code generation framework	95
8.2	"Verify Data" Action	95
8.3	Templates Example	97
8.4	Example Of Generated Code	98
8.5	Results of running the generated code	99

List of Tables

3.1	Cases Details	19
3.2	Overview of Exploratory Case Studies	27
4.1	Summary of the DAML meta-classes and related references	33
4.2	Outline of use cases and roles of the evaluators	39
4.3	Real Case Evaluation: Main Components	40
5.1	Outline of use cases and roles of the evaluators	51
6.1	The Roles of the Interviewees	61
7.1	The Roles of the Experts	78
7.2	Part Of Mapper	84
8.1	Part Of Mapper	96
8.2	Comparing the Time Required for Manual and Automated Code	99
9.1	Comparative table on supporting different engineering capabilities	102
9.2	Comparative table on supporting different data-intensive modeling features	104
9.3	Real Case Evaluation: Main Components	106
A.1	Mapper Table from QD to GE	124

Acronyms

AD Architecture description

ADW Analytics Data Warehouse

BDA Big Data Architecture

DA Data Architecture

DAA Data Analytics Architecture

DAF Architecture Description Framework for Data-intensive Applications

DAFQA Data Architecture Quality Framework for Data-Intensive Applications

DAML Data Modeling Language

DAT Data Architecture Modeling Tool

DD Data-Driven

DQ Data Quality

DQDs Data Quality Dimensions

EMF Eclipse Modeling Framework

GE Great Expectations

HLA high-level architecture

LLA low-level architecture

MDE Model Driven Engineering

ODW Operational Data Warehouse

PyDaQu Python Data Quality Code Generation Based on Data Architecture

Chapter 1

Introduction

Data's transformative impact will certainly extend to all areas of our lives, including our work and thoughts [61]. The quantity of data is rapidly increasing due to advancements in mobile and sensing devices, social media, and web technologies. For example, Twitter handles over 70 million tweets daily, generating over eight terabytes of data daily [114]. Google generates around 2.5 million Terabytes per day. IDC expects data to reach 175 zettabytes by 2025 [29]. Data products are products designed to facilitate achieving a goal through the use of data. Google Analytics is an example of a data product whose primary purpose is to provide users with a quantitative understanding of online behavior. Instagram is not considered a data product, but its features, such as tagging and searching, can be classified as data products. These products include raw data, derived data, algorithms, decision support, and automated decision-making [113]. Data can be either raw or derived. Raw data is collected directly from the source and stored for later use, while derived data is processed from raw data. Algorithms are used to analyze data and provide insights. For instance, Google Image enables users to input an image and get similar images as output. The product matches images and provides similar ones. Decision support systems like Google Analytics help with decision-making, but users interpret results. Automated decision-making systems present the final output to the user. One instance of an automated decision-making system is the movie suggestions provided by Netflix [113].

Companies are utilizing the data they have gathered throughout the years for various purposes. Data enhances quality assurance and diagnostics, leading to a notable reduction in troubleshooting efforts. Additionally, some companies use data to improve features, functionality, and performance optimization.

1.1 Data

Data is a collection of information expressed in various forms, such as facts, measurements, or observations. It finds application in diverse fields, from business intelligence to analytics, artificial intelligence, and machine learning. The role of data in driving innovation and progress cannot be overstated. It is the foundation upon which IoT, personalization, healthcare, finance, transportation, social media, climate analysis, education, smart cities, scientific research, security, surveillance, and environmental monitoring are built. As technology advances, the importance and impact of data will only continue to grow, shaping our lives and transforming how we interact with the world.

1.2 Data Architecture

In the realm of information technology and data management, *Data Architecture (DA)* is a crucial concept. It involves *arranging, blueprinting, and managing an organization's data resources, encompassing how data is collected, stored, managed, and utilized* [123] [54] [57]. A well-established *DA* is essential for effectively managing and capitalizing on data to support an organization's strategic objectives, decision-making procedures, and overall operations [101].

When designing a data architecture, it's important to consider factors such as the data's amount, variety, speed, and accuracy [60]. Additionally, it's necessary to consider the technologies, tools, and platforms that will best fit the organization's requirements and align with its business goals. *DA* is essential for managing internal data and integrating and exchanging data with external partners, vendors, and customers [101].

Effective *DA* is essential for advanced data analytics, business intelligence, and arti-

ficial intelligence applications [124]. It allows organizations to extract valuable insights from their data, make data-intensive decisions, and stay ahead of the competition in today's data-intensive world [92] [14]. *DA* is essential in today's fast-paced digital world. It helps organizations utilize the power of data, fosters innovation, and guides them toward success [9]. It's an integral part of any comprehensive data management strategy and is constantly refined to adapt to changing business needs and emerging technologies [1].

1.3 Need For Data Architecture

The significance of *DA* lies in its ability to serve as a strategic framework that helps organizations *manage their data* efficiently [109] [53].

Effective *DA* allows for *integrating data* from various sources, systems, and applications, resulting in a cohesive data view. Such integration is essential for making informed decisions that rely on a comprehensive understanding of the available data [53].

A well-designed *DA* is important because it establishes *data governance* principles, *data standards*, and *data validation rules*. This helps ensure the data is consistent, accurate, and high-quality, ultimately leading to better decision-making and more reliable insights [55] [53]. As the amount of data increases, proper *DA* becomes essential to ensure that data storage, processing, and retrieval systems can handle the growing demands. By optimizing the flow of data, it leads to improved *system performance* [110].

A well-designed data architecture can *reduce redundancies*, inefficiencies, and data silos, leading to significant cost savings in managing and maintaining data infrastructure [53] [110]. It allows for smooth *data sharing* and collaboration between various organizational departments and stakeholders. This promotes a culture of collaboration and knowledge exchange driven by data [53] [110] [111]. It gives the power to utilize *data-intensive insights* to drive innovation and stay ahead of the competition in their respective markets.

1.4 Data Architecture for Data-intensive

Many small and medium-sized companies are interested in implementing data-intensive applications, but they face challenges in the development phase that prevent them from reaching their full potential. The development of real-time, data-intensive applications is not a straightforward task, and differs from the traditional software engineering process [72]. There is an increasing demand to understand the development, deployment, and maintenance of data-intensive applications in real-world commercial settings. Big data analytics platforms, Real-time data processing systems, data warehousing solutions, content delivery networks (CDNs), and AI applications [20] are stated as data-intensive and need to handle data in each phase of their lifecycle [71] [72] [67].

The DICE project, as presented in the works of Guerriero et al. and Artac et al. [51] [12] [84], provides a domain-specific model (DSM) that offers big data design. This design includes data, computation, technology frameworks, and deployment concepts to design and deploy data-intensive applications. DICE proposes a model-driven engineering approach that automatically transforms application models into IaC (infrastructure as Code). Furthermore, DICE includes quality of service requirements associated with elements within the application, which are similar to QS (Quality of Service).

Accordant [24] presented a solution for designing, deploying, and monitoring performance metrics in Big Data Analytics (BDA) applications. The proposed solution includes a domain-specific model (DSM) and DevOps practices. It defines a design process and a framework to specify architectural inputs, software components, and deployment strategies through high-level abstractions. This enables Quality of Service (QS) monitoring.

M. Gribaudo [50] proposed a modeling framework using graph-based language to evaluate the performance of lambda architecture applications. Users can define stream, batch, storage, and computation nodes with performance indices.

Bosh .J and Raj R. [89] developed a comprehensive *conceptual model for a data pipeline*. The model consists of two fundamental components: nodes and connectors. Nodes are the pipeline's primary building blocks, representing an abstract data unit that can be processed or manipulated. On the other hand, connectors are the channels through

which data is transmitted and carried between the nodes. They define how data flows through the pipeline, ensuring seamless data processing and efficient execution of tasks. The model provides a clear and concise approach to designing and implementing data pipelines, enabling businesses to optimize their data processing capabilities and enhance their overall performance.

Erraissi A. and Belangour A. [43] proposed a *meta-model* that caters to specific layers of the data life cycle. This model encompasses layers such as *the data source layer*, *ingestion layer*, and *visualization layer*.

Nesi P. [81] proposed a *solution consisting of tools* designed to collect and save data from IoT and Smart City devices in real-time. This data will be used to monitor the performance of individual devices and services, detect any issues, and keep track of data traffic.

Wei D. [121] reviewed IoT dataflow management, analyzing its challenges and techniques such as data sensing, mining, control, security, and privacy. He also compared different tools and platforms for IoT dataflow management. Additionally, They explained potential application scenarios for IoT dataflow management, such as smart cities, transportation, and manufacturing.

Kalipe G. [62] examined the major existing *architectures for Big Data* (Lambda, Kappa, Zeta, ...). And explored the benefits and drawbacks of each architecture, as well as their hardware requirements, open source and proprietary software needs, and examples of real-world applications in various industries. Specifically, he identified common problems within each domain that can be addressed by leveraging the respective architecture. Lastly, he concluded by presenting a comparison of the different architectures, highlighting their trade-offs. Gillet A. [49] has developed the *Lambda+ Architecture*, an enhanced version of the Lambda Architecture, that enables both real-time and exploratory data analysis. She applied category theory to the investigation to ensure the preservation of component properties when combined.

1.5 Limitations

Many studies have discovered that practitioners face difficulties in managing data, as evidenced by research such as [88], [87], and [10]. However, these studies have primarily focused on identifying challenges rather than providing solutions. Several studies have not fully addressed the need for a *holistic framework*; they cover specific stages of the data life cycle, such as ingestion [43]. Architectures based on instruments and technologies are proposed to monitor the data flow for smart cities using IoT technology [81] [19]. On the other hand, significant studies have made valuable contributions to data management. These studies have provided conceptual models for data pipelines [89] and insights into the areas of DataOps [80]. Our goal is to improve how data is managed by offering a comprehensive overview of abstract *DA*. This methodology facilitates the creation and upkeep of high-quality data products designed for data-intensive applications.

Data-intensive applications are software systems designed to handle large volumes of data efficiently. These applications are focused on managing and manipulating data to ensure scalability, especially with constantly increasing data loads. They include big data analytics platforms, real-time data processing systems, and data warehousing solutions. Even though creating top-notch data-intensive systems is still crucial, companies specializing in embedded systems now incorporate AI components and data products alongside their electronics and software. Improving data management practices is a critical need for businesses across diverse industries. It is an urgent issue that demands prompt attention. The primary objective of this thesis is to enhance the quality of data-intensive applications utilizing *Model Driven Engineering (MDE)*. This will be accomplished by creating models for *DA* and simplifying the process of producing quality checks.

1.6 Contributions

This thesis presents a multitude of valuable contributions. First, It provides valuable insights into the *DA* challenges that are prevalent across various industry domains with this comprehensive overview. Second, it delves into the ability of data-intensive systems

to adapt and cope with the evolving significance of data, as well as the transformations in *DA* practices that occur over time. Third, it develops a meta-model for *DA* suitable for different real cases like data analytical architecture, operation data warehouse, and data pipeline. Fourth, the graphical modeling of *DA* is based on a meta-model. Fifth, it simplifies data monitoring by utilizing the model to generate *Data Quality (DQ)* check code.

1.7 Structure of the chapters

This thesis is organized as follows.

Chapter 1 introduces the topic of data architecture for data-intensive systems and presents the goals of this Ph.D. research, the research questions, and the key contributions.

Chapter 2 presents the background of the study. Chapter 3 discusses the research methodology, questions, and the motivation for using each study method.

Chapters 4, 5, 6, 7 and 8 are based on the publications and constitute the key contributions of this thesis. Chapter 4 proposes the Architecture Description Framework For Data-Intensive Applications. Chapter 5 presents the DAT, a graphical modeling tool for architecting data-intensive applications. Chapter 6 presents DAT as a modeling tool for overcoming challenges in data analytics architecture. Chapter 7 discusses data quality throughout the data journey. Chapter 8 proposes PyDAQu as a framework for generating *DQ* codes that make it easy to monitor *DQ*.

In summary, Chapter 8 presents an overview of the primary research findings and offers recommendations for future investigations.

Chapter 2

Background

In this thesis, *DA* for data-intensive applications is studied. To help readers better understand the rest of the thesis, this section offers background information and describes related work.

In Section 2.1, various data applications are discussed, allowing the reader to gain insight into the impact of data on different industries. 3Vs of big data are discussed in section 2.2, which helps understand why studying data architecture challenges is essential. IEEE/ISO/IEE 42010 Architecture Description is discussed in section 2.3, We utilized it as a conceptual foundation for constructing our metamodel. Section 2.4 discusses various big data architectures. *DQ* is discussed in section 2.5. Finally, section 2.6 summarizes the chapter.

2.1 Data-intensive Applications

Data-intensive applications are advanced software systems that effectively handle large amounts of data. These applications are specifically designed to manage and manipulate data, primarily focusing on storing and processing significant volumes of data [26]. They can easily adapt to accommodate ever-increasing data loads and often rely on distributed computing, parallel processing, and other techniques to efficiently handle high data volume. Notable examples of data-intensive applications include big data analytics

platforms, real-time data processing systems, data warehousing solutions, and content delivery networks (CDNs). Additionally, Creating AI applications in real-time environments poses unique challenges and requires a distinct development approach compared to traditional software engineering [20].

In today's world, data has become ubiquitous, and its significance has grown exponentially. The increasing usage of data has profoundly impacted our society, affecting various aspects of our lives in numerous ways. From businesses to governments, healthcare to entertainment, data is utilized in countless domains to streamline processes, make informed decisions, and enhance overall performance. The vast potential of data has made it a valuable resource that is being leveraged extensively, and its importance is only set to increase in the future [61].

2.2 4Vs of Big Data

Big data is characterized by four Vs: Volume, Velocity, Variety, and Veracity. Volume relates to the quantity of data, variety pertains to the diversity of data types, and velocity refers to the speed at which data is processed [112].

Volume: Volume is a crucial aspect of the 4 Vs framework, which organizations use to determine the size of the big data they store and manage. It refers to vast amounts of data often expressed in unfamiliar numerical terms. Human interactions with machines, networks on social media, sensors, and mobile devices generate this data.

Velocity: Velocity is a crucial aspect of the 4 Vs framework, which determines the rate at which big data volume grows and how easily it can be accessed. Another term for velocity is "data in motion". For example, approximately 200 million emails are sent every minute, 300000 tweets are posted, and 100 hours of YouTube videos are uploaded. This rapid increase in velocity poses new obstacles when collecting big data consistently and comprehensively.

Variety: The 4 Vs framework uses variety to classify the data types and categories in a big data repository, along with their associated management. Data is collected from various sources, such as sensors, social media, and wireless networks, in various formats,

including website logs, astronomical data, and medical records. Extracting insights from such diverse data requires powerful processing capabilities and accurate algorithms. As technology advances and users find new ways to collect and process data, the amount of data available continues to grow, presenting challenges for effective big data management.

Veracity: is a paramount concern concerning the data’s precision, dependability, and credibility under analysis. It is imperative to have a high level of confidence in the accuracy and integrity of the data, given that it is often derived from diverse sources such as sensors, social media, weblogs, and other sources. Therefore, the significance of data integrity cannot be overstated in today’s data-intensive applications.

2.3 IEEE/ISO/IEE 42010:2022 Architecture Description

In this paper, we build upon the conceptual foundations of the ISO/IEC/ IEEE 42010:2022, Systems and software engineering — *Architecture description (AD)* [58] standard, to investigate the essential elements of *AD* for Data-Intensive applications. The standard addresses *AD*, the practices of recording software, system, and enterprise architectures so that architectures can be understood, documented, analyzed, and realized. Architecture descriptions take on many forms, from informal to rigorously specified models. The content model for an *AD* is illustrated in Figure 2.1. The architecture viewpoint is a fundamental building block representing common ways of expressing recurring architectural concerns reusable across projects and organizations. It encapsulates model kinds framing particular concerns for a specific audience of system stakeholders. The concerns determine what the model kinds must be able to express: e.g., security, reliability, cost, etc. A model determines the notations, conventions, methods, and techniques. Viewpoints, defining the contents of each architecture view, are built up from one or more model kinds and correspondence rules, linking them together to maintain consistency.

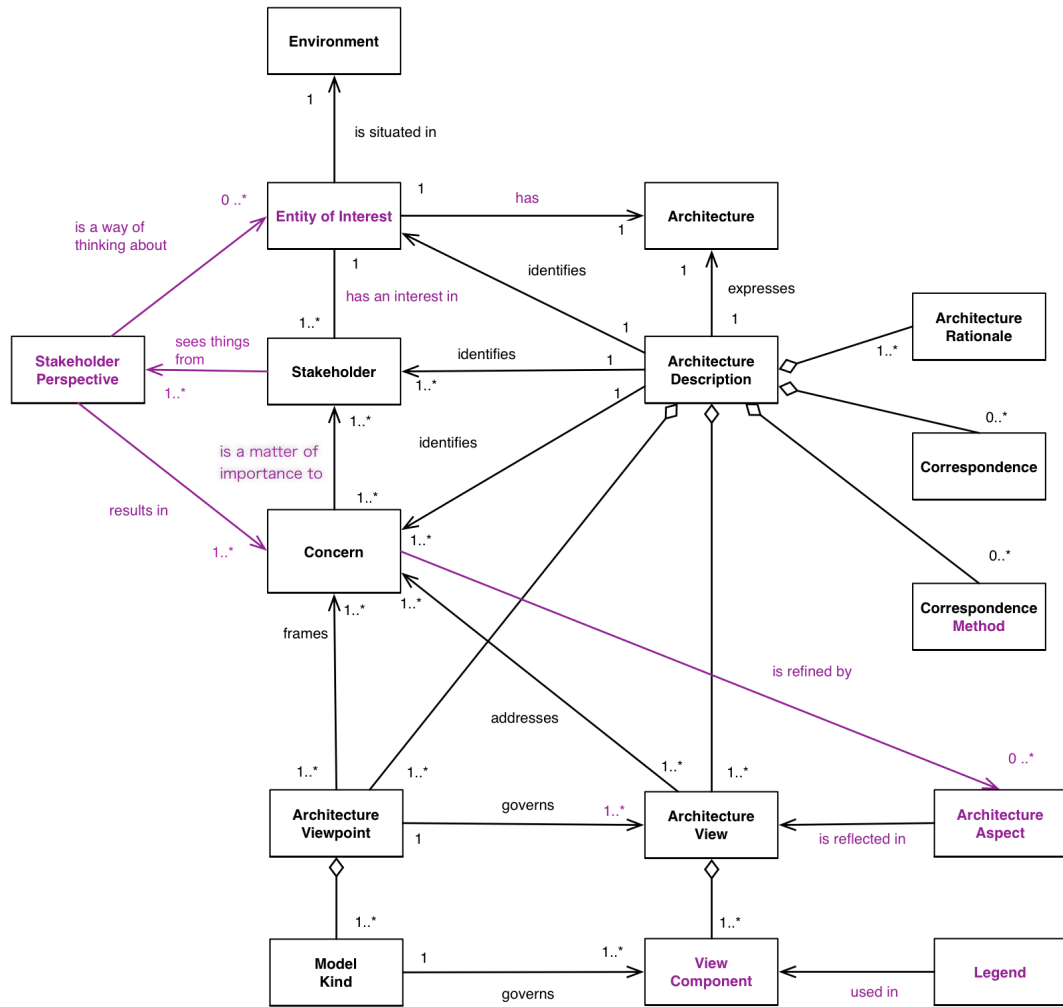


Figure 2.1: Content model of an AD (following ISO/IEC/IEEE 42010:2022)

2.4 Big Data Architectures

Design patterns called *Big Data Architecture (BDA)*s are utilized to handle, store, and manage massive amounts of data in data-intensive applications. Within this realm, two important architectures are Lambda Architecture and Kappa Architecture.

The **Lambda Architecture** has three layers: Batch, Speed, and Serving. Batch processes large-scale data offline for accurate analysis, while Speed handles real-time processing for up-to-date insights. Serving combines results from both layers for interactive

queries, accommodating eventual consistency between batch and real-time views [49] [66].

The **Kappa Architecture** simplifies real-time data processing using a single pipeline for real-time and batch data. It leverages stream processing frameworks like Apache Kafka Streams or Apache Flink, treating all data as a continuous stream. This allows for low-latency processing and analysis but may require additional historical analysis and reprocessing mechanisms [83].

Both architectures have benefits and drawbacks, and the decision of which to choose depends on various factors, such as the type of data, the desired speed, and the tools and resources available. In addition, different architectures like the Zeta Architecture and the Mu Architecture have also been suggested to meet diverse data processing requirements in data-heavy applications.

2.5 Data Quality

DQ is defined as fitness for use [99]. *DQ* should be understood as the degree of data's correctness and usefulness [70]. *DQ* cannot be overstated when making informed decisions based on data. The accuracy and reliability of the insights and decisions derived from data are directly linked to the quality of the data itself. It is essential to ensure the data is accurate, up-to-date, and consistent to avoid any potential mistakes or errors in decision-making. By prioritizing data quality, organizations can ensure that they make the best use of their data and derive the most valuable insights possible [65] [60]. It is imperative for organizations to maintain high-quality data, as poor data quality can have severe ramifications. Inaccurate conclusions, incorrect decision-making, and adverse outcomes are all potential consequences of poor data quality. Therefore, it is crucial to ensure that data is accurate, up-to-date, and reliable to avoid any negative impact on the organization [22].

DQ refers to the reliability, accuracy, completeness, consistency, and overall usability of data for a given purpose. Several dimensions are used to assess data quality, and the six most commonly used data quality dimensions [99] [59] [108]:

1. **Accuracy** pertains to the correctness and precision of information. The correct

information is devoid of any errors, discrepancies, or mistakes.

2. **Completeness** The concept of completeness involves ensuring that all the necessary data elements are present. Incomplete data can lead to skewed analyses and a lack of insights. It is crucial to have data that encompasses all relevant aspects to prevent incomplete conclusions from being drawn.
3. **Consistency** Assessing consistency is key when dealing with data from various sources over a certain time frame. Inconsistent data can lead to misinterpretations and confusion, so it's imperative to guarantee data consistency to ensure precise analyses.
4. **Timeliness** refers to the freshness of data about its intended use. Using outdated data can result in uninformed decisions, but having timely data ensures that insights and actions are based on current and accurate information.
5. **Validity** refers to whether the information complies with established business rules, standards, and limitations. Information that does not meet the requirements can result in erroneous analyses. Validating data guarantees that it is consistent with the intended objective.
6. **Uniqueness** Ensuring the uniqueness of data records is crucial to avoid duplication and maintain analysis accuracy. Duplicates can cause misleading conclusions, leading to confusion and redundancy. Therefore, it's essential to maintain data uniqueness to prevent such errors.

DQ dimensions are essential for organizations that gather, store, and use data. These qualities are critical in assuring data precision, comprehensiveness, dependability, and consistency. Organizations should evaluate their data against external sources and use automated methods to detect inaccuracies to do this. Organizations may confidently manage their data's correctness, completeness, consistency, and dependability in this manner.

2.6 Summary

This chapter presents five main concepts discussed in this thesis, Data-intensive Applications, 4Vs of Big Data, IEEE 42010 Architecture Description *AD*, Big Data Architectures *BDA*, and Data Quality *DQ*.

Chapter 3

Research Methodology

The research methodology is a systematic approach to resolving research problems. The focus of this study is to emphasize the significance of enhancing data architecture practices in data-intensive systems and its impact on the quality of data. Previous research has highlighted professionals' challenges in creating and maintaining data products. Therefore, the primary objective of this study is to establish a data architecture model that can efficiently manage data and produce high-quality data products. This chapter is divided into several crucial parts, which include research design selection, research questions, motivation, informant selection, data collection methods, data analysis, an overview of the research process, and potential threats to validity.

3.1 Research Questions

This research aims to empirically identify the data architecture challenges encountered during the development and maintenance of data-intensive systems, propose an improved data architecture approach, and empirically validate the proposed approach. We have adopted a qualitative research methodology to address the research goals. The research is focused on the five primary research questions.

- **RQ1: What data architecture challenges are experienced in the industry while developing data-intensive systems?**

- **RQ2:** What essential components are needed for data-intensive applications to model data-intensive systems?
- **RQ3:** How can a comprehensive model, which includes main concerns in various data-intensive related domains, be designed?
- **RQ4:** How can the framework be used in practice in software design?
- **RQ5:** How can monitoring data quality be streamlined?

The main aim of RQ1 is to identify the challenges in data architecture that professionals face due to current advancements. Data architecture is a topic under exploration, and a few papers discuss various data architecture practices. Therefore, conducting a thorough study showcasing the challenges experts face in light of technological progress is imperative. This study would effectively establish the urgency for an enhanced data architecture practice. The second research question (RQ2) was set to analyze a list of essential data components practitioners could use to address data architecture challenges in data-intensive systems. The third question (RQ3) explicitly demonstrates that the main concepts and the list of analyzed challenges are firmly rooted in a holistic framework. The fourth question (RQ4) effectively demonstrates the framework's ability to deliver architectural solutions tailored to the demands of various industrial cases. The primary objective of the fifth research question (RQ5) is to establish the impact of improved data architecture practices on the quality of data products. It delves into how simplifying data quality can facilitate automation, ultimately leading to the speedy development and maintenance of top-notch data-intensive systems.

3.2 Qualitative Research

Qualitative research aims to gather, evaluate, and clarify non-numerical data like written words, recorded sounds, and filmed images to comprehend ideas, viewpoints, or personal encounters. It is crucial to understand that qualitative research centers on individuals constructing meaning through their interactions with the world around them [75]. In

qualitative research, results are presented using words instead of numbers. This type of research focuses on themes rather than numerical data [69]. We have strategically adopted this methodology as it not only allows us to effectively construct new and innovative ideas for improving or fine-tuning a product or practice but also empowers us to construct a comprehensive theoretical framework that emerges from the data gathered during the research and logically explains the results.

Qualitative data sources encompass observation, participant observation (fieldwork), interviews, questionnaires, documents, texts, and the researcher’s personal impressions and reactions. Qualitative research provides more comprehensive information, deeper insights into real-time practices, and a better understanding of underlying perceptions [18]. In this type of research, the primary techniques used are individual interviews, group interviews (also known as focus groups), observations, and document analysis [96].

For our study, we opted for individual interviews, group interviews, and observations as they can provide comprehensive information about a limited number of cases. This approach enabled us to gain deeper insights into each case. Our objective was to examine how companies utilize certain practices, identify issues they are facing, and suggest better solutions. As such, using a qualitative methodology was appropriate for our study.

3.3 Case Companies

In our research, we carefully chose companies with data-intensive systems demonstrating a high domain maturity level in adopting data architecture. For confidentiality reasons, the companies involved in the study have chosen to remain anonymous. The research highlights limitations, errors, and current development practices in technology. We have included a concise summary of the case companies examined in this study. The companies allowed collaboration through interviews, in-company workshops, action research, and weekly meetings.

Cases A, B, C, D, and E, actively participated in this research by allowing collaboration through interviews, workshops, interactive sessions, weekly meetings, and action research.

Case A and B, The company offers a next-generation frontline employee experience platform built for companies that have served at the heart of their business. The Company suite of talent attraction, workforce management, employee engagement, and compliance technologies enable organizations to intelligently attract, manage, engage, and retain the best talent to run and improve their business. It serves over 20,000 restaurant and hotel locations and four million hospitality employees globally, with emerging growth in retail and healthcare.

Case C, The company is a multinational information technology corporation with a notable presence in several technology industries. One of its main focuses is on Printers and Imaging Solutions, where it is recognized as a major player in the printing industry. It offers a range of inkjet, laser, and multifunction printers catering to home and office use. The context of this case involves managing error data that arises from these printers.

Case D, The company is An Internet Service Provider (ISP) telecommunications company that enables individuals and businesses to connect to the Internet. ISPs offer various technologies like broadband, wireless, and satellite connections to facilitate data transmission between users and the global network of servers.

Case E, This research project examines the conversations happening in the health and food fields. Analyzing data from social networks aims to detect any emerging trends or issues as they happen. Although this project is challenging, it has the potential to provide valuable insights that could benefit these important aspects of human life. Table 3.1 shows more details about the cases and complexity (The "complexity" column provides information on each case's complexity level concerning the primary data components. These components are typically measured regarding the number of internal data elements that comprise a given case.).

3.4 Research Methods

Research methods play a crucial role in collecting and analyzing empirical data. They are an essential component of any research design. Qualitative research methods offer a distinct advantage as they compel the researcher to explore the intricacies of the prob-

Table 3.1: Cases Details

Company	Cases	Case Name	Case Compixity	Experts Roles
1	A	Analytical Data Architecture	4 (48)	Big Data Team Lead
				Data Architect
				Big Data Engineer
	B	Operational Data Warehouse	9 (72)	Big Data Team Lead
				Data Architect
				Big Data Engineer
2	C	Data Pipeline	5 (34)	Big Data Team Lead
				Data Architect
				Big Data Engineer
				ETL Developer
				Data Quality Engineer
3	D	Operational Data Architecture	9 (66)	Big Data Team Lead
				Data Architect
				Data Quality Team Lead
4	E	Lambda Architecture	7 (67)	Researchers

lem rather than simplifying it [96]. Empirical data is gathered through the meticulous observation and documentation of patterns and behavior, typically via experimentation. Such data is invaluable in answering research questions, as it provides a solid foundation of information based on sensory evidence [96]. Qualitative and quantitative methods can be used to collect and analyze data [105]. Collecting qualitative data involves producing a substantial volume of data [102]. After transcription, the audio or video recording data collection method is employed for data analysis [105].

3.4.1 Case Study

A case study is a detailed examination of a specific occurrence or a few instances of a phenomenon [52]. The purpose of a case study is to explore contemporary phenomena in their real-life setting, particularly when it is unclear where the boundary between the phenomenon and the context lies [85]. The advantages of using the case study method include the ability to collect and analyze data within the context of the phenomenon, combine qualitative and quantitative data during the analysis, and gain a deeper understanding of the complexities of real-life situations. However, the disadvantages of case

studies include a lack of strict methodology, difficulties associated with analyzing data, and limited ability to generalize findings and conclusions [85]. Different research strategies are categorized based on purpose, including exploratory, descriptive, explanatory, and improvement. Case study is a research method initially used mainly for exploratory purposes, but it can also serve descriptive, improvement, and explanatory purposes [93]. It is an ideal choice for exploratory research questions. However, for descriptive research questions, a case study may be feasible only if the sampling-based study's representativeness can be sacrificed for better realism in a case study. If representativeness is crucial, a survey is a better option. When addressing explanatory research questions, case studies can be helpful. However, the evidence collected is not based on a statistically significant quantitative analysis of a representative sample. Instead, it provides a qualitative understanding of how phenomena operate in their specific context. If quantitative evidence is crucial, an experimental strategy is a better option. For improving research purposes, the action research strategy is a natural choice and a variant of case study research [93]. In our case, we adopted an exploratory case study to identify the challenges associated with data architecture practices in a real-world company context. This study captured the complexities of data architecture in a data-intensive system company scenario.

3.4.2 Action Research

In software engineering, action research is a collaborative approach that involves working with concerned actors or problem owners within an organization. This method allows for the real-time proposal, implementation, and evaluation of solutions, with researchers engaging with the company over an extended period. Problem owners are crucial in action research, providing valuable insights and expertise based on their experiences and domain knowledge [86] [74]. The primary goal of this approach is to address real-world issues while exploring the outcomes and learning experiences of problem-solving [35]. For our study, we chose to use the action research method as it allowed us to systematically identify and define issues related to data architecture practices and propose solutions in a participatory manner within the context of our investigation. Additionally, we

participated in the subsequent phases of implementing the solution, known as action [86] [74]. The cycle of the action research process comprises five stages, namely (1) diagnosis, (2) action planning and design, (3) taking action, (4) evaluation, and (5) specifying learning [86] [74]. Action research is beneficial as it can provide sturdy and applicable knowledge to various management and organization researchers [28].

3.5 Research Techniques

Researchers utilize research techniques to gather empirical data to analyze actions in real-world industrial settings. These techniques include semi-structured interviews and literature reviews, which are particularly useful for understanding behavior, the meanings and contexts of events, and the influence of values on decision-making in practical situations [104].

3.5.1 Interviews

Interview-based data collection involves asking subjects questions about the case study's areas of interest. The researcher uses interview questions based on the research question to guide the dialogue with the subject(s). Interviews can be conducted with a group or individual practitioners and can be open-ended or closed. Interviews can be unstructured, semi-structured, or fully structured [78].

Interviews can be structured, semi-structured, or unstructured. Structured interviews have pre-planned questions in a fixed order, similar to a survey. Unstructured interviews are more conversational and based on the researcher's interests and subject. Semi-structured interviews have planned questions but can be asked in any order. We chose semi-structured interviews because they allow for more exploration and clarification of responses and create a rapport between respondents and researchers [15].

3.5.2 Observation

Observation involves carefully observing and examining participants' behavior in a natural setting [30]. This can be used to study how practitioners perform a specific task.

There are various observation methods, such as monitoring a group of practitioners with a video recorder and analyzing the recording later using protocol analysis [82][118]. An additional option is to utilize a "think aloud" protocol during research by frequently asking subjects questions like "What is your strategy?" and "What are you thinking?" to prompt them to vocalize their thoughts. Meetings can also generate information about the studied topic through attendees' interactions. Another approach involves using a sampling tool to gather participant feedback and data [64]. Researchers at a research site must observe and take detailed notes on the people, concepts they discuss, and occurring interactions [30]. These notes, called field notes, are an important part of the research process. During the action research, the researchers performed participant observation and took field notes. Observation can either be structured or unstructured. Structured observation involves collecting data using specific variables and a pre-defined schedule. Unstructured observation, on the other hand, is more open-ended and has no pre-determined variables or objectives [93]. This research used unstructured observation mainly during weekly stand-up meetings, pair programming, and weekly result presentations.

3.6 Data Analysis

Qualitative research involves collecting unstructured text data from various sources, including interview transcripts, observation notes, diary entries, and medical records. In addition, multimedia materials such as pictures, audio, or video recordings may also be included in the data. Given the diverse nature of the data, analysis methods must be flexible, intuitive, and imaginative, involving inductive reasoning, critical thinking, and theorizing.

3.6.1 Qualitative Data Analysis

In qualitative research, data analysis involves systematically searching and arranging interview transcripts, observation notes, and other non-textual materials to understand a phenomenon better [122]. The process primarily involves coding or categorizing the

data. Coding involves breaking down a large amount of raw information and assigning it to categories. Two types of coding used in this thesis are thematic coding using the NVivo tool and open coding [34]. Thematic coding involves finding text themes by analyzing words' meanings and sentence structure. The NVivo software automates the data [31]. Open coding is a qualitative data coding method that creates codes from scratch. The coder manually creates these codes to cover the entire transcript. The codes are then applied to the remaining transcripts, with necessary adjustments made to ensure that the codes apply to all transcripts in the study [94].

3.7 Research Design

The initial and most important research phase involved identifying an idea to investigate. The research idea originated from participating with one of the other team members to develop a data-intensive application.

Our primary intention was to empirically identify the data architecture challenges encountered during the development and maintenance of data-intensive systems, propose an improved data architecture approach, and empirically validate the proposed approach. Our approach to formulating RQs involves problem identification, analyzing existing approaches, identifying challenges, proposing solutions, and validating them. We chose qualitative research to gain insight into practitioners' perspectives on the current data architecture approach and their willingness to transition to a new one. Additionally, it allows us to identify practitioners' needs, the difficulties they face with current approaches, and how these difficulties impact the final product. Additionally, utilizing qualitative analysis can assist in generating ideas for enhancing the data architecture approach. While quantitative research methodology can measure behaviors and answer questions such as "how often" and "how many," our objective is to delve into data architecture challenges, current practices, and their evolution. Gathering data from informants is the most effective technique for comprehending embedded system industries' challenges. In quantitative methodology, free-text responses are not allowed, potentially missing vital contextual details. Thus, we opted for a qualitative research methodology

for our study.

- **Case Study 1: Problem Identification**

A literature review was conducted to learn more about the topic under investigation, which enabled the researcher to identify similar studies conducted in the past. We identified papers on data architecture concepts and challenges through the literature review. However, all the previous studies were focused on data pipelines, big data architectures (Lambda, Kappa, and ...), real-time and batch processing, and architecture-based technologies. Therefore, the researcher decided to study data architecture challenges for data-intensive applications. To explore and identify the challenges encountered by practitioners during the development and maintenance of data products, a case study method was used as it allows for the examination of the phenomenon in depth using various kinds of evidence obtained from interviews with those involved, direct observation of events and analysis of documents and artifacts. Data collection primarily relied on interviews, as illustrated in Table 3.2. We utilized an exploratory case study method and conducted 6 in-depth interviews with practitioners for two different cases from Company 1, to collect comprehensive information. The selected informants also provided relevant documents to aid in understanding their use case descriptions and data formats, among others. The interviews were semi-structured, with open-ended questions, which provided a great opportunity to delve deeper into the issues. All interviews were recorded and transcribed with the practitioners' permission. We also made observations and analyzed the documents provided. The challenges were categorized, and the results were sent to the practitioners for review, with data analysis conducted using the NVivo tool for thematic coding. The codes were analyzed, and the results were formulated with confidence.

Participant reflections were meticulously recorded, and the researcher conducted follow-up interviews with three senior practitioners to clarify any uncertainties. The final results were then sent to two esteemed senior practitioners for review, and modifications were seamlessly made according to their invaluable feedback. The

published paper is a testament to our research’s thoroughness and professionalism.

- **Case Study 2: Analysing the existing Practices**

We took the next step in our research by investigating the challenges of data architecture. To do this, we conducted a study that involved interviewing multiple teams from companies A, B, C, and D. We looked at 5 different use cases and conducted 6 in-depth semi-structured interviews. The researcher was granted permission to participate in the weekly stand-up calls and meetings. This allowed us to gain insight and contribute significantly to the study. We presented the results to the practitioners involved in the study, as well as the company’s steering committee, for approval. We also incorporated the reflections and comments from the informants into the final results, which were published as a paper.

- **Case Study 3: Propose a Solution**

We created a data architecture strategy for building high-quality systems that rely heavily on data. Our meta-model consisted of structural and behavioral components informed by previous research. We conducted extensive interviews with individuals and focus groups to understand better the current data architecture at one of our case companies.

In addition, the researcher gathered information by observing various meetings inside and outside the company. They took notes during these meetings and also conducted interviews to gain insights. Using this information, we created a meta-model for data architecture. To ensure the model’s accuracy, we presented it to teams involved in the study for feedback. They then presented it again to teams working on data products not involved in the study. The model was also presented at two other companies for external validation, and feedback was incorporated. The meta-model was eventually published as a paper and presented at several workshops. Table 3.2 provides a comprehensive overview of the empirical endeavors regarding exploratory case study research. Using our meta-model, we developed DAT, a graphical modeling tool specifically designed for data architecture.

- **Action Research: Implement and Validate the Solution**

The study aimed to establish a robust data architecture using MDE. It was conducted across several companies, including B, C, and D, and the findings were thoroughly examined. Company B, specializing in technology solutions such as printers and imaging, served as a practical test-bed for framework implementation.

A literature review was conducted to dive into the data-intensive systems. The review helped to identify diverse data architectures utilized across various industries. The study aimed to enhance the quality and structure of data via pragmatic research and analysis, which was carried out with utmost confidence in its potential to improve the field.

The researcher, a Team Lead, and a QA Engineer analyzed cases from companies B and C and developed a framework to enhance the quality of the existing data pipelines. During the development of the framework, it was realized that there was a need to improve the process for monitoring data quality. To address this issue, it was suggested to generate automatic data quality codes using the widely used Python library, "Great Expectation".

Table 3.2: Overview of Exploratory Case Studies

Company	Cases	Case Name	Experts Roles	Research Objective	Research Technique	Data Collection Technique
1	A	Analytical Data Architecture	Big Data Team Lead	1. To identify the DA Challenges 2. To develop a meta-model 3. To develop an Architecture tool	1.Interview 2.Observation	1. Semi-structured interviews 2. Observation from the weekly meeting 3. Follow-up interviews
			Data Architect			
	B	Operational Data Warehouse	Big Data Engineer			
			Big Data Team Lead			
2	C	Data Pipeline	Data Architect	1. To identify the existing data pipeline 2. To identify data quality challenges 3. To Identify data quality approaches	1.Interview 2.Observation	1. Semi-structured interviews 2. Observation from the weekly meeting 3. Focus group interview 4. follow-up interview
			Big Data Team Lead			
			Big Data Engineer			
			ETL Developer			
3	D	Operational Data Architecture	Data Quality Engineer	To Develop a Data Quality Tool	1.Interview 2.Observation	1. Semi-structured interviews 2. Observation from the weekly meeting 3. Follow-up interviews
			Big Data Team Lead			
			Data Architect			
			Data Quality Team Lead			
4	E	Lambda Architecture	Analytics System Architect	To identify big data architectures	Interview	Semi-structured interviews
			Researchers			

3.8 Threats to Validity

This section discusses threats to validity regarding how our research questions were answered.

3.8.1 Construct Validity

Construct Validity comprises two integral components: the measure must be all-encompassing and discriminatory in that it solely encompasses aspects of the intended theoretical construct. A few cases were excluded from the results due to interviewers not fully understanding discussed concepts to maintain construct validity. Our study has some limitations due to the screening process and a limited number of interviews. However, this limitation can be considered an opportunity for future inquiry. To minimize researcher bias, the researcher conducted the interviews. Before conducting the interviews, we sent the interviewees a semi-structured interview guide and a brief description of the topic to explore.

During the interview, we started by introducing the topic of the study. We rephrased questions if the interviewee's response seemed off-topic or unclear. Additionally, during our analysis of the interview transcripts, we contacted interviewees to clarify any confusion or lack of clarity.

3.8.2 Internal Validity

Internal validity refers to how accurately the observed outcome reflects the truth in the studied population without being affected by methodological errors [86]. It is important to note that a potential threat to the validity of this thesis could arise due to the fact that the strategies and results of RQ2 and RQ3 were developed within a specific company context. The researcher only had limited access to the strategy descriptions, which makes it impossible to determine if other factors had a more significant impact on the final result than the proposed strategies. The co-author validated the findings with in-depth knowledge of company data to enhance internal validity. The respective company's steering committee also reviewed the results.

3.8.3 External Validity

The presented work is derived from the cases studied with different teams in the domains of telecommunication, software development, and information technology. Some parts of the work can be seen differently in different parts of the company. All the terminologies used in the companies are normalized, and the implementation details are explained with the necessary level of abstraction [73]. We do not claim that the opportunities and challenges will be the same for industries from different disciplines.

3.9 Summary

This chapter delves into the research questions that were the focus of this thesis, providing a detailed examination of each. The research methods used to tackle these questions are discussed comprehensively, particularly their effectiveness. The research design section further elaborates on how these methods successfully addressed each research question.

Chapter 4

Architecture Description Framework For Data-Intensive Applications

Managing data has become increasingly difficult due to its exponential growth and multiple sources. In the business world, *DA* provides a systematic approach to describing, collecting, storing, processing, and analyzing data to meet business needs. It plays a crucial role in transforming data into valuable information by providing an abstract view of data-intensive applications. A framework for data-intensive applications is presented in this article, which utilizes *MDE* to achieve this goal.

The framework's effectiveness was evaluated through five thorough case studies, with valuable feedback provided by seven practitioners and two prominent companies on its capabilities.

4.1 Introduction

According to the International Data Corporation (IDC) [29], the amount of useful data is predicted to exceed 175 zettabytes by 2025, with a compounded annual growth rate of 61%. About 90 zettabytes of this data will come from IoT devices, and roughly 30% of the generated data will be consumed in real-time. Additionally, digital transformation programs are estimated to cost around \$7.4 trillion between 2020 and 2023.

Data is the backbone that plays a decisive role in various aspects such as decision-making, machine learning, generating insights, and creating reports [17]. To become data-intensive, it is essential to follow a strategic process involving effectively utilizing data insights. Therefore, adopting the most efficient methods for collecting, storing, analyzing, and safeguarding valuable data is crucial.

A well-designed data architecture *DA* is crucial for any organization to manage and utilize its data assets effectively. Providing a comprehensive set of specification artifacts enables businesses to align their data investments with their overall strategy and seamlessly integrate different data sources. With a series of master blueprints at various levels of abstraction [57], *DA* empowers organizations to take a top-level view of their data collection, storage, analysis, and presentation, unlocking endless possibilities for growth and success [56].

We have discovered an urgent demand for a comprehensive blueprint to guide data-intensive projects through our discussions with researchers and practitioners across various industry domains. While recent research has offered meta-models and conceptual models for specific layers of these projects, such as data sources and ingestion, as well as technology-based architecture, these contributions are limited in scope. Our findings confirm the importance of a more holistic approach to data-intensive projects; this study addresses this need.

We developed an innovative architectural description to assist in creating efficient data-intensive applications. Built-in accordance with the IEEE/ISO/IEC 42010 standard [58], this framework provides comprehensive support for every stage of the architecture description, reasoning, and design decision process. Our research is proud to offer a modeling and analysis framework that will help facilitate an architecture-driven development of data-intensive applications.

The framework is called *DAF*¹ [5]; it is based on an architectural approach to describe a data flow for data-intensive Applications. The *DAF* framework is realized by exploiting the advanced Model-Driven Engineering (*MDE*) technique which is meta-modeling. *DAF* covers all data architecture components and their best practices [42] [120].

¹*Architecture Description Framework for Data-intensive Applications (DAF)*

We explored the main data architecture elements and challenges in our previous work [2] [4]. Based on the study objectives.

The questions and contributions that we address in this paper are:

- **RQ1.** What essential components are needed for data-intensive applications to model a data-intensive architecture? For this, we reviewed the relevant literature to understand the needs and potential scientific value of data-intensive (Section 4.2).
- **RQ2.** How can a comprehensive model, which includes main concerns in various data-intensive related domains, be designed? We conducted an exploratory study [2] that includes interviews with domain experts from academia and industry to answer this question. From this, we designed a well-grounded architecture description framework. The framework includes information about trends, design, and challenges, and future solutions (Sections 4.2.2).
- **RQ3.** How can the *DAF* be used in practice in software design? We used *DAF* to model real systems (Section 4.3, Section 4.4).

The rest of this paper is organized as follows. The Background is presented in Section 2, and the methodology in Section 3. The application of *DAF* to a real case study is described in Section 4. The *DAF* evaluation is presented in Section 5. Related work is discussed in Section 7, while conclusions are drawn in Section 8.

4.2 Methodology

The main focus of this paper is to provide an architecture description language *Data Modeling Language (DAML)* that describes the data architecture of data-intensive applications. Section 4.2.1 reports some literature review we used to identify the *DAML* meta-model. Section 4.2.2 reports the meta-model and related meta-classes.

Table 4.1: Summary of the DAML meta-classes and related references

Concept	Reference
Data Formats (structured, semi-structured, unstructured)	[13] [43] [77] [8] [7]
Data Processing Type (Batch, Real-Time)	[8] [77] [14] [7]
Data Storage (File System, NoSQL, NewSQL, ...)	[25] [63] [40] [115] [19] [8] [13] [7]
Data Location (Cloud, on-Premise)	[14] [77]
Data Generation (data sources(sensors, camera, API, ...))	[8] [77] [89]
Data Ingestion (identify, validate, compress, ...)	[43] [95] [19] [89] [33]
Data Processing (classify, filter, transform, ...)	[14] [19] [89]
Storing Data (retrieve, save, archive, ...)	[14] [19] [89]
Analyzing Data (describe, diagnose, predict, ...)	[90] [14] [11]
Consuming Data (API, query, visualize)	[8] [11]

4.2.1 Literature Review

This section presents an overview of the primary studies found in the literature to discover the data architecture elements. Table 4.1 shows the main meta-classes composing the *DAML* and related references.

To make informed decisions, high-quality data is essential [22]. It's crucial to ensure that data is monitored, traced, validated, and transformed from the source to its final destination. One way to manipulate data is through a data pipeline, which automates data flow from one node to another. Data pipelines consist of nodes and connections. Nodes carry out tasks such as generation, collection, processing, and storage, while connections transmit data between them [89]. Data can be structured, unstructured, or semi-structured and may require transformation after processing. Data speed and processing method (batch or real-time) are important. Storage type and location (Cloud or on-premise) must be determined for effective retrieval. Data Analytics extracts insights from stored data for informed decisions using descriptive, diagnostic, predictive, and prescriptive techniques like quantitative, qualitative, and statistical methods. To interpret data for business use, it must be presented to users for analysis. Analysts alone cannot fully interpret the data without visualization. In order to enable the meta-model to depict two tiers of architecture, namely the *high-level architecture (HLA)* and *low-level architecture (LLA)*, we must incorporate additional information into the primary

representation includes data formats, data storage technologies, location and processing type.

Our meta-model covers all **DataFormats** to describe data diversity of different data sources, **Structured** (RelationalDB), **Semi-Structured** (Email, SMS, CSV, JSON, XML etc.) and **Unstructured** (GPS data, Multi-media and Office Files). **Data ProcessingType** to describe how data is going to be processed as a **Batch** or **Real-time**. To describe where data will be stored, we use data storage technologies **DataStorageTech.** that includes **NoSQL Databases** (Document, Key-value, graph, and column), **NewSQL Databases** (Historical, Real-Time, Stream, Timestamp), and **File System** (HDF, GFS, AFS, GPFS and Blobseer). Moreover, **Location** describes the location of the data nodes on the cloud or locally.

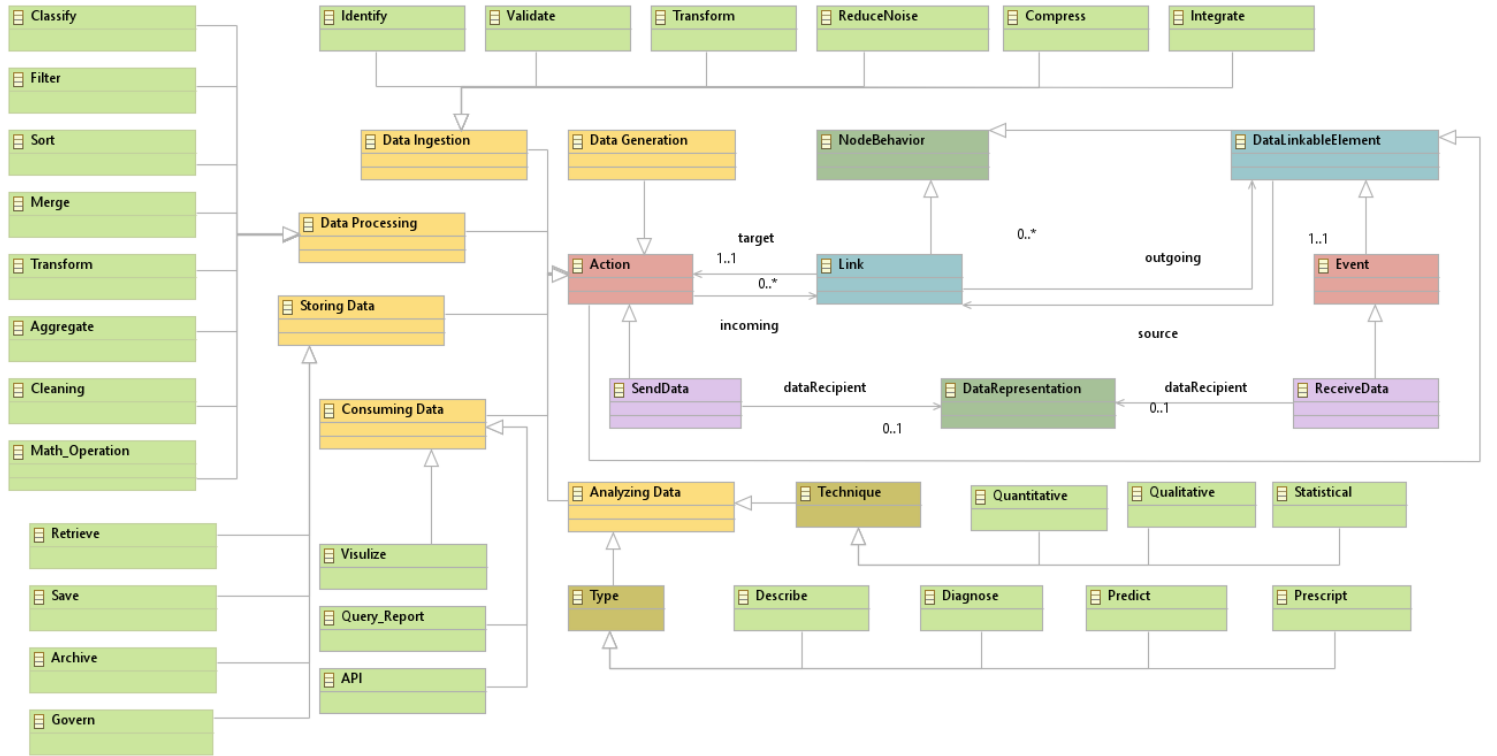


Figure 4.2: Meta-model: behavioral concepts

A **NodeBehavior** indicates the current status of the component that describes the

data in a specific data node. For example in a generation data node, we can find elements that describe the source of the data and the format. Every `NodeBehavior` has a set of behavioral elements denoted by actions, and events that all together depict the data flow within the component. Data nodes can interact by passing data through data ports (`DataPort`). For receiving incoming data input data ports (`InDataPort`) are used while output data ports (`OutDataPort`) are used for sending outgoing data. The Actual communication methods of a message are shown in Figure 4.2. In this context, a connection represents a unidirectional communication channel between two data ports of two different components. An action is considered an important behavioral element that represents an atomic task that could be performed inside the data node. This element can be executed when a previous action in the behavioral data flow has been achieved or it could be triggered by an event like `ReceiveData`. Figure 4.2 shows the main actions of data behavioural elements to describe a data behavior inside the data node. For example, `Generation` represents the source of the data. `Ingestion` describes how data can move from source to data lake. `Processes` include a list of sub-processes that could be used to describe a complete processing node. `Store` shows the main tasks to save, retrieve, archive, and govern the data. `Analyze`, is used to describe which type or technique could be used for analyzing the data. `Consume` to show how data could be consumed like visualize, reports and API. An `Event` is triggered in response to the external stimulus of the component (e.g., the data reception on an input data port). To show the data flow and connection between the events and actions , we use `links`, and we could use them to decide the order in which actions can be performed and which one must be executed directly after an event.

4.3 Application of *DAF* Model to the Case Study

This section will discuss one of the five evaluated case studies and their corresponding applications.

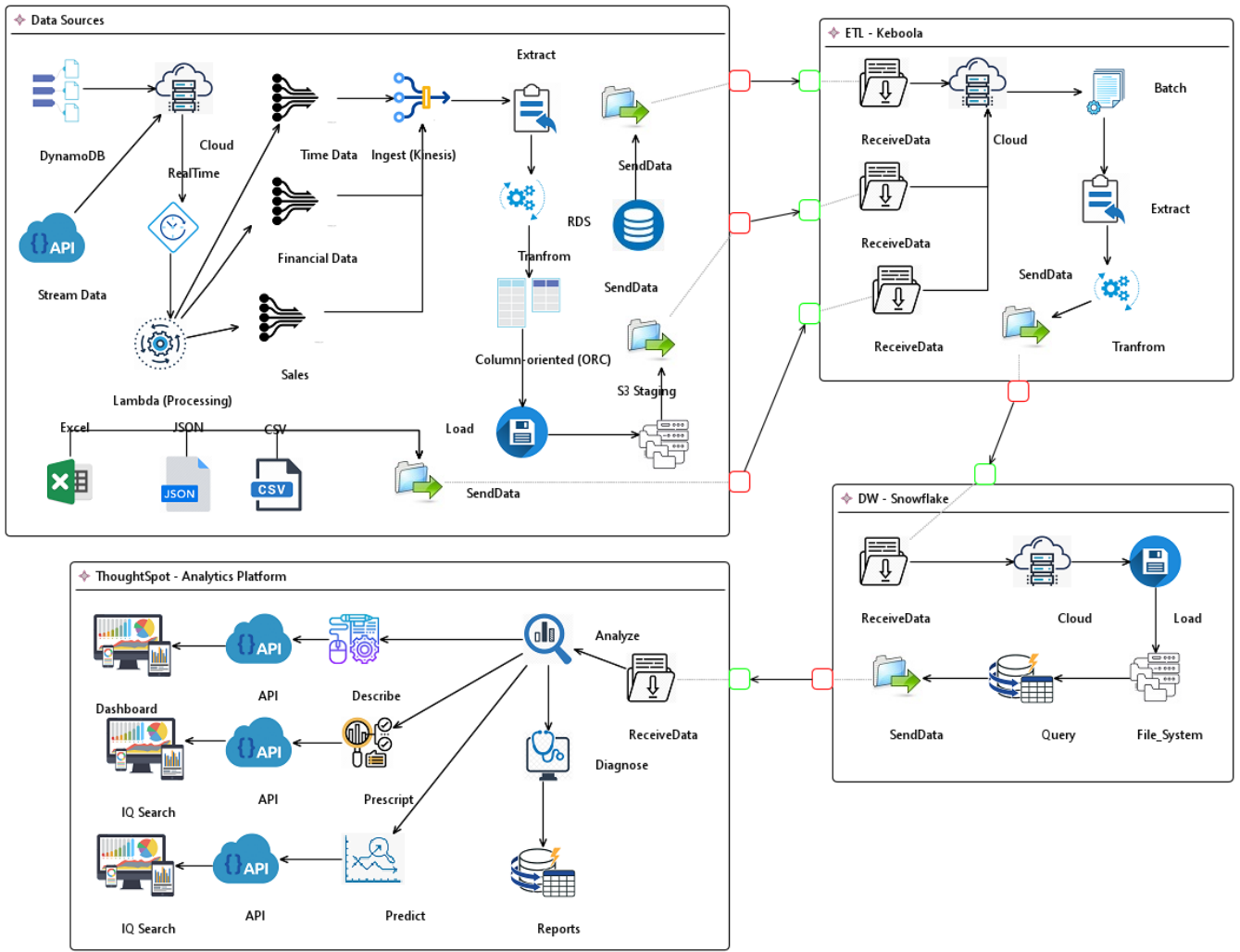


Figure 4.3: Modeling of the *Analytics Data Warehouse (ADW)*

4.3.1 Analytical Data Warehouse

The *ADW 3.3* collects data from diverse sources, including AWS RDS instances, staged ORC files on S3, CSV files, worksheets, and JSON objects to enhance analytics.

Keboola Connectors is a cloud-based tool with connectors for various data sources. It also provides a suitable environment for data transformations and scheduling of extractions. Keboola prepares data for the Snowflake data warehouse, which is used by the analytics framework ThoughtSpot for creating reports, dashboards, and KPIs.

ThoughtSpot provides customized reports and dashboards as well as AI-powered data

insights and searches. These seamlessly integrate with the Company’s system, offering a unified working environment and white-labeled embedding capabilities.

4.3.2 The model applied to the Case Study

This section will illustrate how *DAF* was used to model the ADW case study. Figure 6.2 displays the primary 4 data nodes, namely Data Sources, Processing Node (ETL - Keboola), Data warehouse (Snowflake), and Analytics platform (ThoughtSpot), from a structural standpoint. It is important to note that the figure presented here is a screenshot of our *Data Architecture Modeling Tool (DAT)* tool in action [4]. *DAT* is built on the top meta-model of *DAF*.

Data is collected from various sources and integrated with Time, Financial, and Sales data. It’s transformed and saved on Amazon S3’s File System using Keboola Node, which handles Excel, JSON, and CSV files. This tool connects to different data sources, extracts and prepares data, stores it in a cloud-based warehouse, and generates reports, dashboards, and KPIs for customers.

4.4 Evaluation

The purpose of Industrial Evaluations is to assess the effectiveness of *DAF* in accurately describing real-world industrial scenarios and to develop a useful tool for industrial purposes. To achieve this, we involve companies in the evaluation process to gather their feedback.

We have successfully modeled *five distinct cases* for various companies, and the feedback we received has allowed us to improve our tool and make it more efficient for industrial usage. In some instances, we discovered that a higher level of architecture was necessary. For example, instead of utilizing the sub-action of ingestion, it was sufficient to have ingestion as the primary action. This led us to develop two levels of architecture, namely the **High Level of Architecture (HLA)** and the **Lower Level of Architecture (LLA)**. We also received feedback regarding **messaging patterns**, and we realized that we needed to incorporate additional messaging patterns into the *DAF*.

These patterns included request/response, publish/subscribe, asynchronous messaging, and synchronous messaging, among others.

These modeled cases were a data pipeline, operational data warehouse, analytical data warehouse, and big data architectures like Lambda and Kappa.

we have gotten feedback from different stockholders, Big data team leaders, Big data Architects, and Big data engineers, table 4.2 shows more details about the evaluation.

Table 4.2: Outline of use cases and roles of the evaluators

Company	Use cases	Experts Roles
Harri	Operational Data Warehouse	Big Data Team Lead
		Big Data Architect
	Analytical Data Warehouse	Big Data Engineer
		Big Data Architect
HP Co.	Data Pipeline	Big Data Team Lead
		Big Data Architect
		Big Data Engineer
Hydre	Data Architectures (Lambda, ...)	Researcher
Locally	NdR Data Architecture	Students

Table 9.3 presents an overview of the evaluated real cases, including their main components, concerns, and corresponding models.

The *DAF* framework is an exceptional solution that accurately models real-life industry cases like data pipelines, data warehouses, and big data architectures such as Lambda and Kappa.

Table 4.3: Real Case Evaluation: Main Components

Cases	Data Formats	Processing Type	Storage Type	Location	Data Stages	Figure
ADW	Structured, Semi-structured	RealTime Batch	NoSQL NewSQL File	Cloud	Generation Ingestion Processing Storing Analyze Visualize Share	4.3
ODW	Structured, Semi-structured	RealTime Batch	NoSQL NewSQL File	Cloud	Generation Ingestion Processing Storing Analyze Visualize Share	4.4
Hydre	Structured, Semi-structured	RealTime Batch	NoSQL NewSQL File	Cloud	Generation Ingestion Processing Storing Analyze Visualize	4.5
Data Pipeline	Semi-structured	Batch	NoSQL NewSQL File	Cloud	Generation Ingestion Processing Storing Visualize	4.6
NDR	Structured, Un-structured	RealTime Batch	NoSQL NewSQL File	Cloud	Generation Ingestion Processing Storing Analyze Visualize Share	4.7

4.5 Threats To Validity

The research was conducted with reference to the *ADW*, which is an analytics architecture that was created by multiple data teams within a single company, situated in different regions across the globe. The model was validated by an experienced practitioner and

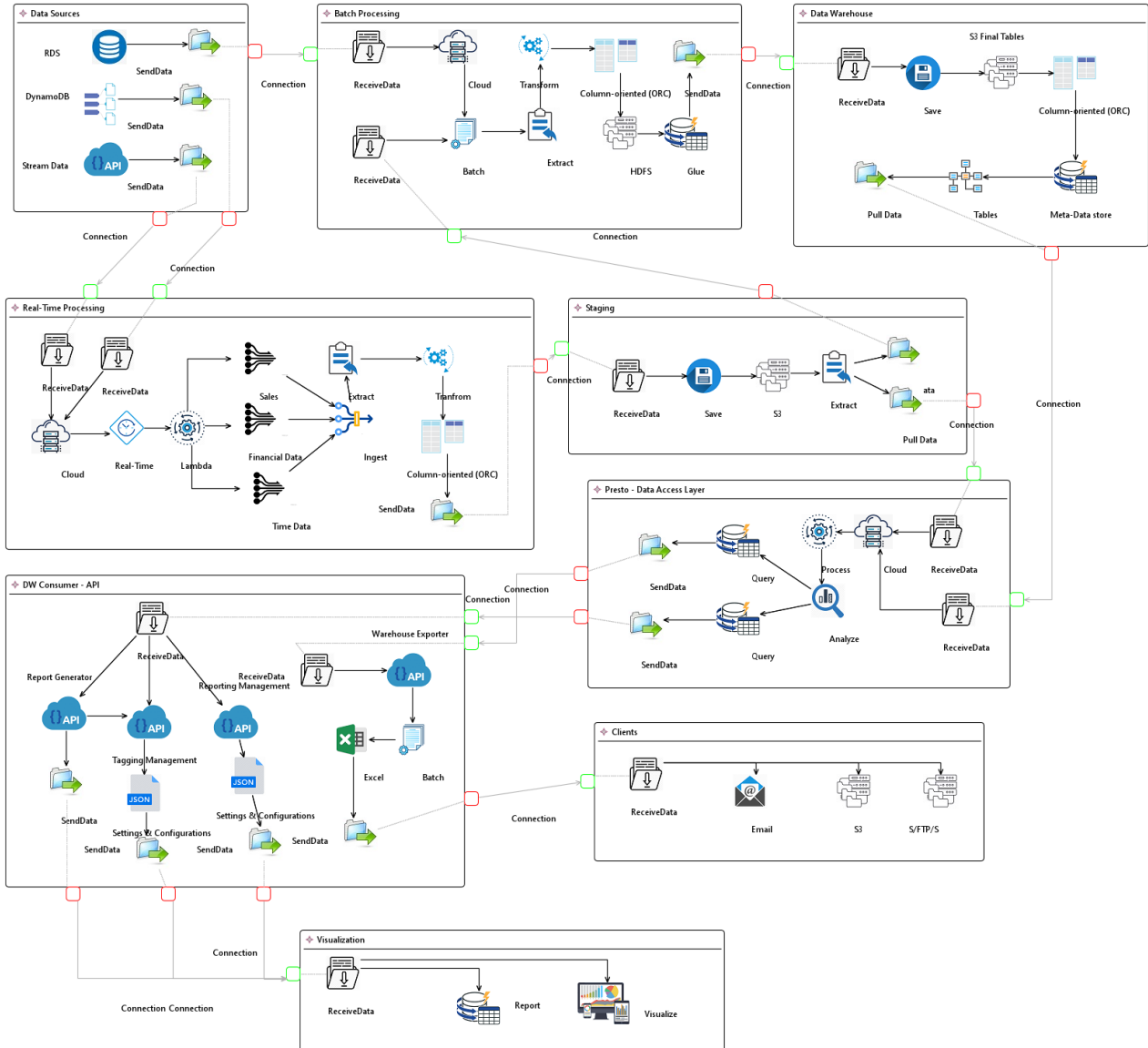


Figure 4.4: Operational Data Warehouse

other team members not involved in the study to address internal validity.

Furthermore, the study was validated again by the Hayder (From Lambd+ Author) case study Fig. 4.5, Error Data Pipeline Fig. 4.6, *Operational Data Warehouse (ODW)*, in the validation process by one more external company from the telecommunications domain (Internet Service Provider).

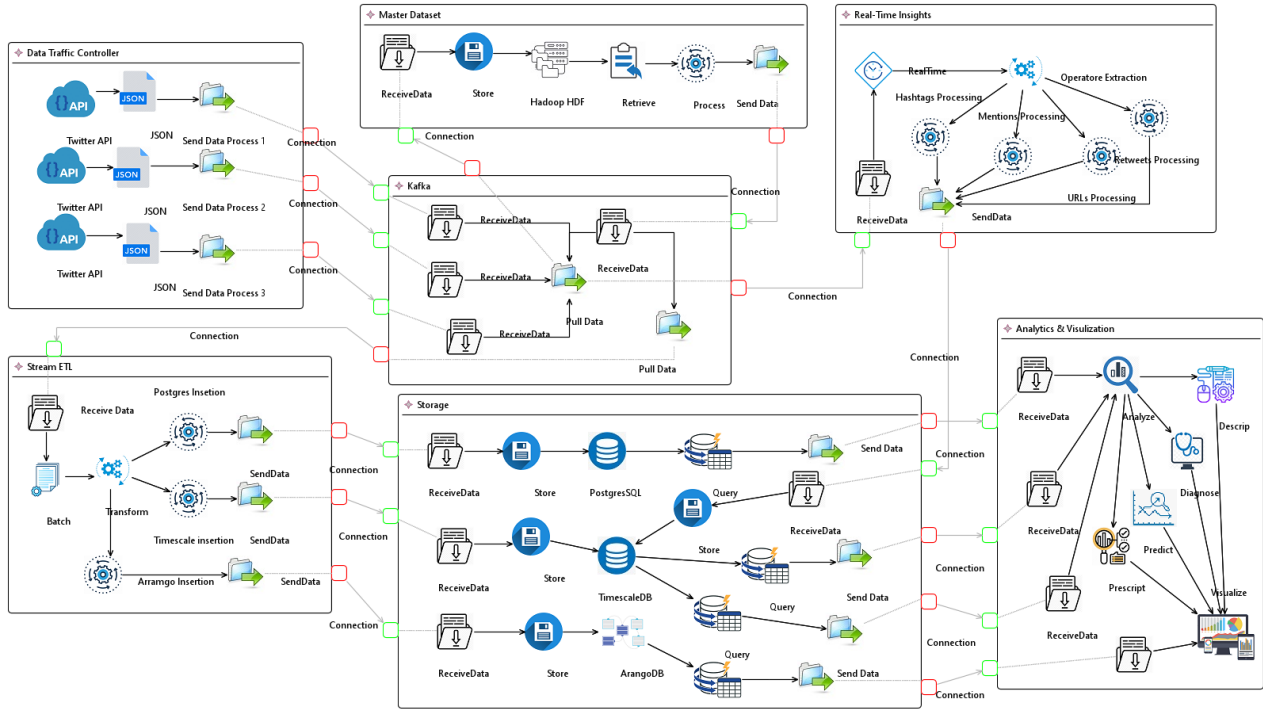


Figure 4.5: Hydre (Lambda+ Example)

4.6 Related Work

This section reviews relevant studies that are related to exploiting the most related research to data-intensive IoT. Raj and Bosch [89] proposed a conceptual model for a data pipeline, which contains two main components (nodes and connectors); the node represents the main abstract data node, and the connection represents the way to carry and transmit the data between nodes. *DAF* includes all data life cycle phases to be able to provide two levels of architecture, *HLA* (High-Level Architecture) and *LLA* (Low-Level Architecture); *LLA* gives the ability to describe the behavior of data nodes(different data formats from different sources, processing types, operations on the data, location, and etc.). Borelli [19] A proposed classification for main software components and their relationships to model a software architecture for particular IoT applications. These components represent the abstract components. *DAML* has the ability to represent all of the mentioned components and their behavior too. Erraissi [43] [44] proposed a meta-model for data sources, ingestion layers, and Big Data visualization layer. *DAML* has the

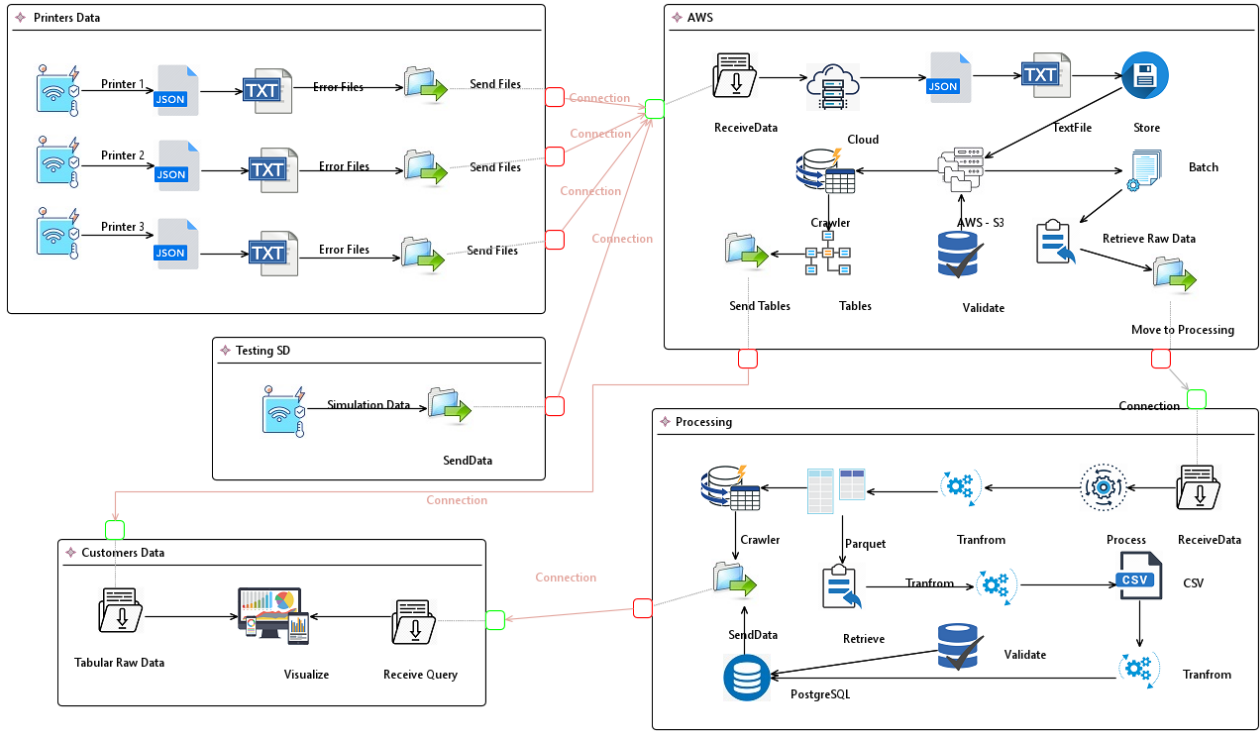


Figure 4.6: Errors Data Pipeline

ability to describe the data in each layer (Source, Ingestion, Processing, Store, Analyze, and Consume). Nesi [81] provided a solution based on a set of instruments to collect the data in real-time, store, and audit data flow for IoT smart City architecture. Using *DAF*, we can describe data flow from the source to the final destination at an abstract concept level; It is not technologies-based. Bashir [16] proposed a meta-model for Data Management and Analytics for IoT smart building. Our meta-model provides a details description of ingestion, data processing, storing data, analyzing, and visualizing the data.

4.7 Conclusion and Future Work

Our paper introduced the *DAF*, a framework for engineering Data Architecture for Data-Intensive applications. With the *DAF*, stakeholders can easily describe both High-Level Architecture (*HLA*) and Low-Level Architecture (*LLA*). The framework encompasses all

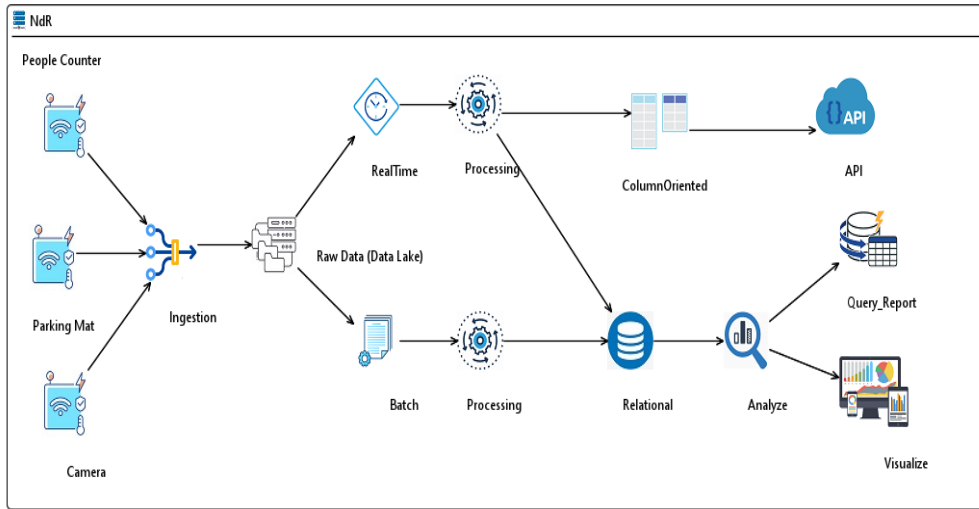


Figure 4.7: NDR case study - (HLA)

essential components of Data Architecture and follows industry best practices. The *DAF* can model data pipelines, data warehouses, and big data architectures such as Lambda and Kappa.

This is an initial starting point for our future work plan, which can be extended to include. First, finishing the current running evaluations with other industrial cases. Second, extending this work to focus more on data quality. Third, integrating the *DAF* with other existing technologies and tools.

Chapter 5

DAT: Data Architecture Modeling Tool for Data-Driven Applications

Data is the key to success for any *Data-Driven (DD)* Organization, and managing it is considered the most challenging task. Data Architecture (*DA*) focuses on describing, collecting, storing, processing, and analyzing the data to meet business needs. In this tool demo paper, we present the *DAT*, a model-driven engineering tool enabling data architects, data engineers, and other stakeholders to describe how data flows through the system and provides a blueprint for managing data that saves time and effort dedicated to Data Architectures for IoT applications. We evaluated this work by modeling five case studies, receiving expressiveness and ease of use feedback from two companies, more than six researchers, and eighteen undergraduate students from the software architecture course.

5.1 Introduction

The International Data Corporation (IDC) [29] expects that by 2025 there will be more than 175 zettabytes of valuable data for a compounded annual growth rate of 61%. Ninety zettabytes of data will be from IoT devices, and 30% of the data generated will be consumed in real-time. A *data architecture* DA is an integrated set of specification

artifacts used to define data requirements, guide integration, control data assets, and align data investments with business strategy. It also includes an integrated collection of master blueprints at different levels of abstraction [57].

This tool demo paper presents the *Data Architecture Modeling Tool* (DAT), an architecture modeling tool for the model-driven engineering of data architecture for data-driven applications, DAT is a graphical modeling tool for DAF [5].

DAT is a data architecture modeling tool for IoT applications that shows how data flows through the system and provides a blueprint for it. It allows the stakeholders to describe two levels of data architecture: high-level Architecture (*HLA*) and Low-Level Architecture (*LLA*). It focuses on representing the data from source to destination and shows formats, processing types, storage, analysis types, and how to consume it.

The rest of this tool demo paper is organized as follows. The methodology is presented in Section 2. The application of *DAT* to a real case study is described in Section 3. The *DAT* evaluation is presented in Section 4. Related work is discussed in Section 5, while conclusions are drawn in Section 6.

5.2 Background

The main focus of this paper is to describe the data architecture of IoT applications through the *Data Modeling Language* (DAML). CAPS Framework in Section 5.2.1. Section 5.2.3 shows *DAML* and reports on the technologies used to implement the DAT.

5.2.1 The CAPS Modeling Framework

CAPS [79] is an environment where Situational Aware Cyber-Physical Systems (SiA-CPS) can be described through software, hardware, and physical space models. The CAPS found three main architectural viewpoints of extreme importance when describing a SiA-CPS: the software architecture structural and behavioral view (SAML), the hardware view (HWML), and the physical space view (SPML).

This environment is composed of the CAPS modeling framework¹ and the CAPS

¹CAPS: <http://caps.disim.univaq.it/>

code generation framework [98] [97] that aim to support the architecture description, reasoning, design decision process, and evaluation of the CAPS architecture in terms of data traffic load, battery level and energy consumption of its nodes.

5.2.2 The Important of Data Architecture

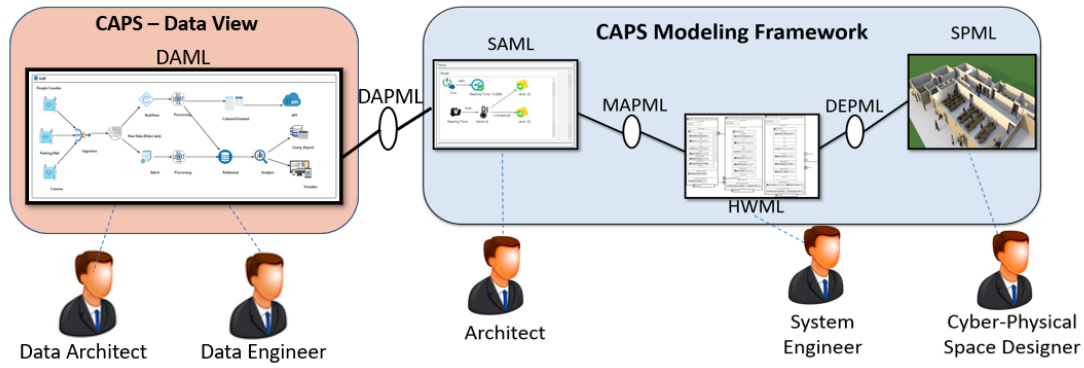
Data architecture is important because it helps organizations manage and use their data effectively. Some specific reasons why data architecture is important to include:

1. Data quality: it helps to collect, store, and use data consistently and accurately. This is important for maintaining data integrity and reliability and avoiding errors or inconsistencies impacting business operations.
2. Data security: it helps to protect data from unauthorized access or modification and ensure that it is used compliantly and ethically. This is particularly important in industries with strict regulations, such as healthcare or finance.
3. Organizational efficiency: it helps organizations better understand and manage their data, increasing efficiency and productivity. By defining the structures, policies, and standards that govern data within an organization, data architecture can help streamline processes and improve decision-making.
4. Business intelligence and analytics: organizations need to collect, store, and analyze large amounts of data. This can support better decision-making, improve customer relationships, and drive business growth.
5. Scalability and flexibility: A well-designed *DA* can support the growth and evolution of an organization. It allows organizations to easily add new data sources, incorporate new technologies, and adapt to changing business needs.

5.2.3 The DAT Tool

The *DAT* modeling framework^{2 3} gives data architects the ability to define a *data view* for *DD IoT Applications* through the *DAML* modeling language [2].

Figure 5.1: The Data View of CAPS



Technologies.

Our tool is based on *MDE*. For that, we use *Eclipse Modeling Framework (EMF)* [37] for building tools and applications based on the structure data model, which consists of three main parts, EMF Core, includes a meta-model for describing the models. EMF Editors contains generic reusable classes for building editors for EMF models. Eclipse Epsilon [36] is a Java-based scripting language for model-based software engineering tasks (e.g., model-to-model transformation and model validation) which strongly support EMF and works with UML, XML, Simulink, etc. To create graphical editors and views for the glsemf models, we used Eugenia [38]. It is a tool to create a graphical model editor by generating the .gmfgraph, .gmftool, and .gmfmap models that the GMF editor from a single annotated Ecore meta-model needs.

DAT implements the DAML meta-model to be considered a fourth view (data view)

²DAT Tool Source Code can be found at <https://github.com/moamina/DAT>

³DAT Tool video demo: <https://youtu.be/Du0VDg1CL1Q>

for the CAPS, as shown in Figure 5.1. How the DAT supports the modeling of data views and its application to actual use cases will be presented in Section 3.

Methodology.

DAT is built based on a meta-model containing a data architecture as a top root meta-class. Any **data architecture** of IoT can contain a set of **DataNodes** (components) and **connections**. A Component is considered a computational unit with an internal state and a known interface [21]. Data nodes can interact by passing data through **data ports**. A component's internal state is denoted by the current behavior of data representation and its values. Data representation includes *data formats, storage technologies, location, and processing type*. Every Node Behavior has a set of behavioral elements denoted by actions and events that depict the data flow within the component. This element can be executed when a previous action in the behavioral data flow has been achieved or triggered by an event like **ReceiveData**. Other main actions are **Generation, Ingestion, Process, Store, Analyze, and Consume**. An **event** is triggered in response to the external stimulus of the component. To show the data flow and connection between the events and actions, we use **links**.

Steps To Use.

The architect needs to follow the following steps to use the tool to model any case:

1. Download the source code from The GitHub ⁴ and follow the steps in the tool demo video to launch the tool ⁵.
2. Define which level of abstraction you need (High-Level or Low-Level). You could use a single Data Node at the High-Level, whereas you need to define the structure and behavior at the low level.
3. Define the main data nodes and the connection between them to determine the order of each node, such as the data source is the first data node. Ingestion comes

⁴DAT Tool Source Code can be found at <https://github.com/moamina/DAT>

⁵DAT Tool video demo: <https://youtu.be/Du0VDg1CL1Q>

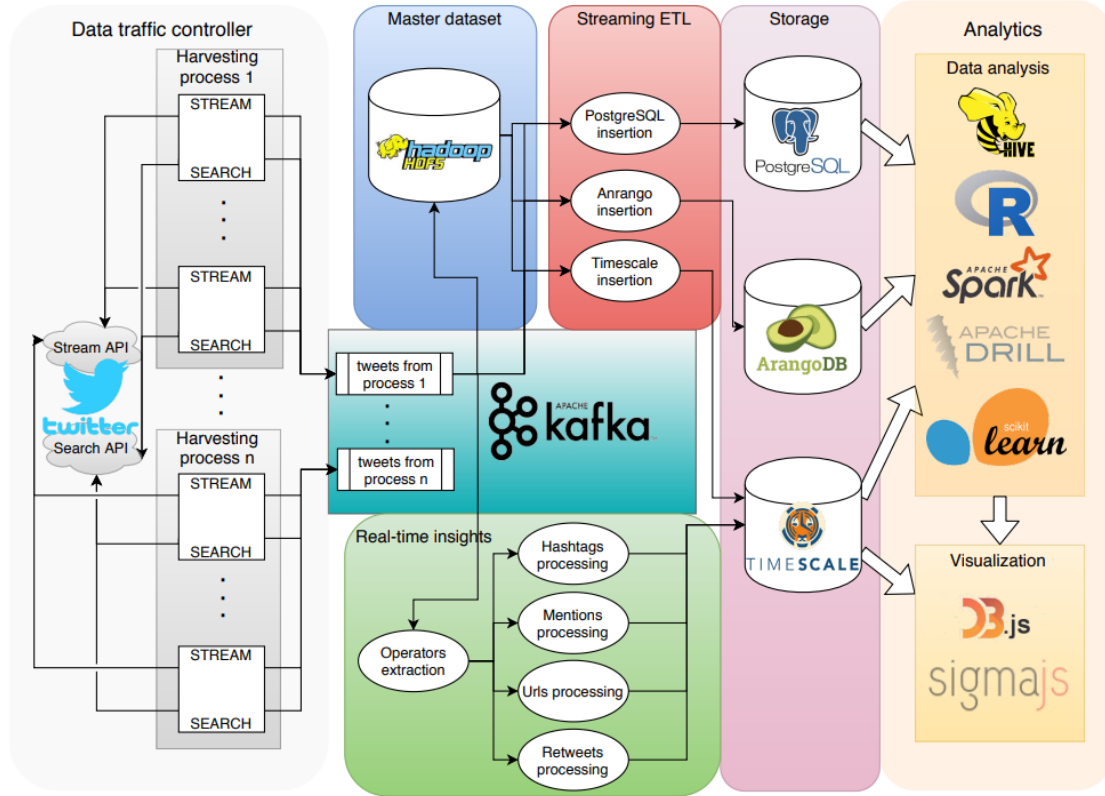


Figure 5.2: The Hyde architecture

the second one, and the connection shows the data flow from the source to the ingestion data node.

4. At the internal behavior, you could use data elements (low-level elements), such as data formats (JSON, XML, Video, ...), and sub-operation, such as (classification, data reduction, cleaning, validation, filter, classification, ...).

5.3 Real Use Cases

This section introduces the existing data architecture description used by three companies contributing to the DAT tool. We have chosen three cases from five cases to present our tool in the tool paper, these cases are from A (5.3), B, and D (5.2) from section 3.3.

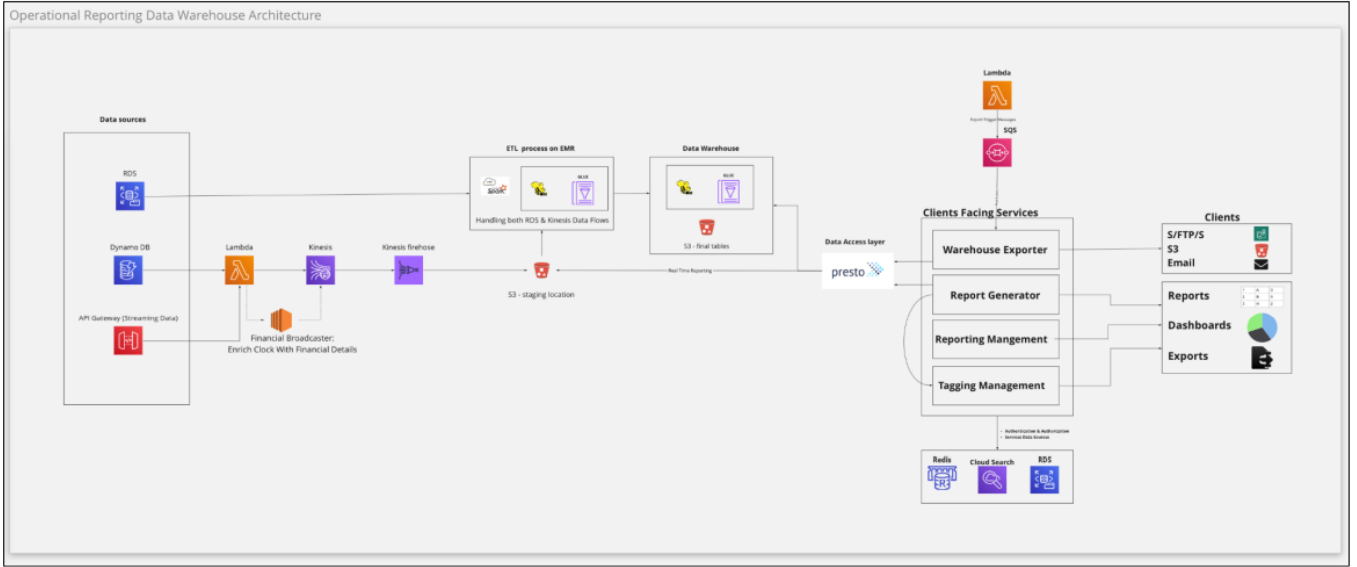


Figure 5.3: Operational Data-warehouse

5.4 Evaluation

The DAT cases were evaluated through interviews with seven industry professionals from two companies of different domains and different maturity levels and one external researcher; table 5.1 shows more about the roles of the evaluators. The evaluation section will be structured in terms of agreements and suggestions for improvement.

Table 5.1: Outline of use cases and roles of the evaluators

Company	Use cases	Experts Roles
Company 1	Operational Data Warehouse	Big Data Team Lead
		Big Data Architect
	Analytical Data Warehouse	Big Data Engineer
		Big Data Architect
Company 2	Data Pipeline	Big Data Team Lead
		Big Data Architect
		Big Data Engineer
Company 3	Lambda Architecture	Researcher
Locally	NdR Data Architecture	Students

5.4.1 Errors Data Pipeline

Figure 5.4 shows the Errors Data Pipeline using DAT. The first author presented the models and collected the practitioners' feedback.

Agreements: The model described the data flow from generation to destination. The model was easy to understand for new data engineers. The tool has the flexibility to change and add new nodes.

suggestion: It is good to include the data quality metrics that could apply to the data at each stage.

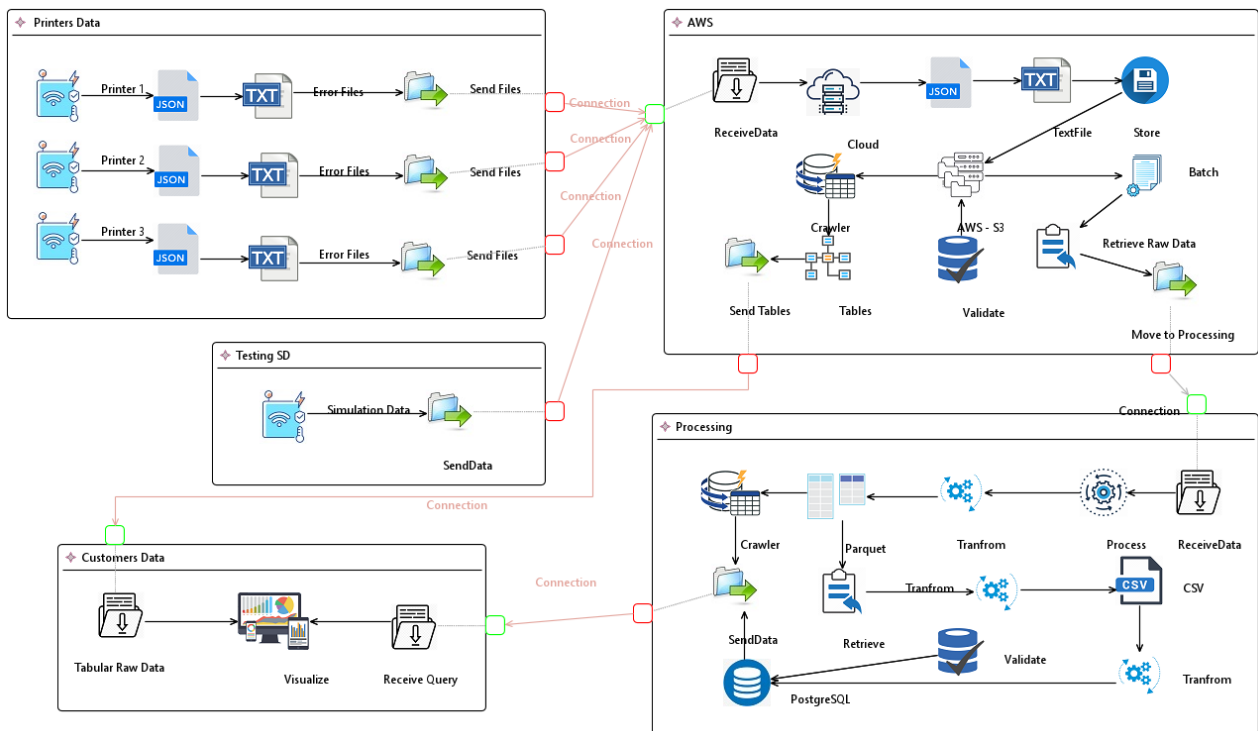


Figure 5.4: Errors Data Pipeline

5.4.2 Hyde (Lambda Architecture)

Figure 5.5 shows the Hyde model using DAT. The first author presented the models and collected feedback from Lambda+'s author.

Agreements: The model represents very well the Hyde case. The indications of when data are stored on disk are helpful, especially when working on a big data architecture with people who don't know each technology's details. The real-time and batch elements are useful as well.

suggestion: the first suggestion is similar to the first case related to data interaction patterns. The second suggestion was a starting point for us to provide two levels of architecture, High-level architecture (HLA) and Low-Level architecture (LLA).

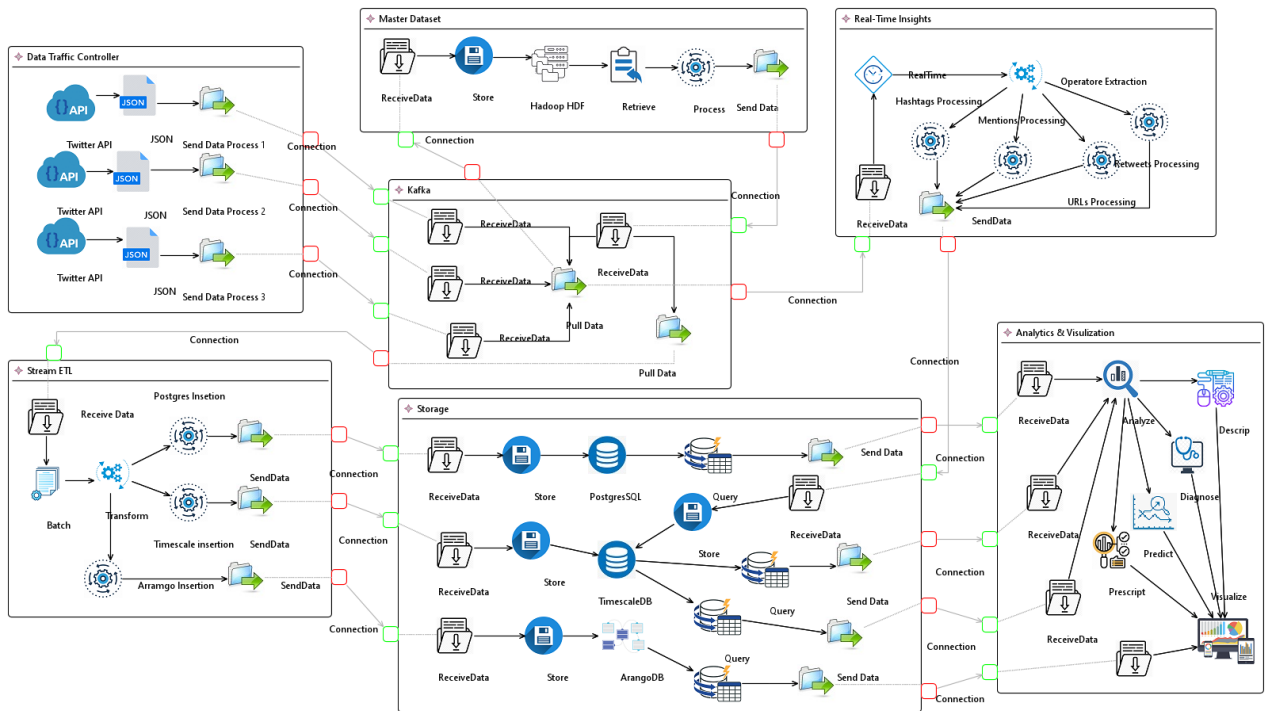


Figure 5.5: Hyde (Lambda+ Example)

5.4.3 Operational Data Warehouse

Figure 5.6 shows the ODW model using DAT. The first author presented the models and collected the practitioners' feedback.

Agreements: The model was able to describe the details of the case and was easy to share and understand by different teams in other parts of the world. The model was

a good communication language between team members, which means easy to avoid misinterpretations.

suggestion: In the current version of the DAT, the only way to show how different components interact with each other is to send/receive data. The suggestion was to include all data interaction patterns (request/response, publish/subscribe, pull/push, and others).

5.5 Conclusion and Future Work

This tool demo paper has presented the DAT, an architecture description, and the associated modeling platform for the model-driven engineering of Data Architecture for IoT. It is implemented on top of the Eclipse Modeling Framework. It can allow the stack-holders to describe two levels of data architectures, high-level Architecture (HLA) and Low-Level Architecture (LLA).

This is an initial starting point for our future work plan, which can be extended to include finishing the current running evaluations with other companies and trying to model different big data patterns and architectures. Second, try to integrate the DAT with other existing technologies and tools.

5.6 Acknowledgment

The authors would like to thank Prof. Giovanni Stilo, Prof. Annabelle Gillet (Lambda+), Prof. Karthik Vaidhyanathan, Mostafa Shaer, Itay, and Roi from HP Team, Mustafa Tamim and Anas Eid from Harri Team, Mudassir Malik, Apurvanand Sahay, and Arsene Indamutsa as a researcher for their contributions in the evaluation.

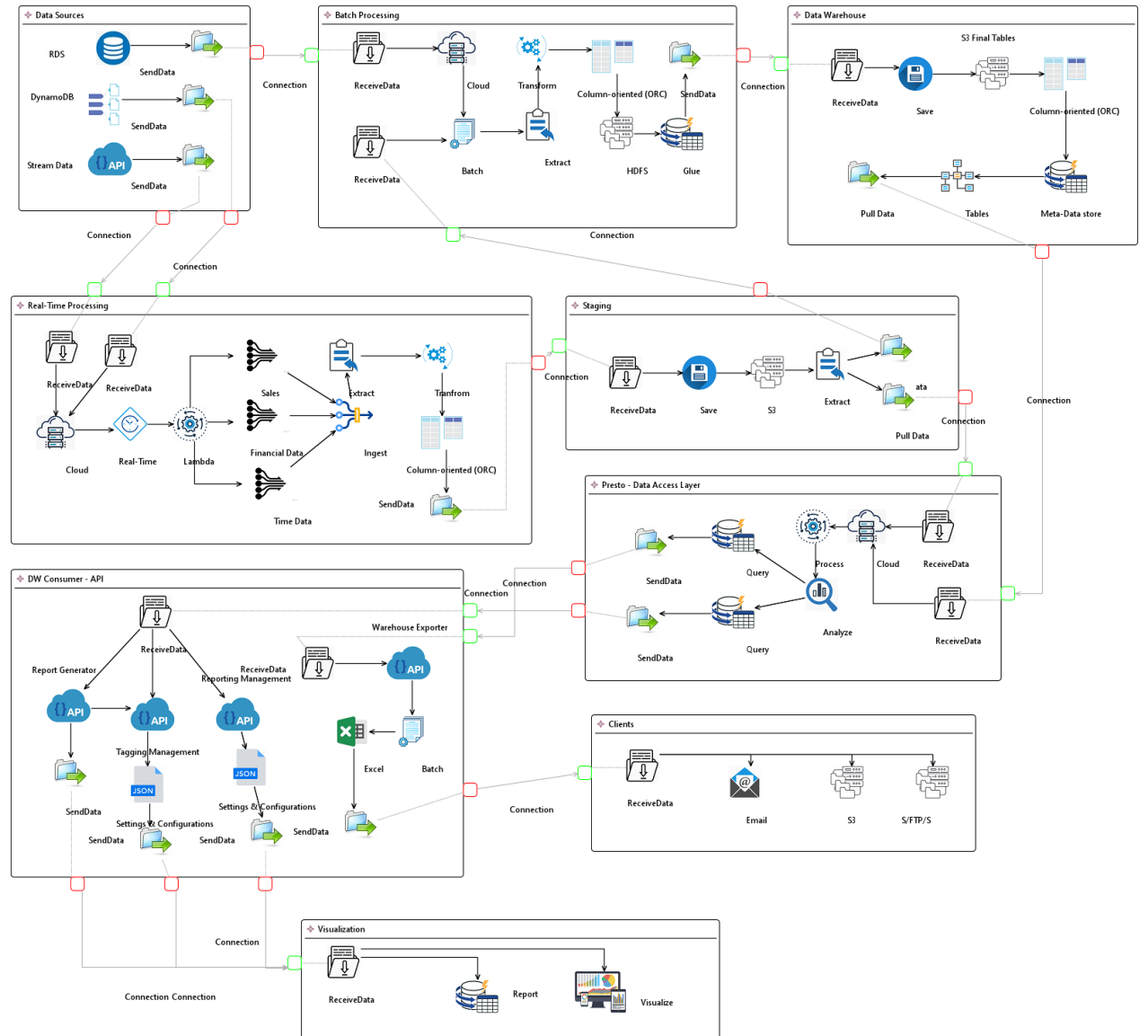


Figure 5.6: Operational Data Warehouse

Chapter 6

Modeling Data Analytics Architecture for Data-Driven Applications using DAT

Data analysis plays a significant role in extracting meaningful information from big data. Data analysis consists of acquisition, storage, management, analytics, and visualization. Providing an abstract view of data analytics applications is crucial to ensure that the data will transfer into meaningful information. Data Architecture (*DA*) is one of the ways to provide that. This article shows industrial experiences in building *Data Analytics Architecture (DAA)*. We use model-driven engineering to model a data analytics architecture for applications using DAT. We evaluated this work by modeling Analytics Data Warehouse (*ADW*) as a case study from one company, receiving feedback.

6.1 Introduction

The International Data Corporation (IDC) [29] expects that by 2025 there will be more than 175 zettabytes of valuable data for a compounded annual growth rate of 61%. Ninety zettabytes of data will be from IoT devices, and 30% of the data generated will be consumed in real-time.

Data analytics for the Internet of Things (IoT) involves using a range of techniques and technologies to gather, store, and analyze data generated by connected devices and sensors [6] [100]. This can help better to understand the performance and behavior of the IoT systems and to identify patterns and trends in the data that can be used to improve their operations.

There are several fundamental IoT data analytics approaches, including real-time, predictive, and prescriptive analytics [39] [6] [14]. Real-time analytics [49] involves using algorithms and software to analyze data generated by IoT devices to identify and act on data trends and patterns in near-real time. Predictive analytics uses historical data and machine learning algorithms to forecast future outcomes and behaviors. In contrast, prescriptive analytics uses optimization algorithms to recommend actions and decisions that can help businesses to achieve their goals.

In the IoT domain, data analytics can be used for various applications, including *predictive maintenance, energy management, supply chain optimization, and customer behavior analysis* [39]. By leveraging data analytics, we can gain valuable insights into the performance and behavior of their IoT systems. We can use this information to improve their *operations, reduce costs, and drive innovation*.

Analytics architectures are the systems and technologies used to collect, store, process, and analyze data to gain insights and make data-driven decisions [124]. These architectures typically involve using distributed computing systems, such as clusters of servers or cloud-based platforms, to handle the processing and storage of data. In addition to these systems, analytics architectures typically include specialized software tools and algorithms for analyzing and gaining insights from the data. These tools may consist of data mining, machine learning algorithms, and visualization and reporting tools to present the analysis results. An analytics architecture's specific components and design can vary depending on the organization's needs and goals.

Based on our meetings with researchers and practitioners from different companies, we found that there is an urgent need to have a way to provide a blueprint for Analytics Applications.

To support the engineering of data analytics IoT applications, we use an architec-

tural description designed according to the IEEE/ISO/IEC 42010 standard [58]. The framework supports the architecture description, reasoning, and design decision process applications.

The framework is DAT¹. It is based on an architectural approach [58] to provide a data architecture for IoT applications. It is considered as a data view for CAPS [79] [98] [97]. The DAT [4] [5] is realized by exploiting the advanced Model-Driven Engineering (*MDE*) technique which is meta-modeling.

The main contributions of this paper can be summarized as follows:

1. Using An architecture description of IoT analytics applications that can provide analytics architectures.
2. Industrial experience in building IoT analytics applications.
3. The application of the modeling languages to an Analytics Data Warehouse case study is outlined.

The rest of this paper is organized as follows. The background is presented in Section 2. The research methodology is presented in Section 3. Data analytics architecture challenges are presented in Section 4. The application of DAT to a real case study is described in Section 5. The DAT evaluation is presented in Section 6. Threats to validity in Section 7. Related work is discussed in Section 8, while conclusions are drawn in Section 9.

6.2 Background

This section provides an overview of big data analytics and the DAT framework.

6.2.1 Big Data Analytics

Big Data analytics (BDA) is the process of analyzing the massive raw data by data scientists to uncover unidentified relationships and associations, market trends, and valuable

¹DAT: **D**ata **A**rchitecture Framework for IoT

information to make business decisions [92] [14] [124]. Business intelligence (BI) is more focused and refers to the technologies, applications, and practices used to gather, store, access, and analyze data to help them make more informed business decisions. BI can help businesses identify trends and data patterns and forecast future performance and outcomes [14].

BDA differs from Data Warehouse (DW); it is considered a storage architecture for the data that various organizations and business enterprises collect [14]. BI data consists of data from the storage (previously stored data) and streaming data, supporting the organizations in making strategic decisions. Big data analytics is the science of handling (examining or analyzing) large data sets with a different format, that is, structured, semi-structured, or unstructured data, which may be streaming or batch data.

6.2.2 DAT: Data Architecture Modeling Tool

DAT is a data architecture modeling tool for IoT applications [4] that shows how data flows through the system and provides a blueprint for it [2]. It allows the stakeholders to describe two levels of data architecture: high-level Architecture (*HLA*) and Low-Level Architecture (*LLA*). It focuses on representing the data from source to destination; it shows formats, processing types, storage, analysis types, and how to consume it. It is built based on a structural and behavioral meta-model. The *DAT* supports the understanding and documentation of *DD* applications. For this purpose, it provides a data-view architecture modeling approach designed according to the IEEE/ISO/IEC 42010 standard [58], based on modeling language to describe a *DD* application: the Data architecture structural and behavioral view (*DAML*).

6.3 RESEARCH METHODOLOGY

The objective of this study is to understand the data analytics architecture as well as the challenges experienced at the case company. Based on the study objectives, we formulated the following research question:

- **RQ1:** What challenges related to data analytics architecture do practitioners in the case company experience?

The research methodology adopted for the study is illustrated in Figure 6.1.

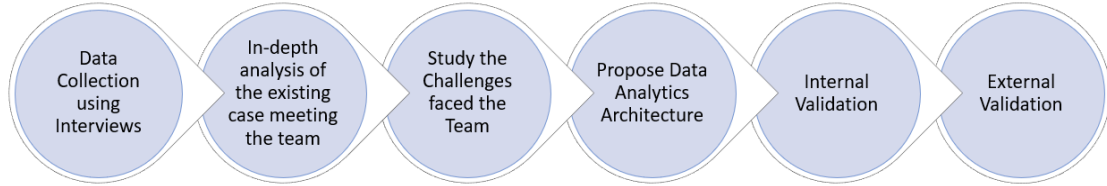


Figure 6.1: Research Methodology

6.3.1 About The Company

The company offers a next-generation frontline employee experience platform built for companies that have served at the heart of their business. The Company suite of talent attraction, workforce management, employee engagement, and compliance technologies enable organizations to intelligently attract, manage, engage, and retain the best talent to run and improve their business. It serves over 20,000 restaurant and hotel locations and four million hospitality employees globally, with emerging growth in retail and healthcare.

6.3.2 Exploratory Case Study

A qualitative approach was chosen; it is a research method that collects and analyzes non-numerical data, such as words, images, and observations. It is often used to explore the experiences, perspectives, and motivations of individuals or small groups in-depth and to gain a deep understanding of real-world cases [116]. In an exploratory case study, the researcher may use various methods to gather data, such as interviews, observations, and document analysis. The aim is to collect as much rich and detailed information as possible about the case to understand better the context and the factors that influence it.

6.3.3 Data Collection

The data was collected through interviews and meetings [102]. Based on the research objective to study and explore the applications consuming data in the company, an interview guide with 30 questions categorized into three sections was formulated. The first author prepared and reviewed the interview guide with the second author. Each interview lasted 50 to 60 minutes. The first section focused on the background and experience of the interviewees, table 6.1 shows the role of each interviewee. The second section focused on the primary data components of data collection, storage, processing, analysis, and visualization. The last section inquired about best practices and the obstacles faced during each step of building the application. The interviews were recorded with the interviewees' permission and transcribed later for analysis. Data collected through the meetings and discussions are also incorporated.

Table 6.1: The Roles of the Interviewees

Company	Use cases	Experts Roles
Case A	Analytical Data Warehouse	Big Data Team Lead
		Big Data Architect
		Big Data Engineer

6.3.4 Data Analysis

The first author prepared a summary of all recorded audio. The transcripts were investigated for relations, similarities, and dissimilarities. The first author prepared notes during the meetings with the team and analyzed them further. The main contact point helped with interpreting the parts of the Data Architecture. These notes and the codes from transcripts were further analyzed to obtain an end-to-end view of different use cases. It also helped to understand the parts common to all use cases. After careful analysis of collected data and based on the inputs from the author, the first author developed the first model for Data Analytics Architecture, which got refined through iterations.

The practitioners shared the case with a detailed description. The first author prepared notes during the meetings with the practitioners and analyzed them further, which

required more in-depth interviews and additional questions to understand the architecture better. After deep analysis of collected data and based on the inputs from the second author and the practitioners, the first author enhanced DAT to provide the Data Analytics Architecture model.

6.3.5 Validation Study

The validation study is performed internally and externally through qualitative interviews and feedback sessions. First, The first author presented the Data Analytics Architecture model of the case to the teams inside the case company. The reflections about the model, overall opinion, agreements, and suggestions for improvement were collected from every practitioner. These reflections are considered internal validation. One telecommunication company was selected for external validation of our findings and is in the validation process. another case is a Lambda+ [49] as another external validation Fig. 6.3 shows Data Analytics Architecture for the provided case.

The architecture model was then modified to address issues raised during the talks. Also, the inputs from the practitioners are incorporated into the architecture model. The remaining issues will be addressed in future works.

6.4 DATA Analytics Architecture Challenges

In this section, insights into data analytics architecture challenges are presented based on the findings from our case analysis. We have identified eight major challenges with data analytics architecture used in the company :

1. *Big Data* analytics architectures should be able to handle (process, analyze, and combine) a huge amount of data from different sources at high velocity.
 - (a) *Volume* the sheer size and scale of the data sets that are used in big data analytics. These data sets can be massive, often consisting of millions or billions of records, and may be generated from various sources, from the case we have weblogs, social media, and transactional systems. The volume of big

data can present challenges for data processing and analysis techniques, which may not be able to handle such large volumes of data efficiently. As a result, the company uses distributed computing systems, such as clusters of servers or cloud-based platforms, to process and store big data.

- (b) *Variety* the wide range of data types included in big data sets. These data sets may consist of structured data, such as records in a database, and unstructured data, such as text, audio, and video. They may also include semi-structured data, such as XML or JSON, which has some inherent structure but is not organized similarly to structured data. From the case we have Column-oriented, Excel, JSON, CSV, and relation database formats. The variety of data types in the case can present challenges for data processing and analysis techniques, which may not be designed to handle such a wide range of data types. As a result, the company uses specialized software tools and algorithms, such as natural language processing and machine learning, to analyze and gain insights from big data. In addition, data integration and management strategies may be used to combine and organize the different data types to support analysis and decision-making, such as Extract, Transform, Load (ETL) strategy: This involves extracting data from various sources, transforming it to fit a common data model, and loading it into a central repository for analysis and reporting.
- (c) *Velocity* the speed at which data is generated and processed in big data analytics. This can be a significant challenge, as big data sets may be generated at very high rates, often in real-time, and may need to be processed and analyzed quickly to extract valuable insights. From the case we data in different speed. The velocity can present challenges for data processing and analysis techniques. As a result, the company uses distributed computing systems, such as clusters of servers or cloud-based platforms, to process and store big data. In addition, specialized software tools and algorithms, such as stream processing and in-memory databases, are often used to analyze and gain insights from big data in real-time.

2. *Data Collection and Storage* refers to gathering and storing data from various sources, such as sensors, databases, or user input. Once the data has been collected, it needs to be stored in a suitable location for further processing and analysis. There are several options for storing data, Flat files, Relational databases, NoSQL databases, Local storage, and Cloud storage.
3. *Data Processing* is related to processing large datasets. Converting the data from different forms into one form for analysis is challenging.
 - (a) *Data Processing Type* Stream processing; This type of processing involves analyzing data as various sources generate it, in real-time or near real-time. This contrasts batch processing, where data is collected and processed in batches at regular intervals. This approach is suitable for processing data that is not time-sensitive and does not need to be processed in real-time.
 - (b) *Scalable data processing* refers to the ability of a data processing system to handle increasing amounts of data and maintain or improve its performance as the volume of data grows. Scalability is an essential consideration in the design of data processing systems, mainly when dealing with large and complex data sets, such as those encountered in big data analytics. A scalable data processing system can handle data volume and complexity increases without significant performance degradation. This may be achieved through distributed computing systems, such as clusters of servers or cloud-based platforms, which can provide additional resources and processing power as needed. In addition, scalable data processing systems may use specialized software tools and algorithms, such as map-reduce and parallel processing, to distribute and manage data processing across multiple computing resources.
4. *Personalization of analytics* refers to the use of data and algorithms to adapt the analysis and presentation of data to individual users or groups of users. This may involve considering factors such as the user's role, preferences, and past behavior, as well as the specific goals and objectives of the user or organization. Personalization

in analytics aims to provide users with a more adapted and relevant experience, which can help improve the usefulness and impact of the data and insights generated from the analysis. The Personalization of analytics can be achieved through machine learning algorithms, which can learn from user data and interactions to make personalized recommendations or predictions. It may also involve data visualization and reporting tools that support the customization of data views and presentations for different users.

5. *Share Data* is the practice of making data available to multiple users or systems for analysis and decision-making. This may involve sharing data within the company, or with external parties such as researchers or other stakeholders. Sharing data for analytics can help to improve the accuracy and relevance of the data being analyzed, as well as the insights and decisions that are made based on the analysis. It can also help to promote collaboration and innovation by making data available to a broader community of users and systems. To share data for analytics, companies may use various techniques, such as data warehousing, data lakes, and data sharing platforms, to store and manage the data in a way that allows for easy access and analysis. Data sharing may also be subject to legal and regulatory requirements, such as privacy laws and data governance policies, which must be considered when sharing data for analytics.
6. *Security and Privacy* as data sets may contain sensitive or confidential information that must be protected from unauthorized access or disclosure. To ensure the security and privacy of data used for analytics, organizations may implement various measures, such as encryption, access controls, and data anonymization. These measures can help prevent unauthorized access to data and protect the privacy of individuals whose data is being analyzed. In addition, organizations may also develop data governance policies and procedures to ensure that data is collected, processed, and shared in compliance with applicable laws and regulations. Ensuring the security and privacy of data used for analytics can help protect the rights of individuals and maintain stakeholders' trust and confidence in the data and insights

generated from the analysis.

7. *Analyzing data* is examining data to draw conclusions and make informed decisions.

There are several different types and techniques of data analytics. The four types of analytics are:

- (a) *Descriptive analytics* This type of analytics involves summarizing and describing data, often using statistical techniques such as mean, median, and standard deviation.
- (b) *Diagnostic analytics* This type of analytics involves drilling down into data to understand the root cause of problems or issues.
- (c) *Predictive analytics* This type of analytics involves using data to predict future outcomes.
- (d) *Prescriptive analytics* This type of analytics involves using data to identify the best action to take in a given situation.

The Three techniques of analytics are:

- (a) *Quantitative analysis*; the data based on numbers. it can be performed as (Nominal, Ordinal, Interval, or Ratio data). For Example, financial ratios evaluate the performance of a company. Ratios such as the price-to-earnings ratio or the debt-to-equity ratio provide a way to compare different companies and make decisions about investing in them.
- (b) *Qualitative analysis*; basically answers to “how,” “why” and “what” questions can be performed by one of two approaches (deductive and inductive). For Example, Analyzing Customer reviews on a website.
- (c) *Statistical analysis*; that uses statistical methods for analyzing data such as (A/B testing, Correlation, and Regression). An example is a survey of customer satisfaction with a product. The survey data is collected, and then statistical methods are used to analyze the data and make inferences about the population of all customers.

8. *Data visualization* uses visual representations, such as charts, graphs, and maps, to make complex data sets more straightforward to understand and interpret. Visualization can be a powerful tool for exploring and analyzing big data. It can help identify patterns, trends, and relationships in the data that may not be immediately apparent from raw data. In addition, visualization can make it easier for stakeholders, such as business leaders and data scientists, to communicate and share insights from extensive data analysis. Many visualization tools and techniques can be used for big data, including 2D and 3D charts, geographic maps, and interactive dashboards. These tools may be integrated into specialized big-data analytics platforms or used with general-purpose data visualization software.

6.5 Application of DAT Models to the Analytics Data Warehouse Case Study

This section introduces The Analytics Data Warehouse (ADW) from company (A)3.3 as a case study and its DAT.

6.5.1 The Analytics Data Warehouse

The data sources of this Analytics Data Warehouse are the AWS RDS (Relational Database Service) instances, the duplicate staging ORC files saved on S3 that are generated from the data streams. In addition, this data warehouse consumes data from CSV files, worksheets, and JSON objects that augment the data in the warehouse to make the data-rich for analytics purposes. The ETL process in this data warehouse is developed and run using a cloud-based third-party tool called Keboola Connectors. This tool provides all the needed connectors for data sources like AWS MySQL RDSs, ORC(Optimized Row Columnar) file extractors, Google Drive sheets, CSV readers, etc. Keboola also offers the required environment and tools to perform data transformations on the extracted data. Moreover, it provides the needed capabilities to schedule exe-

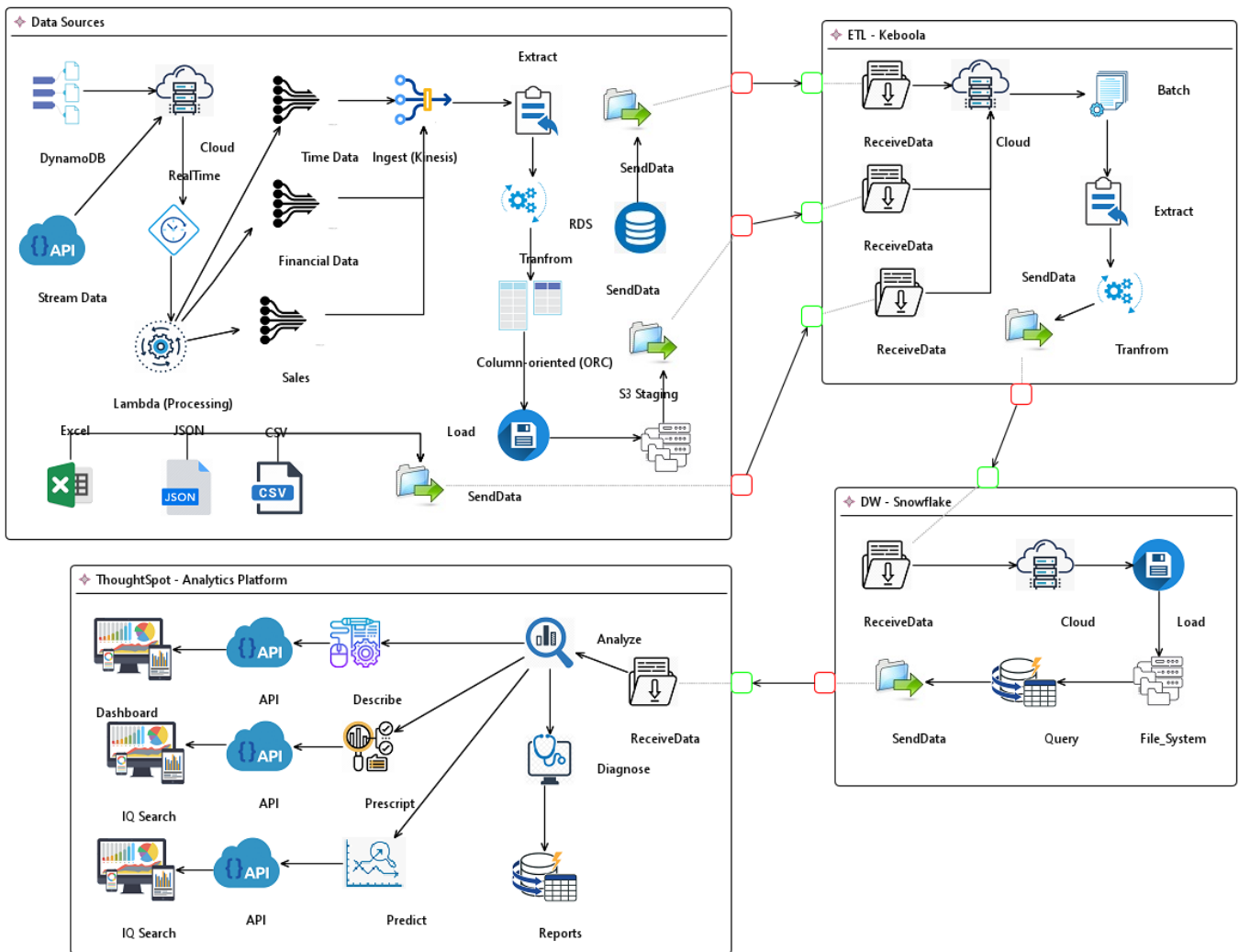


Figure 6.2: Modeling of the Analytics Data Warehouse

cuting all extractions and transformation and orchestration between them. In the case implementation of this data warehouse, the data prepared using Keboola is stored on the cloud-based Snowflake data warehouse. The data in the Snowflake data warehouse is consumed by another cloud-based advanced BI and analytics framework called ThoughtSpot. This framework allows the customers to build analytical reports, dashboards, and KPIs. Through ThoughtSpot, The Company provides customers with an out-of-the-box set of reports, dashboards, and KPIs. Also, customers can build reports and dashboards based on their needs and business inquiries and perform advanced AI-driven data insights and

searches. Thoughtspot reports, and dashboards are integrated with company's system (the Frontend portal and mobile) applications through the SSO mechanism with white-labeled embedding capabilities to offer the customers and end users one integrated and unified working environment and experience.

6.5.2 The DAT model applied to the ADW Case Study

This section shows the modeling of the ADW case study using DAT. From a structural point of view, Figure 6.2 shows the primary 4 data nodes; Data Sources, Processing Node (ETL - Keboola), Data warehouse (Snowflake), and Analytics platform (ThoughtSpot).

The Data source shows how to collect the data from different data sources in different formats, integrate with (Time, Financial, and Sales data), and ingest data to be transferred into a Column-oriented format to be saved on File System(Amazon S3). Other data source is Excel, JSON, and CSV files will be sent directly to the Keboola node to be processed in the cloud. This Node provides all the needed connectors with other data sources. It provides the needed capabilities to schedule (Batch) extractions and transformation. The extracted prepared data is stored in the cloud-based Snowflake data warehouse. Then it will be consumed by another cloud-based advanced BI and analytics framework. The analytics node shows the ability of the framework to provide analytical reports, dashboards, and KPIs to the customers.

6.6 Validation Study

The Analytical Data Architecture model was validated through interviews with three industry professionals from one company. The first author conducted the interview study for validation. The research aimed to provide data analytics architectures that address the challenges in the section 6.4.

This validation of the case will be structured in terms of agreements and suggestions for improvement. Agreements refer to situations where practitioners agree and confirm, while suggestions for improvement refer to situations in which the interviewees had different opinions.

Analytical Data Warehouse Figure 6.2 shows the ADW model using DAT. The first author presented the models and collected the practitioners' feedback.

Agreements: The provided model shows the data flow from different data sources with different formats (Variety) to the final destination, the real-time data (Velocity), the type of data processing (batch and real-time), storing data in more than one storage technologies in the cloud (Scalability), ways of analyzing data, and Finally, explains the consuming data by sharing and visualizing it. Moreover, The model was easy to share and understand by different teams in other parts of the world. The model was a good communication language between team members, which means easy to avoid misinterpretations.

Suggestion: In the provided model, the only way to show how different components interact is to send/receive data. The suggestion was to include all data interaction patterns (request/response, publish/subscribe, pull/push, and others). Current Visualization is enough, but it is better to show how to Visualize data; the suggestion was to show visualization ways such as charts, pies, and others. It is good to provide a way to show sensitive data and a way to protect the data.

Finally, DAT showed the ability to provide analytics architecture for IoT applications.

6.7 Threats to Validity

This study was based on an existing case (Analytics Data Warehouse) related to analytics architectures, developed by more than one data team from the same company located in different parts of the world. To address the internal validity threat, one of the company's practitioners with good experience with data analytics architecting was asked to validate the model. Also, the model was validated with other teams in the company who were not involved in the study.

Furthermore, the study was validated again by the Hayder (From Lambd+ Author) case study Fig. 6.3 and in the validation process by an external company from the telecommunications domain (Internet Service Provider). Hyder is from a (Cocktail)

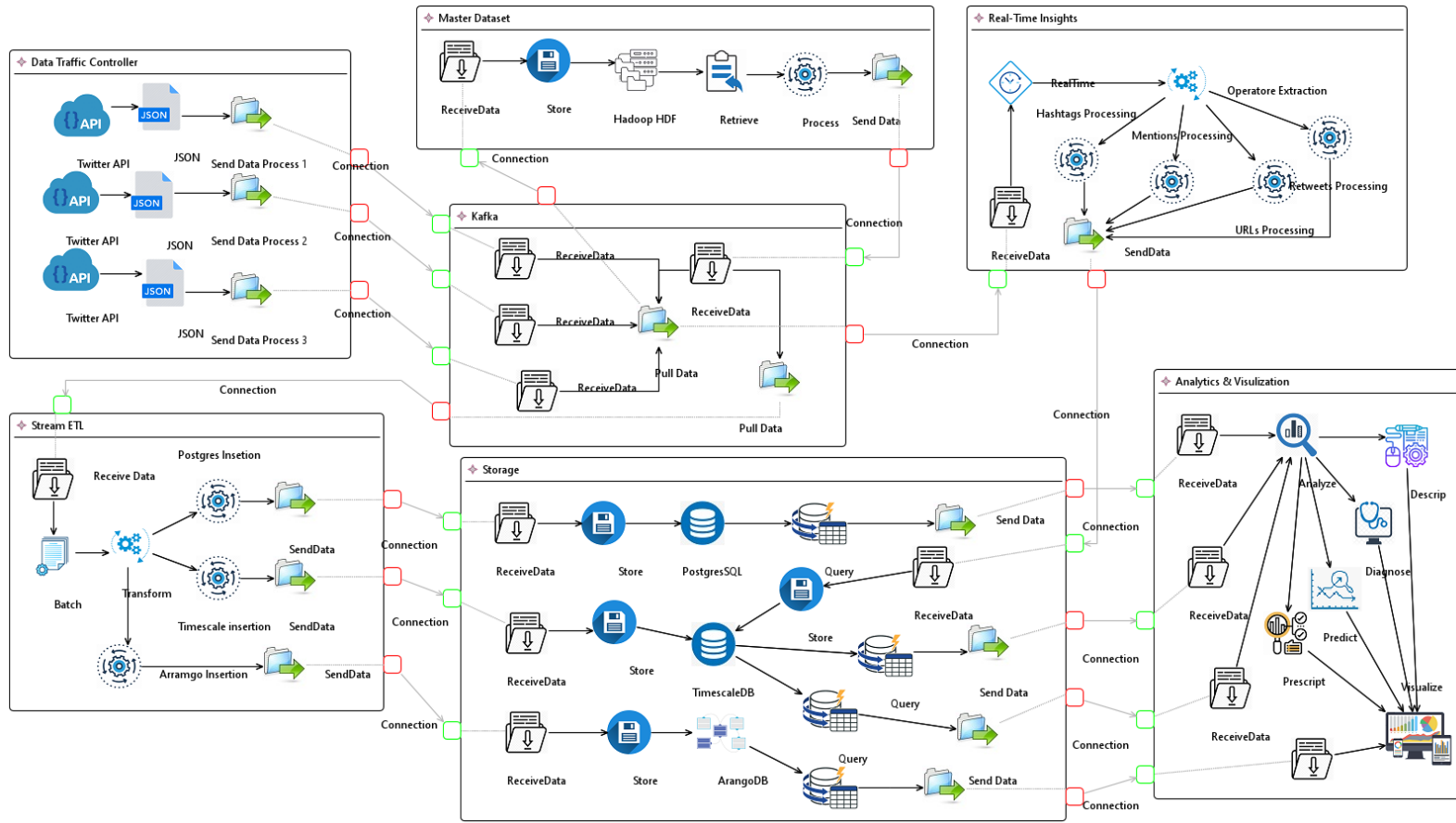


Figure 6.3: Lambda+ Model

research project which aims to study the discourses in two domains in, health and food, and identify weak signals in real-time using social network data. The data come from Twitter, compute real-time insights, and store data for exploratory analysis.

6.8 Related Work

This section reviews relevant studies that are related to exploiting the most related research to data analytics for IoT. Mishra and Misra [76] show that structured and unstructured data can be mined to provide meaningful insight into the business. DAT can model all data formats (structured, Semi-structured, and unstructured).

Raj and Bosch [89] proposed a conceptual model for a data pipeline, which contains

two principal components (nodes and connectors), the node represents the primary abstract data node, and the connection represents the way to carry and transmit the data between nodes. DAML includes all data life cycle phases and gives the ability to describe the behavior of data nodes(different data formats from other sources, processing types, operations on the data, location, etc.).

Erraissi [43] [44] proposed a meta-model for data sources, ingestion layers, and Big Data visualization layer. DAML can describe the data in each layer (Source, Ingestion, Processing, Store, Analyze, and Consume).

Bashir [16] proposed a meta-model for Data Management and Analytics for IoT smart building; our meta-model provides detailed descriptions of ingestion, data processing, storing data, analyzing, and visualizing the data.

6.9 Conclusion and Future Work

In the immediate future, it will be inexorable that the daily analysis will not be able to keep up with the daily influx of data. Along with the increased popularity of data and its products, challenges associated with it also increased. Data scientists and other practitioners working with data spend considerable time combating those challenges.

This paper highlighted those challenges and showed the applicability of DAT in providing an analytical architecture by evaluating a real case (Analytical Data Warehouse). The modeled case was validated using an exploratory case study where practitioners from one company participated.

Our future work plan can include finishing the current running evaluations with other companies.

Chapter 7

Ensuring Data Quality Throughout the Data Journey

In data-intensive applications, ensuring data quality has become increasingly crucial for effective decision-making and operational processes. A comprehensive architecture framework for modeling and monitoring data-quality (DQ) has been proposed to address this challenge. This framework covers the entire data lifecycle, from data ingestion to consumption. It emphasizes the importance of continuous monitoring and feedback loops to proactively identify and rectify potential data quality issues. By adopting this approach, data-intensive applications can optimize their performance and ensure reliable and high-quality data for decision-making purposes. The paper concludes with real-world case studies and practical recommendations to support the adoption and implementation of the proposed framework.

7.1 Introduction

More than 175 zettabytes of valuable data are anticipated to exist by 2025, representing a compound annual growth rate of 61%, according to the International Data Corporation (IDC) [29]. It's quite concerning that organizations lose an average of \$12.9 million every year due to poor data quality [48]. In any enterprise system, data plays a pivotal role as

it aids in decision-making, training machine learning (ML)/deep learning (DL) models, generating insights, creating reports, and building smart, sustainable cities [17]. The "data-intensive" refers to the strategic process of utilizing insights from data. Adopting the best practices for collecting, storing, analyzing, and safeguarding valuable data is crucial to ensure a systematic data-driven approach.

A Data Architecture (*DA*) is essential for any data-intensive application. It refers to the collection of models, policies, rules, and standards that regulate data collection, storage, formatting, integration, and utilization in data systems and organizations[57]. *DQ* is of utmost importance for making informed decisions based on data analysis. Data accuracy and reliability directly affect the credibility of the insights and decisions derived from it [65] [60]. Incorrect data quality can result in inaccurate conclusions, flawed decision-making, and adverse effects on the organization [22].

Developing and implementing quality data architecture solutions can pose a significant challenge for organizations that lack effective and adequate data architecture tools. In such cases, manual processes are often relied upon to manage data architecture, which can be time-consuming and prone to errors. The scalability and flexibility of these manual processes may also be limited, making it challenging to address the growing complexity of modern data environments [23] [27].

Our *DAFQA*¹ framework, which adheres to the internationally recognized IEEE/ISO/IEC 42010 standard [58], empowers data architects to effortlessly construct data-intensive applications. Through clearly defined data quality metrics and key dimensions, including completeness, accuracy, timeliness, consistency, and conformity [119]. Our framework effectively covers all data lifecycle stages for comprehensive monitoring and continual improvement.

The main contributions of this paper can be summarized as follows:

1. Using An architecture description of data-intensive applications that can provide high-quality architectures.
2. A set of model-to-text transformations that transforms Modeling Language to

¹*Data Architecture Quality Framework for Data-Intensive Applications (DAFQA)*

Python Code that uses *Great Expectations (GE)* Library [45] to monitor the data quality.

3. The application of the modeling languages to the Errors Printers Data Pipeline and Telecommunication case studies is outlined.

Automatically generating code for data quality checks is a highly efficient and *time-saving* solution. It ensures *consistency and standardization* of data across various datasets, improving accuracy and reliability. This promotes sound data governance practices, facilitating agility in adapting to changing data sources. Additionally, this method offers documentation for traceability and compliance purposes, simplifying the assessment process and enabling informed decisions based on reliable data.

The rest of the paper is organized as follows. The background is presented in Section 2. The research methodology is presented in Section 3. Data Quality challenges are presented in Section 4. Modeling data-intensive applications in Section 5. Data quality code generation in section 6. The application of *DAFQA* to a real case study is described in Section 7. The results are presented in Section 8. Related work is discussed in Section 9, while conclusions are drawn in Section 10.

7.2 Background

7.2.1 Great Expectations

GE is a data validation and profiling tool that helps organizations maintain data quality and improve team communication [45]. It enables data scientists, engineers, and business stakeholders to work together to ensure clean, accurate, and consistent data. Expectations can be defined, data quality can be tracked, and reports and visualizations can be generated. With *GE*, organizations can build trust in their data and make informed decisions based on reliable information.

7.3 Research Methodology

7.3.1 Definition of research questions

This study aims to understand the industry's data quality metrics for *DA*. Based on the study objectives, we formulated the following research questions:

- **RQ1:** What critical challenges do practitioners face in maintaining high-quality data for data-intensive applications?
- **RQ2:** In order to ensure data quality, what methods can be employed for monitoring? Additionally, what dimensions of data quality may be evaluated in the process?

Figure 7.1 illustrates the research methodology utilized for the study.

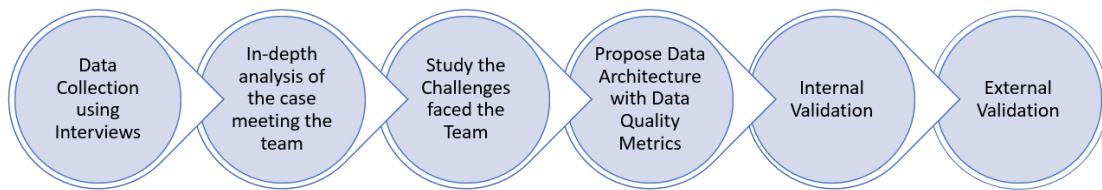


Figure 7.1: Research Methodology

7.3.2 Overview of use cases

Case A : Errors Data Pipeline

This company is a multinational in the information technology industry and has a significant presence in various technology sectors. One of its areas of specialization is Printers and Imaging Solutions, where it's a prominent player in the printing industry. They provide a variety of inkjet, laser, and multifunction printers designed for home and office use. The case at hand deals with managing error data from these printers.

Case B: ISP - Internet Service Provider

The company is an Internet Service Provider (ISP) telecommunications company that provides individuals and businesses with Internet connectivity. ISPs offer different technologies, such as broadband, wireless, and satellite connections, to facilitate data transmission between users and the global network of servers.

7.3.3 Exploratory Case Study

For this research, a qualitative approach was selected. This method involves gathering and analyzing non-numerical information, such as images, observations, and words. It is particularly useful for investigating individual experiences, perspectives, and motivations and gaining a comprehensive understanding of real-world situations [116]. When conducting an exploratory case study, the researcher might employ several data collection methods, including interviews, observations, and document analysis. The objective is to gather extensive and detailed information about the case to comprehend better the context and factors that impact it.

Data Collection

Interviews and meetings were conducted to study and explore data applications in companies [102]. A guide with 35 questions was used, categorized into three sections. The first section focused on background and experience, the second on data components and data quality metrics, and the last on best practices and obstacles. Interviews lasted 50-60 minutes and were recorded with permission. Data from meetings and discussions were also collected.

Data Analysis

The main contact point provided an interpretation of the Data Components. Analyzing the notes and codes from the transcripts gave a comprehensive view of different use cases, including the common parts across all cases. The first author used the collected data

Table 7.1: The Roles of the Experts

Company	Use cases	Experts Roles
Case A	Data Pipeline	Data Quality Engineer
		Big Data Architect
		Big Data Engineer
Case B	ISP	Big Data Team Lead
		Big Data Architect
		Big Data Engineer

and inputs to develop the first model for Data Architecture, which underwent several iterations for refinement and included data quality metrics.

By collaborating with productive practitioners, the author enhanced the data architecture model, encompassing quality matrices. The improved architecture model was meticulously designed by incorporating feedback from the author and the practitioners, analyzing the gathered data, and verifying its precision.

Validation Study

The model underwent internal and external validation, including interviews and feedback sessions with the case company teams. The practitioners' opinions and suggestions were considered for internal validation.

A telecommunication company was selected to undergo the external validation process to validate our findings further. Any outstanding issues will be addressed in future work.

7.4 Data Quality Challenges

It is critical to have high-quality data for systems that rely extensively on data, such as Machine Learning and Deep Learning. If these systems provide poor data, the result will also be poor. As a result, ensure the data is of the highest possible quality to obtain the greatest outcomes. In this section, insights into data Quality challenges based on the findings from our cases analysis mainly affect the quality of the data :

1. *Data inaccuracy* Several factors can contribute to data inaccuracies. One such factor is human error during the data entry process. Additionally, problems can arise

during the processing and integration of different data sets, leading to inconsistencies and inaccuracies.

2. *Missing Data* In data transmission, some data can be lost either partially or entirely. Fortunately, there are means of detecting where the loss occurred. However, retrieving the lost data can be a complex process. Unfortunately, the identification of missing data often happens towards the end of the data lifecycle, leading to suboptimal data products. This challenge is common across all use cases and can occur at different points in the data lifecycle.
3. *Duplicated Data* occur when a dataset contains multiple copies of the same data or data records, leading to inefficiency, confusion, and inaccuracies.
4. *Outdated Data* In data analysis, the importance of accurate and up-to-date information cannot be overstated. Data becoming outdated can result in misleading conclusions and flawed decision-making processes. This is because outdated data no longer reflects current affairs or trends and may contain inaccuracies or biases that have since been corrected. Therefore, it is crucial to ensure that data is regularly updated and refreshed to maintain relevance and usefulness.
5. *Validity Data Errors* Data that does not conform to predetermined rules, constraints, or standards for data integrity can lead to validity data errors. This type of error is also known as a validity or data validity error. It indicates that the data is "invalid" because it fails to meet the established criteria or requirements. These errors can manifest in various ways and arise at different stages of the data life cycle, Examples of Validity Data Errors.
6. *Consistency Data Errors* Minimizing any disparities, deviations, or discrepancies within a dataset or across multiple datasets is paramount. Such variations, commonly known as data consistency errors, can significantly negatively impact the accuracy and effectiveness of data analysis. Therefore, detecting these inconsistencies is critical in optimizing data quality and maximizing the value of its insights.

We also consider the **ProcessingType** to determine whether the data will be processed as a **Batch** or **Real-time**. To specify the data storage location, we incorporate various **DataStorageTech** such as NoSQL Databases (Document, Key-value, graph, and column), NewSQL Databases (Historical, Real-Time, Stream, Timestamp), and **File System** (HDF, GFS, AFS, GPFS and Blobseer). Furthermore, we use **Location** to indicate the data node's location, whether it is in the cloud or locally.

7.5.2 Behavioral Meta-model

NodeBehavior refers to the current status of a component that describes the data in a specific node. For instance, in a data node that generates data, we can find elements that define the source and format of the data. Every **NodeBehavior** has a set of behavioral elements that include actions and events, depicting the flow of data within the component. Data nodes can interact by passing data through data ports (**DataPort**). **InDataPort** is used to receive incoming data, while **OutDataPort** sends outgoing data. Figure 3 shows the actual communication methods of a message. In this context, a connection represents a unidirectional communication channel between two data ports of two components.

Actions in data nodes serve as fundamental behavioral components representing individual tasks. These tasks can be executed once a preceding step in the data flow has been achieved or when an event, such as receiving data, has been triggered. Figure 7.3 illustrates the direct actions of data behavior elements that describe data behavior within the node. For instance, **Generation** denotes the data source, while **Ingestion** outlines how data can be transferred from the source to the data lake. **Processes** comprise a set of sub-processes that can describe a complete processing node, while the **store** outlines the primary tasks to save, retrieve, archive, and govern data. **Analyze** describes the type or technique used to analyze data, while **Consume** shows how data can be consumed, such as through visualization, reports, and APIs. *Verifying data* is a critical step in the data life-cycle that involves checking for accuracy and completeness to ensure that it is valid and reliable. Our selection comprised the primary six data quality dimensions [91].

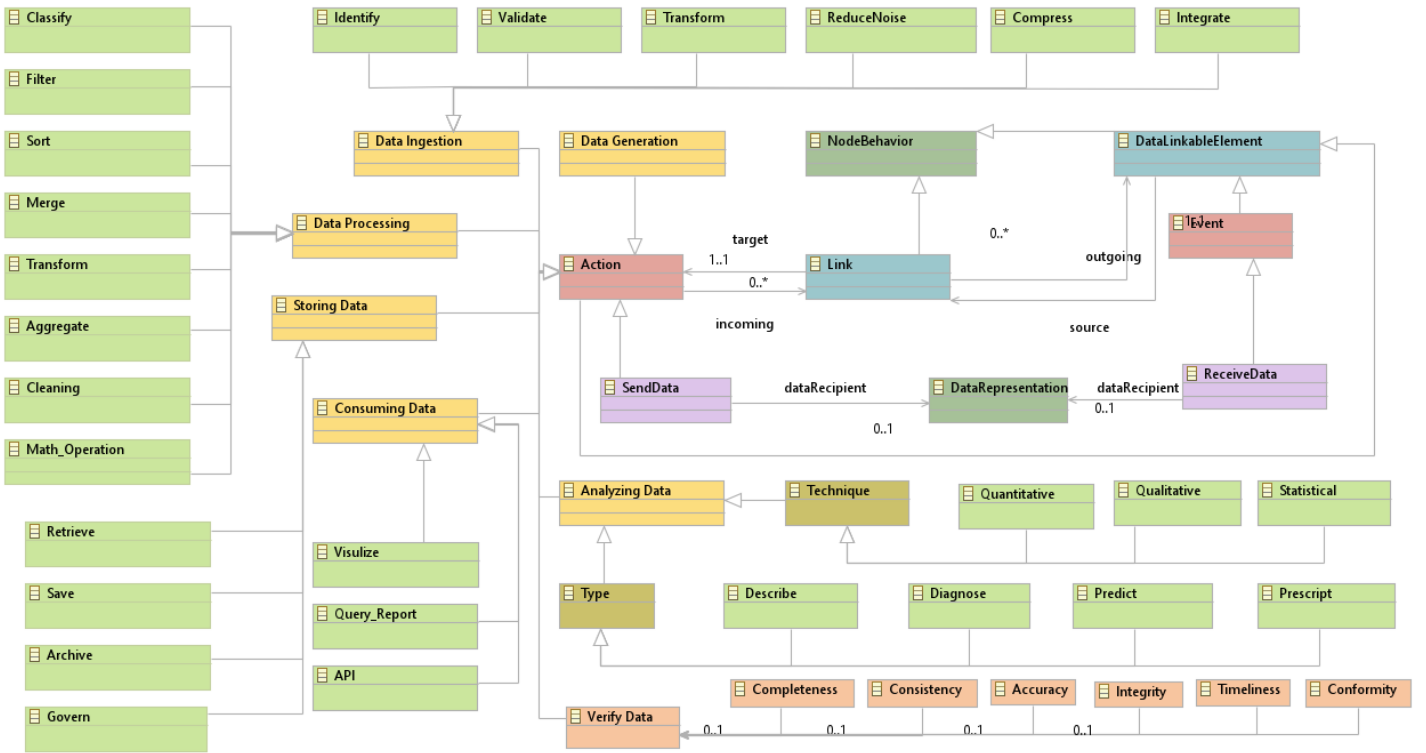


Figure 7.3: Quality Driven Meta-Model For Data Life Cycle

When a component receives an external stimulus, such as data on an input data port, it triggers an **Event**. **Links** are used to visualize the flow of data and connections between events and actions. These links can also determine the order in which steps should be taken and which must be executed immediately after an event.

7.6 Data Quality Code Generation

Our framework streamlines the process of conducting data quality checks and produces Python code that identifies potential issues with data quality, such as missing values, outliers, and incorrect data types. *GE* offers comprehensive features to define and execute *DQ* checks, and we utilize them to generate the code for these checks. The code generated can be easily integrated into existing data pipelines, allowing for a seamless assessment of data correctness and usefulness. By utilizing *DAFQA* and *GE*, you can significantly improve the accuracy and reliability of your data analysis, as depicted in Figure 8.1.

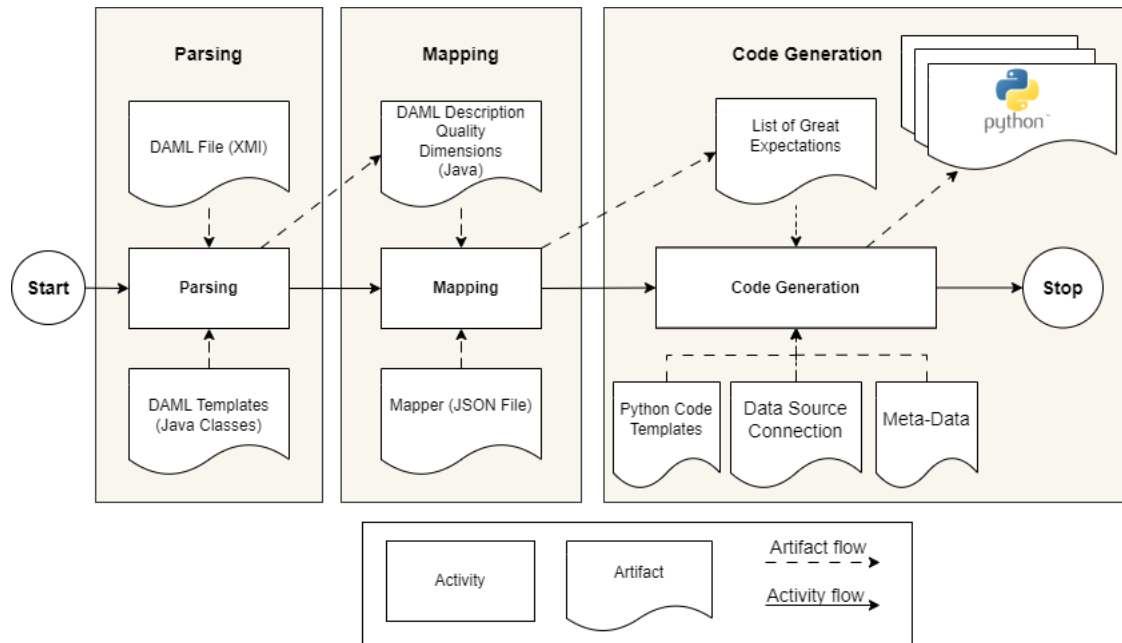


Figure 7.4: Code Generation Framework

7.6.1 Modeling

A "Verify Data" data action includes six data quality metrics. This action can be attached to any data source. Figure 7.3 shows the part of the behavioral meta-model.

7.6.2 Parsing

To extract the appropriate subset of DAML model values that adhere to the DAML meta-model, it is typical for the DAML model to be saved in an XMI (XML Metadata Interchange) file. Nevertheless, this file frequently includes various optional data that must be removed through filtering. When converting a DAML model into code, we must define templates for the different DAML sub-models like data nodes, connectors, ports, and data. These templates outline the structure and logic of the code, following the DAML meta-model. The parser scans the DAML XMI file and extracts relevant information based on defined templates. This involves generating objects for each data piece and a list of data quality characteristics that can be linked to any data source. The resulting data architecture object contains a DAML model description in Java, which

Table 7.2: Part Of Mapper

DQDs	Great Expectations
Uniqueness	expect_column_values_to_be_unique
Completeness	expect_column_values_to_not_be_null
	expect_column_values_to_be_null
Validity	expect_column_values_to_be_of_type
	expect_column_values_to_be_in_type_list
	expect_column_values_to_be_increasing
Consistency	expect_column_value_lengths_to_be_between
	expect_column_value_lengths_to_equal
	expect_column_values_to_match_regex
	expect_column_values_to_match_regex_list
Timeliness	expect_column_min_to_be_between
	expect_column_max_to_be_between
Accuracy	expect_column_values_to_be_in_set
	expect_column_values_to_not_be_in_set

can be utilized to generate code. Implementing code generators frequently follows the pattern of template-based code generating [117]. It entails creating templates that define the logic and code structure based on the meta-model and utilizing a parser to extract the information required to instantiate these templates from the input model. Accordingly, the data architecture object includes The Data Quality Dimensions (DQDs) related to each data source.

7.6.3 Mapping

Through the Mapper activity, we can effortlessly map the quality dimensions to expectations for each data source. This is made possible by the Great Expectations feature that provides a detailed list of expectations. The result is a comprehensive list of data sources, including their respective quality dimensions with mapped expectations, which can be used for code generation. The major six DQDs from the model are displayed in table A.1 along with the mapped expectation from the Great Expectations tool.

7.6.4 Generate Data Quality Check Code

This task is reliant on four specific items:

1. Output from earlier activities, a collection of data sources with associated expectations.
2. Code templates in Python.
3. The connection details for the data source.
4. The meta-data of the data source.

When generating code, we utilize a list of Python code templates. Connection details and metadata determine which data needs to undergo quality checks. This results in a list of expectations. Figure 8.4 displays an example of generated code that connects to a MySQL database using the table "userinfo" to obtain the batch of data needed to run the test; lines 22 to 26 demonstrate two expectations that will be applied to the "username" column ("expect_column_values_to_be_unique"). Runnable Python files that use the Great Expectation library to validate, document, and profile the data are the end product of the code creation activity. This helps the team communicate more effectively and maintains the quality of the data.

7.7 Results

Industry professionals have expertly validated proposed data architecture models to create a framework supporting exceptional data architecture. This blueprint is invaluable for organizations looking to efficiently organize and manage their data assets while ensuring top-notch quality. Through feedback from professionals, the framework has been tailored to meet real-world requirements and address unique industry challenges. Validation from experts further enhances the framework's credibility, making it an essential resource for maintaining high-quality data architecture.

To maximize opportunities in data management, one can either establish a **proactive design data architecture** or use the generated **Python code (Great Expectation) to monitor data quality** in any data pipeline efficiently. For the best outcome, combining both approaches is also a viable option.

```

1 import great_expectations as ge
2 from Entities.batches import Batch
3 from Entities.checkpoints import CheckPointConfig
4 from Entities.datasources import DataSource
5 from Entities.expectationUtil import ExpectationUtil
6 from Entities.expectationsuites import ExpectationSuite
7 # Get Context
8 context = ge.get_context()
9 if __name__ == '__main__':
10
11     # Data Source Code
12     dataSource=DataSource("root","root","127.0.0.1","3306","testdb",DataBaseType.MySQL)
13     context.add_datasource(**dataSource.getMySQLDataSourceConfig())
14
15     condition = ""
16     list_of_expectation_configuration=[]
17     # Table Name
18     tableName='userinfo'
19     #-----
20     # List of expectations for each column
21     #Expectations For Column [username]
22     expectation_configuration = ExpectationUtil.expect_column_values_to_be_unique("username")
23     list_of_expectation_configuration.append(expectation_configuration)
24
25     expectation_configuration = ExpectationUtil.expect_column_values_to_not_be_null("username")
26     list_of_expectation_configuration.append(expectation_configuration)
27
28     # add list of expectation configuration to test suite
29     suite = ExpectationSuite.create_expectation_suite(tableName,context ,list_of_expectation_configuration)
30     context.save_expectation_suite(suite)
31     list_of_expectation_configuration.clear()
32
33     # getting batch requests for the tables
34     batch_request = Batch.get_batch(dataSource.dataSchema, tableName, condition)
35
36     # running tests on tables
37     print(tableName, " ", batch_request)
38     CheckPointConfig.run_check_point_config(context,batch_request, tableName)

```

Figure 7.5: Example Of Generated Code

Figure 7.6 outlines the challenges of developing data-intensive applications and ensuring the required data quality dimensions are met during the code generation process.

For both cases, we engaged in the process of manually constructing Python code and integrating Great Expectations. Based on the evaluations conducted, it has been observed that:

- Embedding data quality dimensions right from the outset ensures consistent data quality maintenance throughout the entire data lifecycle, from data collection, storage, and processing to analysis and reporting. This proactive approach *prevents any possibility of error propagation*, thereby making sure that the data remains accurate and reliable at every stage.

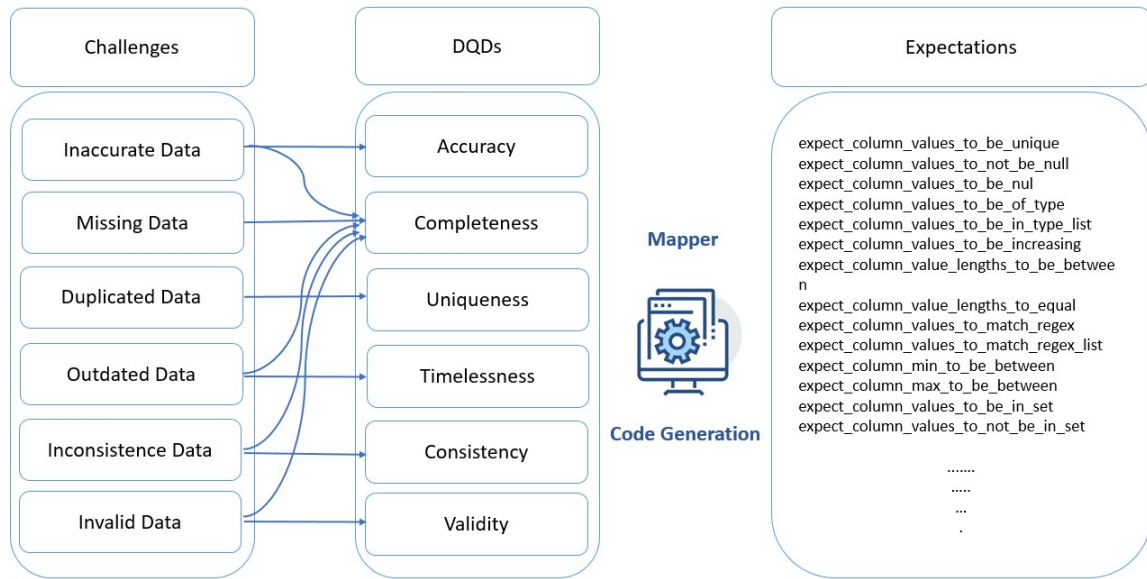


Figure 7.6: Challenges Mapped to DQDs and Great Expectations

- By implementing data quality measures at the architectural level, *proactive error detection* and resolution can be achieved. This allows for identifying and resolving data quality issues as they arise, which eliminates the need for complex and costly issue resolution after the fact.
- By addressing data quality at the source, organizations can greatly improve their efficiency and cut costs associated with correcting data errors. This approach ensures smooth operations and can have a substantial impact on reducing overall expenses.
- Organizations can rely on high-quality data at each stage of the lifecycle to ensure accurate analytics and informed decision-making. By instilling confidence in the insights derived from their data, businesses can make better decisions that positively impact their bottom line.
- Designing systems with an architectural approach to data quality is essential for supporting scalability and future growth. As data-intensive systems become more

complex and larger in scale, ensuring data quality can pose significant challenges. However, an architectural approach to data quality from the outset guarantees that systems are equipped to handle these challenges, making them highly adaptable and scalable to confidently meet any future needs.

7.8 Related Work

This section will examine studies pertinent to utilizing the most related research for the data quality framework for data-intensive applications.

DAFQA automates generating data quality checks for any data source, mapping them to predefined expectations from GE, generating runnable Python code, and monitoring data quality. It can also generate Data Docs reports and take customizable actions based on evaluation outcomes to save time and effort while ensuring high-quality data.

BIGQA [47] simplifies data quality assessment for experts and management specialists across domains and contexts. It generates tailored reports and runs smoothly on parallel or distributed computing frameworks. It uses simple operators for large datasets and facilitates incremental assessment to save time. BIGQA creates plans for assessing data quality, while DAFQA generates Python code for data quality that can be easily integrated into CI/CD processes. In [103] proposed A new method to measure dataset reliability introduces a "believability factor" by sampling a portion of the dataset and calculating execution time and Mean Absolute Error. DAFQA can handle all data formats and creates Python code to verify the six primary data quality dimensions. In [41] proposed a Big Data Quality Assessment Framework with 12 metrics, including four new ones: Integrity, Accessibility, Ease of Manipulation, and Security, and measured data quality with weights at three levels and five quality aspects for a macro view. In [107] proposed a Big Data Quality Management framework that works on the entire Big Data lifecycle. The framework relies on a Data Quality Profile enriched with valuable information as it progresses through various stages, such as Big Data project parameters, quality requirements, quality profiling, and quality rule proposals. DAFQA uses a meta-model that can model the entire data life cycle. This allows DAFQA to work on every stage

and storage, regardless of format.

7.9 Conclusion and Future Work

In This paper, we proposed the DAFQA, an architecture description and associated modeling framework for High-quality data-intensive applications. DAFQA can give a solution to many data management challenges like Data Silos. It provides a data quality code generation to monitor the quality that could integrated easily with any data pipeline. The DAFQA has been evaluated by two companies, with six practitioners in the evaluation phase.

This is an initial starting point for our future work plan, which can be extended to include. Data Contracts to ensure the provider can provide the quality of data for consumers based on the provided quality metrics.

Chapter 8

PyDaQu: Python Data Quality Code Generation Based on Data Architecture

Accurate and dependable data is critical when making crucial business decisions. However, verifying the accuracy of complex and extensive datasets can be both error-prone and time-consuming when done manually. We developed a PyDaQu, our automated framework that creates data quality checks code based on a standardized template. PyDaQu offers a variety of quality assurance measures, including validation, completeness, and consistency checks. These measures ensure exceptional data quality while simultaneously streamlining your data management processes. With PyDaQu, creating data quality checks requires significantly less time and effort. We have thoroughly evaluated PyDaQu using data from two different industry domains.

8.1 Introduction

The significance of ensuring high data quality cannot be overstated for data-driven applications that seek to thrive in today's data-driven business landscape [65] [60]. Reliable data is a critical asset that enables us to make well-informed decisions, boost operational

efficiency, enhance customer satisfaction, comply with regulations, manage risks, save costs, facilitate data integration, and maintain trust and credibility. Accurate data is essential in streamlining processes, improving customer experiences, mitigating compliance risks, avoiding unnecessary expenses, and building stakeholder trust [22].

Great Expectations GE[46], Deequ [68], and Dbt [32] are powerful data quality enhancement tools. However, configuring and setting up the environment can be time-consuming and prone to human error. Furthermore, utilizing their predefined methods (expectations, constraints, ...) may be challenging, requiring additional code writing.

This paper introduces *PyDaQu*¹ [3], a cutting-edge code generation framework that leverages the powerful *DAT* modeling framework, including data quality metrics. With *PyDaQu*, developers can easily convert their *DAT* models into Python code while incorporating the popular Great Expectation library. This results in a more streamlined workflow and the ability to efficiently produce high-quality code that meets their expectations.

Generating code for data quality checks automatically is a *time-saving* and highly efficient solution. It guarantees *consistency and standardization* across many datasets, thus enhancing accuracy and reliability. Promoting agility in adapting to changing data sources improves data governance. Moreover, it offers *documentation for traceability and compliance purposes*, streamlining the assessment process and enabling informed decisions based on reliable data.

The rest of this tool demo paper is organized as follows. The background is presented in Section 2. The related work is in Section 3. The Use Case of *PyDaQu* is described in Section 4. The *PyDaQu* tool, approach, and architecture are presented in Section 5. Evaluation is discussed in Section 6, while conclusions are drawn in Section 7.

8.2 Background

This section provides an overview of the *DAT* modeling framework and GE.

¹*Python Data Quality Code Generation Based on Data Architecture (PyDaQu)*

8.2.1 DAT Modeling Framework

DAT is a modeling tool for Data-Driven applications [4] [5] that helps visualize the flow of data through the system and create a blueprint for it. Stakeholders can use it to describe high-level (HLA) and low-level data architecture (LLA) [2], including formats, processing types, storage, analysis types, and consumption methods. The tool is based on a structural and behavioral meta-model and supports the understanding and documentation of data-driven applications; it includes metrics for data quality. It uses a modeling language called Data Architecture Structural and Behavioral View (DAML), designed according to the IEEE/ISO/IEC standard [58].

8.3 Related Work

This section will examine studies pertinent to utilizing the most related research for the *DQ* framework.

PyDaQu automates generating data quality checks for any data source, mapping them to predefined expectations from Great Expectations, generating runnable Python code, and monitoring data quality. It can also generate Data Docs reports and take customizable actions based on evaluation outcomes to save time and effort while ensuring high-quality data.

BIGQA [47] simplifies data quality assessment for experts and management specialists across domains and contexts. It generates tailored reports and runs smoothly on parallel or distributed computing frameworks. It uses simple operators for large datasets and facilitates incremental assessment to save time. BIGQA creates plans for assessing data quality, while PyDaQu generates Python code for data quality that can be easily integrated into CI/CD processes. In [103] proposed A new method to measure dataset reliability introduces a "believability factor" by sampling a portion of the dataset and calculating execution time and Mean Absolute Error. PyDaQu can handle all data formats and creates Python code to verify the six primary data quality dimensions.

In [107] proposed a Big Data Quality Management framework that works on the entire Big Data lifecycle. The framework relies on a Data Quality Profile enriched with

valuable information as it progresses through various stages, such as Big Data project parameters, quality requirements, quality profiling, and quality rule proposals. PyDaQu uses DAT and can model the entire data life cycle. This allows PyDaQu to work on every stage and storage, regardless of format.

8.4 Use Cases

This section provides a brief case description utilized by PyDaQu to test the generated code on actual industry data, these cases from companies B and C in section 3.3.

8.4.1 Case A: ISP - Internet Service Provider

As part of our research project, we carried out an investigation utilizing the raw data provided by an ISP Telecommunication company. The dataset featured a vast array of details about the internet usage patterns of their customers, all of which were stored in a complex relational database. Our experiment was conducted to check the quality of the data, which could be used to inform better decision-making strategies.

8.4.2 Case B: Errors Data Pipeline

This case outlines the data pipeline for error data from different printers. Files come in JSON format, including printer version, location, ink type, software version, and time. The data is saved on Amazon S3 and can be accessed using a query engine. After processing, the data is converted to parquet format, CSV, and relational database format for customer queries. We were able to run the quality check on raw data (same format of generation) and relational databases.

8.5 PyDaQu Framework

8.5.1 Approach

PyDaQu^{2 3} is a framework that automates the process of *DQ* checks and generates Python code to check for *DQ* issues such as missing values, outliers, and incorrect data types. Great Expectations is a tool that provides features to define and perform data quality checks, and *DAT* uses Great Expectations to generate the code for these checks. The *DAT* models are transformed into Great Expectations code, which can be integrated into existing data pipelines. By running this code, *DAT* can assess the correctness and usefulness of the data, providing insights into the quality of the data being analyzed. Overall, *DAT* and Great Expectations offer a powerful solution for automating data quality checks and improving the accuracy and reliability of data analysis.

8.5.2 Architecture

To fully comprehend the architecture of *PyDaQu*, it is crucial to examine each software component that comprises the system. Figure 8.1 offers an overview of the tool as an excellent reference point. In-depth explanations and demonstrations for each component will be provided to ensure a comprehensive understanding of *PyDaQu*.

DAT Model A new data action "Verify Data" has been added to ensure the model includes data quality metrics. This action can be attached to any data source. Figure 8.2 shows the part of the behavioral meta-model.

Parsing, The DAML model is typically stored in an XMI (XML Metadata Interchange) file, which contains various optional information that needs to be filtered out to extract a relevant subset of DAML model values conforming to the DAML meta-model.

To generate code from the DAML model, templates are defined for different DAML sub-models, such as data nodes and connectors, ports, and data. These templates define the code structure and logic based on the DAML meta-model.

The parser reads the DAML XMI file and extracts the necessary information based

²PyDaQu Tool Source Code : <https://github.com/khitam90/PyDaQu>

³PyDaQu Tool Demo Video : <https://youtu.be/697F6h0q7ss>

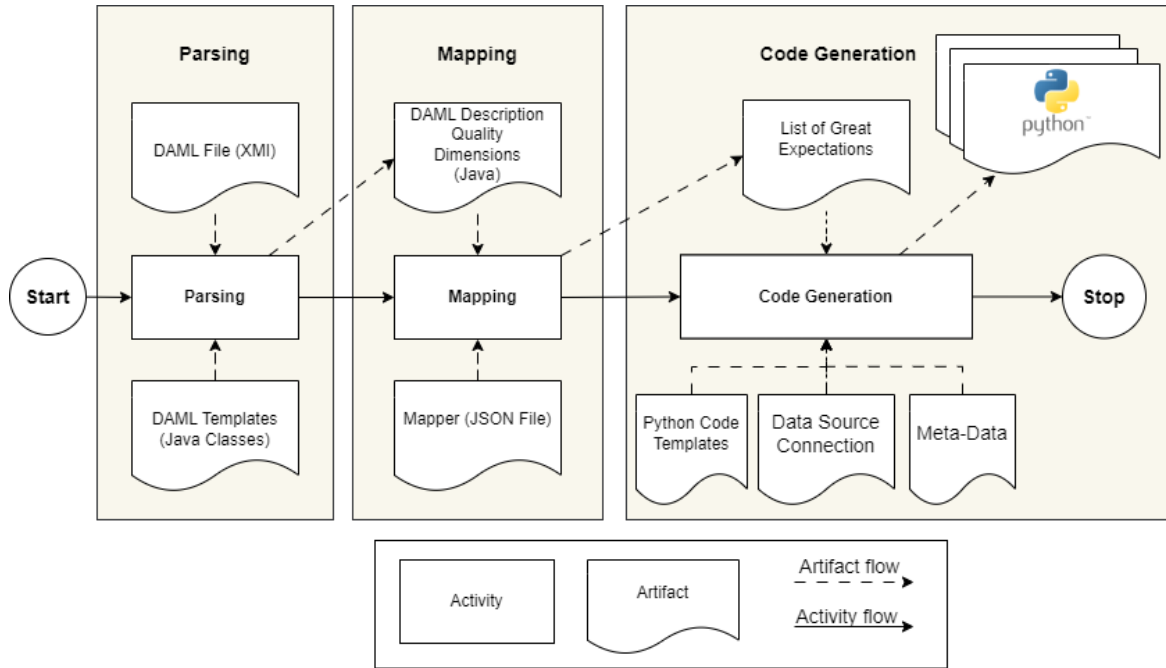


Figure 8.1: PyDaQu automatic code generation framework

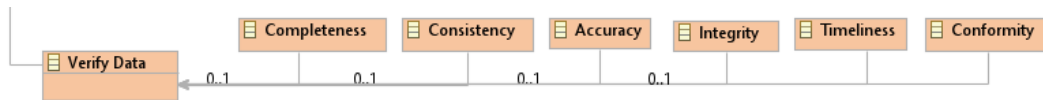


Figure 8.2: "Verify Data" Action

on the defined templates. This process involves creating objects representing each data element and a list of data quality dimensions linked to any data source. The resulting object of data architecture carries the DAML model description in Java, which can be used to generate code.

Template-based code generation is a common pattern in implementing code generators [117]. It involves defining templates that describe the code structure and logic based on the meta-model and using a parser to extract the necessary information from the input model to instantiate these templates. Developing a *coding template* involves creating reusable code snippets that can be saved as plain text files. The process includes identifying common patterns, crafting concise templates with placeholders, categorizing and organizing them, thoroughly testing, and keeping them up-to-date as necessary [106].

Accordingly, the resulting data architecture object contains the data quality dimen-

Table 8.1: Part Of Mapper

Quality Dimensions	Great Expectations
Uniqueness	expect_column_values_to_be_unique
Completeness	expect_column_values_to_not_be_null
	expect_column_values_to_be_null
Validity	expect_column_values_to_be_of_type
	expect_column_values_to_be_in_type_list
	expect_column_values_to_be_increasing
Consistency	expect_column_value_lengths_to_be_between
	expect_column_value_lengths_to_equal
	expect_column_values_to_match_regex
	expect_column_values_to_match_regex_list
Timeliness	expect_column_min_to_be_between
	expect_column_max_to_be_between
Accuracy	expect_column_values_to_be_in_set
	expect_column_values_to_not_be_in_set

sions linked to each data source.

Mapping, This activity inputs the data architecture object and the DAML model description in Java, resulting from the previous parsing activity. This Mapper represents the core section of the code generation process and is responsible for mapping quality dimensions to their corresponding expectations for each data source. Table A.1 shows the list of quality dimensions and corresponding expectations provided by GE.

Finally, the result of Mapper is a list of data sources defined with their list of quality dimensions and mapped predefined expectations (from Great Expectations). These objects will be an entry to the following code generation activity.

Generate Python Data Quality Checkpoints Code, This activity depends on four artifacts:

1. Previous activity's output, a list of data sources with related expectations.
2. Python code templates.
3. The connection details for the data source.
4. The meta-data of the data source.

During the code generation, we use the list of Python code templates. Every part of the code within braces "{}" will be replaced with values or generated code see Figure 8.3. We use the connection details and the meta-data to select which part of the data we need to run the quality checks (the list of expectations).

```

1 import great_expectations as ge
2
3 from Entities.batches import Batch
4 from Entities.checkpoints import CheckPointConfig
5 from Entities.datasources import DataSource
6 from Entities.expectationUtil import ExpectationUtil
7 from Entities.expectationsuites import ExpectationSuite
8 from Entities.databases import DataBaseType
9
10
11 # Get Context
12 context = ge.get_context()
13
14 if __name__ == '__main__':
15
16     # Data Source Code
17     dataSource=DataSource({UserName},{Password},{IPAddress},{Port},{DatabaseName},{DatabaseType})
18     context.add_datasource(**dataSource.{DatabaseConfigurationMethod}())
19
20     condition = ""
21     list_of_expectation_configuration=[]
22
23     {TableCode}
24
25     # List of expectations for each column
26     {ExpectationCode}
27
28     # add list of expectation configuration to test suite
29     suite = ExpectationSuite.create_expectation_suite(tableName,context ,list_of_expectation_configuration)
30     context.save_expectation_suite(suite)
31     list_of_expectation_configuration.clear()
32
33     # getting batch requests for the tables
34     batch_request = Batch.get_batch(dataSource.dataSchema, tableName, condition)
35
36     # running tests on tables
37     print(tableName, " ", batch_request)
38     CheckPointConfig.run_check_point_config(context,batch_request, tableName)
39

```

Rendered Code:

```

1 expectation_configuration = ExpectationUtil.{expectationMethod}({ExpectationParameter})
2 list_of_expectation_configuration.append(expectation_configuration)
...
condition):
...
ed to a Datasource
...
ata that we want to test.
...
rce",
...
ime_data_connector_name",
...
table_name)",
...
LECT * from {schema_name}.{table_name} {condition}",
...
ifier_name": "default_identifier",
...
emp_table": False}

```

Callout Box:

```

1 expectation_configuration = ExpectationUtil.{expectationMethod}({ExpectationParameter})
2 list_of_expectation_configuration.append(expectation_configuration)

```

Figure 8.3: Templates Example

Figure 8.4, shows an example of generated code that connects to MySQL database, using table "userinfo" to get the batch of data to run the test; lines 22 show two of expectations ("expect_column_values_to_be_unique") will be applied to "username" column.

The results of code generation activity are runnable Python files that use the GE library to validate, document, and profile the data to maintain quality and improve team communication. Figure 8.5 shows the results of running the generated Python files.


```

1  import great_expectations as ge
2  from Entities.batches import Batch
3  from Entities.checkpoints import CheckPointConfig
4  from Entities.datasources import DataSource
5  from Entities.expectationUtil import ExpectationUtil
6  from Entities.expectationsuites import ExpectationSuite
7  # Get Context
8  context = ge.get_context()
9  if __name__ == '__main__':
10
11     # Data Source Code
12     dataSource=DataSource("root","root","127.0.0.1","3306","testdb",DataBaseType.MySQL)
13     context.add_datasource(**dataSource.getMySQLDataSourceConfig())
14
15     condition = ""
16     list_of_expectation_configuration=[]
17     # Table Name
18     tableName='userinfo'
19     #-----
20     # List of expectations for each column
21     #Expectations For Column [username]
22     expectation_configuration = ExpectationUtil.expect_column_values_to_be_unique("username")
23     list_of_expectation_configuration.append(expectation_configuration)
24
25     expectation_configuration = ExpectationUtil.expect_column_values_to_not_be_null("username")
26     list_of_expectation_configuration.append(expectation_configuration)
27
28     # add list of expectation configuration to test suite
29     suite = ExpectationSuite.create_expectation_suite(tableName,context ,list_of_expectation_configuration)
30     context.save_expectation_suite(suite)
31     list_of_expectation_configuration.clear()
32
33     # getting batch requests for the tables
34     batch_request = Batch.get_batch(dataSource.dataSchema, tableName, condition)
35
36     # running tests on tables
37     print(tableName, " ", batch_request)
38     CheckPointConfig.run_check_point_config(context,batch_request, tableName)

```

Figure 8.4: Example Of Generated Code

8.6 Evaluation

Our recent publication [4] encompasses a thorough assessment of DAT in diverse domains. We have included one case study from our previous research and another from the telecommunications industry to enhance our argument. The cases were selected based on the data available to execute the code produced by PyDaQu to check its effectiveness.

For both cases, we engaged in the process of manually constructing Python code and integrating Great Expectations. After conducting a thorough evaluation, we found a significant difference in the time required to complete a data quality check when comparing manual Python coding versus utilizing PyDaQu to create and execute the code.

acctstoptime		
Search		
Status	Expectation	Observed Value
✖	values must never be null. 452 unexpected values found. ≈0.001483% of 30480211 total rows. Sampled Unexpected Values null	≈99.999% not null

Figure 8.5: Results of running the generated code

Table 8.2: Comparing the Time Required for Manual and Automated Code

Cases	Engineer Level	Manual	PyDaQu
Case A	Fresh	5 days	10 minutes
	Senior	3 days	
Case B	Senior	3 days	

The table 8.2 shows the estimated time required to manually generate the code based on the engineer’s seniority level. Seniors are expected to complete the task within three days, while juniors may require up to five days. On the other hand, it takes only ten minutes to generate and run the code using PyDaQu.

Utilizing code generation templates is a practical approach to building code consistently aligned with the established coding standards. This practice enables developers to streamline their coding processes and produce high-quality code in a structured and systematic manner. By following these templates, developers can ensure that their code is error-free, optimized for performance, and easily maintainable. This ultimately leads to faster development cycles, improved code quality, and enhanced software reliability.

8.7 Conclusion

In this tool demonstration paper, we have presented PyDaQu, an excellent Python code generator that employs the GE Library to assess data quality. PyDaQu proves to be an

indispensable tool for engineers, as it helps them save time while developing data-checking code, regardless of their level of expertise. The generated code seamlessly integrates with any data pipeline, which makes it a valuable asset for teams working with data. Extensive testing on real-world data across various domains has demonstrated PyDaQu's efficacy in delivering reliable and accurate evaluations. PyDaQu sets the benchmark for quality data assessment and is a must-have tool for anyone working with data. Engineers can confidently create high-quality, error-free code efficiently and systematically by utilizing code generation templates. This approach results in faster development, better code, and more reliable software.

Chapter 9

Results And Evaluation

In this section, we aim to demonstrate the effectiveness of our proposed approach from two distinct angles. Firstly, we will compare our approach to existing studies to highlight our novel approach's unique advantages and necessity. Secondly, we will present the industry feedback to showcase how our approach aligns perfectly with the needs and requirements of the industry.

9.1 Comparative to the existing approaches

In section 1.4, we thoroughly examined various existing studies for engineering data-intensive systems, including modeling, development, and analysis. However, none of these approaches provided a comprehensive modeling solution to the challenges of designing, analyzing, and developing high-quality data-intensive systems. That's where DAT comes in. Our proposed approach offers a novel solution to these challenges and is specifically designed to address the shortcomings of existing techniques. Our evaluation conclusively establishes that DAT is a novel approach superior to existing approaches.

Tables 9.1 and 9.2 show the comparative findings from the assessment related to DAT Modeling features and engineering data-intensive systems and present the following features.

Table 9.1: Comparative table on supporting different engineering capabilities

Related Works	Abstract Level	Graphical Tool	Code Generation	Analysis	Empirical Assessment
					Approach
Accordant [24]	High	No	Yes	Quality of Service (Update time, latency, Deadline)	Proof of concept and a case study
DICE [51] [12] [84]	High	No	Yes	Anomalies detection in real-time data streams	Proof of concept and a case study
Gribaudo [50]	High	No	No	Performance of running applications	Proof of concept and Experiment
Raj and Bosh [89]	High	No	No	No	Proof of concept and multiple cases study
Erraissi [44] [43]	Low	No	No	No	Proof of concept
Bashir [16]	High	No	No	No	Proof of concept and multiple cases study
DAT	High & Low	Yes	Yes	Six DQDs	Proof of concept and multiple cases study

9.1.1 Supporting the engineering of data-intensive systems

In this section, we evaluate the selected data-intensive approaches based on their ability to support various engineering tasks in developing, analyzing, and deploying data-intensive systems. The assessment results are summarized in Table 9.1. The following section presents several exciting findings related to the engineering support provided by DAT.

1. All selected studies provide High-Level Architecture, while DAT provides both high and low levels of architecture. Erraissi [44] [43] is the only study that provides low-level architecture.
2. It is evident that none of the selected studies offer a graphical tool, whereas DAT is the only framework that provides a graphical modeling tool.
3. DIC [51] [12] [84] provides a code for real-time data monitoring. Accordant [24] generates functional Java code, which serves as a Spark driver program. DAT

generates Python code that utilizes the Great Expectations Library to check data quality dimensions in any data source.

4. DAT analyzes six data quality dimensions: completeness, consistency, accuracy, integrity, timeliness, and conformity. Accordant focuses on quality of service, including update time, latency, and deadline. DIC provides performance, reliability, and monitoring data analysis.

9.1.2 Supporting the modeling of data-intensive systems

This section compares different modeling approaches for data-intensive applications. The findings from the assessment presented in Table 9.2 are compared to the modeling features supported by DAT, which will be detailed in the next section:

1. Most approaches don't support storage type modeling except DIC, which has limited database support. It's crucial to model all storage types and keep varying data formats and speeds in mind. DAT includes all storage types in its modeling framework.
2. Monitoring data formats is crucial as it moves from source to destination. Data is generated in various formats and needs to be converted into a unified form. DIC supports limited data formats, while DAT classifies them into three categories: structured, semistructured, and unstructured.
3. Previous studies on data life cycle modeling have focused on specific phases rather than modeling the entire life cycle. Meta-models have been proposed for certain phases, such as Generation, Ingestion, Visualization, Sink, Estimation, and Transformation. DAT has developed an environment that allows for modeling all phases and their sub-details.

It is clear that the current literature on modeling data architecture is limited and incomplete, with some phases of the data life cycle, data formats, storage types, and data processing speeds being overlooked. However, DAT provides a comprehensive and

efficient solution to bridge this gap. DAT ensures a smooth and successful data journey with its advanced approach and fast-generated data quality checks.

Table 9.2: Comparative table on supporting different data-intensive modeling features

Related Works	Storage Type	Data Format	Data Life Cycle		Main Focus
			Main Phases	Details	
Accordant [24]	No	No	Partially	No	Designing and deploying big data analytics (BDA)
DICE [51] [12] [84]	Basic	Basic	Partially	No	Modeling Big Data Tech. (Apache Hadoop, MapReduce and Apache Storm)
M. Gribaudo [50]	No	No	No	No	modeling framework for performance evaluation of systems based on lambda architecture
A. Raj, J. Bosh [89]	No	No	Yes	No	A conceptual model of the data pipeline
A. Erraissi [44] [43]	No	No	Partially	Yes	Meta-model for specific stages of the data life cycle
M. Bashir [16]	No	No	Partially	No	Proposed a metamodel for Big Data Management and Analytics
DAT	Yes	Yes	Yes	Yes	A framework offers system and software design, cutting-edge code generation, and data quality analysis for data-intensive systems.

9.2 Supporting High-quality data-intensive applications

This section demonstrates how to deliver high-quality applications from the data architecture to the final implementation. First: modeling high-quality architecture. Second: Using model-to-text transformation using PyDaQu to generate data quality checks code.

- With DAT, we can offer a comprehensive and well-structured data architecture solution to tackle common data architecture challenges like data silos in new or legacy systems. Our DAT is designed to address these challenges effectively, ensuring seamless and secure data integration across all your systems.

- Managing data efficiently and competently is vital to any organization. Every data lifecycle stage must be handled with utmost care, from creation to consumption. This requires establishing clear policies and procedures for data lifecycle management. At DAT, we provide a holistic framework that helps maintain the quality dimensions for every phase of the data lifecycle.
- It's possible to create data quality checks for any data source at any point in the data lifecycle. This innovative method is significantly faster than the traditional approach, taking only a third of the time and requiring fewer resources. So, with PyDaQu, you can ensure the quality of your data more efficiently and effectively.

9.3 Industry Evaluation

Industrial Evaluations are a crucial step towards developing an effective tool for industries. With our focus on assessing the accuracy of DAT in describing real-world industrial scenarios, we involve companies in the evaluation process to gather their feedback. Our efforts have led to the successful modeling of five distinct cases for various companies. Thanks to the valuable feedback from these companies, we have improved our tool and made it more efficient for industrial usage.

9.3.1 Evaluating The Modeled Cases

The *versatility and adaptability* of the framework in question have been proven through various real-case scenarios. Each case represents a unique set of data management challenges, from data formats and processing types to storage strategies and lifecycle phases. Table 9.3 illustrates that the framework can provide architectural solutions that include structured, semi-structured, and unstructured data. Additionally, the framework's versatility is demonstrated by its ability to handle real-time and batch-processing methods, making it suitable for different data types and operational requirements. Furthermore, the framework provides comprehensive support for various storage strategies, including NoSQL, NewSQL, and traditional file systems, which enhances data retrieval and man-

Table 9.3: Real Case Evaluation: Main Components

Cases	Data Formats	Processing Type	Storage Type	Data Life-cycle Phases	Case Complexity
A	Structured, Semi-structured	RealTime Batch	NoSQL NewSQL File System	Generation, Ingestion Processing, Storing Analyze, Visualize Share	4 (48)
B	Structured, Semi-structured	RealTime Batch	NoSQL NewSQL File System	Generation, Ingestion Processing, Storing Analyze, Visualize Share	9 (72)
C	Structured, Semi-structured Unstructured	RealTime Batch	NoSQL NewSQL File System	Generation, Ingestion Processing, Storing Analyze, Visualize	7 (67)
D	Semi-structured	Batch	NoSQL NewSQL File System	Generation, Ingestion Processing, Storing Visualize	5 (34)
E	Structured, Un-structured Unstructured	RealTime Batch	NoSQL NewSQL File System	Generation, Ingestion Processing, Storing Analyze, Visualize Share	8 (75)

agement.

DAT can ensure a holistic approach towards data management throughout its entire lifecycle - from creation to sharing. It is designed to tackle challenges of varying complexities, as demonstrated by the complexity scores in table 9.3. This indicates our framework's scalability and robustness.

9.3.2 Evaluating Addressed Challenges

With DAT, you can rest easy knowing there is a compelling architectural solution to tackle DA challenges. By leveraging the powerful features of DAT, you can address the key challenges associated with data architecture, as outlined in the previous section (related to the real case study). After conducting an in-depth analysis of various industrial cases, It became evident that utilizing the DAT can provide significant advantages to organizations. DAT can provide:

- Complex and scalable architecture solutions that could help model multiple data

sources with different formats, speeds, and stages of the data life cycle, including different storage types and locations (see table 9.3).

- The Hyder case is a perfect example of how batch and real-time processing can be integrated using Lambda architecture. By modeling this case, we can understand the significance of utilizing both approaches and how they can enhance the overall data processing capabilities.
- A common language, organizations can establish a standardized framework to facilitate seamless communication and comprehension of data assets by implementing DAT. This enables teams across regions to share and interpret data easily, promoting collaboration and improving outcomes.
- Data quality, The key to ensuring high-quality data lies in establishing rigorous *standards and procedures* for data collection, management, and usage. Additionally, implementing quality checks can help ensure that data is accurate, complete, and consistent. By running *data quality checkpoints*, you can automate processes and reduce the need for manual data processing. This can improve operational efficiency and productivity, saving valuable time and effort in development.

In conclusion, the framework's proven *robustness and flexibility* in designing complex data environments make it a powerful solution for modern data-intensive industries. Its comprehensive approach to data format diversity, processing requirements, storage systems, and lifecycle management phases ensures that data is stored, processed, and presented efficiently, meeting the multifaceted demands of today's industries.

Chapter 10

Concluding Remarks and Future Work

This research examines data architecture practices, data-intensive models, and quality monitoring. Its primary objective is to identify the challenges faced in developing data-intensive systems and propose an innovative approach to modeling and monitoring high-quality data-intensive applications.

This thesis outlines the research on data architecture and quality for data-intensive systems. The primary focus of the research was on the challenges related to data architecture and the development of a robust data architecture framework, which is a crucial step in creating data-intensive applications. Through an exploratory case study, the key challenges at each stage of the data architecture were identified.

Our research has brought to light the importance of enhancing data architecture practices. This realization has led us to develop a comprehensive framework for modeling data architecture. The framework offers a standardized protocol and specialized terminology for data architecture to facilitate effective communication among teams within and among organizations.

We created a state-of-the-art code generation framework for ensuring data quality in data-intensive applications. This framework effectively tackles the challenges that arise during the implementation and maintenance of such applications.

Finally, We validated our work by testing it on an industrial case from a different domain company. Our methods improved data quality during modeling and code generation.

10.1 Key contributions

The key contributions of this thesis are listed below.

Objective 1: To model data-intensive applications

- Identification of the key challenges related to data architecture during the development of data-intensive models.
- Identify the key elements for Data Architecture.
- Defines a meta-model for Data Architecture data-intensive applications.
- Develop graphical model language on top of the meta-model.
- Provide real industry cases as an evaluation for the meta-mode.

Objective 2: To facilitate the monitoring of data quality

- Identification of the key challenges related to DQ.
- Enhance the meta-model to contain DQDs.
- Develop code generation framework to monitor the DQ.

10.2 Future Work

In this dissertation, we explore the topic of data quality from two different perspectives. Firstly, we develop a model that incorporates various quality dimensions. Secondly, we integrate these dimensions into the code generation process to enable the monitoring of data quality. Our research emphasizes the importance of maintaining high-quality data in both business and academic settings and proposes a solution-oriented approach to address the challenges that arise.

Our future work aims to streamline the process of communicating data details and quality requirements from data consumers to data providers. Our solution will ensure that the provided data meets the necessary quality standards, thereby benefiting all parties involved.

Bibliography

- [1] Evolution of data management. *Computer*, 29(10):38–46, 1996.
- [2] M. Abughazala and H. Muccini. Modeling data analytics architecture for iot applications using dat. In *2023 IEEE 20th International Conference on Software Architecture Companion (ICSA-C)*, pages 284–291, 2023.
- [3] M. Abughazala, H. Muccini, and K. Qadri. Pydaqu: Python data quality code generation based on data architecture. In *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, pages 60–64. IEEE, 2023.
- [4] M. Abughazala, H. Muccini, and M. Sharaf. Dat: Data architecture modeling tool for data-driven applications. In *European Conference on Software Architecture*, pages 90–101. Springer, 2022.
- [5] M. Abughazala, H. Muccini, and M. Sharaf. Architecture description framework for data-intensive applications. In *2023 Fourth International Conference on Intelligent Data Science Technologies and Applications (IDSTA)*, pages 99–106. IEEE, 2023.
- [6] A. F. Ahmad, M. S. Sayeed, C. P. Tan, K. G. Tan, M. A. Bari, and F. Hossain. A review on iot with big data analytics. In *2021 9th International Conference on Information and Communication Technology (ICoICT)*, pages 160–164. IEEE, 2021.
- [7] B. Alabdullah, N. Beloff, and M. White. Rise of big data—issues and challenges. In

- 2018 21st Saudi Computer Society National Computer Conference (NCC), pages 1–6. IEEE, 2018.
- [8] A. Alexandru, C. Alexandru, D. Coardos, and E. Tudora. Big data: concepts, technologies and applications in the public sector. *Int J Comput Electr Autom Control Inform Eng*, 10:1629–35, 2016.
- [9] F. Almeida and C. Calistru. The main challenges and issues of big data management. 2013.
- [10] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 291–300. IEEE, 2019.
- [11] C. A. Ardagna, V. Bellandi, M. Bezzi, P. Ceravolo, E. Damiani, and C. Hebert. Model-based big data analytics-as-a-service: take big data to the next level. *IEEE Transactions on Services Computing*, 14(2):516–529, 2018.
- [12] M. Artac, T. Borovšak, E. Di Nitto, M. Guerriero, D. Perez-Palacin, and D. A. Tamburri. Infrastructure-as-code for data-intensive architectures: a model-driven development approach. In *2018 IEEE international conference on software architecture (ICSA)*, pages 156–15609. IEEE, 2018.
- [13] P. Azad, N. J. Navimipour, A. M. Rahmani, and A. Sharifi. The role of structured and unstructured data managing mechanisms in the internet of things. *Cluster computing*, 23(2):1185–1198, 2020.
- [14] B. Balusamy, S. Kadry, A. H. Gandomi, et al. *Big Data: Concepts, Technology, and Architecture*. John Wiley and Sons, 2021.
- [15] K. L. Barriball and A. While. Collecting data using a semi-structured interview: a discussion paper. *Journal of Advanced Nursing-Institutional Subscription*, 19(2):328–335, 1994.

- [16] M. R. Bashir, A. Q. Gill, G. Beydoun, and B. Mccusker. Big data management and analytics metamodel for iot-enabled smart buildings. *IEEE Access*, 8:169740–169758, 2020.
- [17] S. E. Bibri. The anatomy of the data-driven smart sustainable city: instrumentation, datafication, computerization and related applications. *Journal of Big Data*, 6(1):1–43, 2019.
- [18] R. Bogdan and S. K. Biklen. *Qualitative research for education*. Allyn & Bacon Boston, MA, 1997.
- [19] F. Borelli, G. Biondi, F. Horita, and C. Kamienski. Architectural software patterns for the development of iot smart applications. *arXiv preprint arXiv:2003.04781*, 2020.
- [20] J. Bosch, H. H. Olsson, and I. Crnkovic. Engineering ai systems: A research agenda. *Artificial Intelligence Paradigms for Smart Cyber-Physical Systems*, pages 1–19, 2021.
- [21] C. Szyperski. *Component Software. Beyond Object Oriented Programming*. Addison Wesley, 1998.
- [22] L. Cai and Y. Zhu. The challenges of data quality and data quality assessment in the big data era. *Data science journal*, 14, 2015.
- [23] C. Castellanos, B. Pérez, C. A. Varela, M. d. P. Villamil, and D. Correal. A survey on big data analytics solutions deployment. In *Software Architecture: 13th European Conference, ECSCA 2019, Paris, France, September 9–13, 2019, Proceedings 13*, pages 195–210. Springer, 2019.
- [24] C. Castellanos, C. A. Varela, and D. Correal. Accordant: A domain specific-model and devops approach for big data analytics architectures. *Journal of Systems and Software*, 172:110869, 2021.

- [25] J. M. Cavanillas, E. Curry, and W. Wahlster. *New horizons for a data-driven economy: a roadmap for usage and exploitation of big data in Europe*. Springer Nature, 2016.
- [26] C. P. Chen and C.-Y. Zhang. Data-intensive applications, challenges, techniques and technologies: A survey on big data. *Information sciences*, 275:314–347, 2014.
- [27] H.-M. Chen, R. Kazman, and S. Haziyevev. Agile big data analytics for web-based systems: an architecture-centric approach. *IEEE Transactions on Big Data*, 2(3):234–248, 2016.
- [28] D. Coghlan. Action research: Exploring perspectives on a philosophy of practical knowing. *Academy of Management Annals*, 5(1):53–87, 2011.
- [29] I. D. Corporation. How idc’s industry cloudpath and saaspath surveys can inform your cloud/saas strategy, 2019.
- [30] N. Cowie. Observation. *Qualitative research in applied linguistics: A practical introduction*, pages 165–181, 2009.
- [31] D. S. Cruzes and T. Dyba. Recommended steps for thematic synthesis in software engineering. In *2011 international symposium on empirical software engineering and measurement*, pages 275–284. IEEE, 2011.
- [32] DBT. Data build tool, 2022.
- [33] J. Densmore. *Data Pipelines Pocket Reference*. " O’Reilly Media, Inc.", 2021.
- [34] I. Dey. *Qualitative data analysis: A user friendly guide for social scientists*. Routledge, 2003.
- [35] S. Easterbrook, J. Singer, M.-A. Storey, and D. Damian. Selecting empirical methods for software engineering research. *Guide to advanced empirical software engineering*, pages 285–311, 2008.
- [36] I. Eclipse Foundation. Eclipse epsilon, 2009.

- [37] I. Eclipse Foundation. Eclipse modeling framework, 2009.
- [38] I. Eclipse Foundation. Graphical model editor development with eugenia/gmf, 2009.
- [39] O. Elijah, T. A. Rahman, I. Orikumhi, C. Y. Leow, and M. N. Hindia. An overview of internet of things (iot) and data analytics in agriculture: Benefits and challenges. *IEEE Internet of things Journal*, 5(5):3758–3773, 2018.
- [40] A. Elomari, A. Maizate, and L. Hassouni. Data storage in big data context: A survey. In *2016 Third International Conference on Systems of Collaboration (SysCo)*, pages 1–4. IEEE, 2016.
- [41] W. Elouataoui, I. El Alaoui, S. El Mendili, and Y. Gahi. An advanced big data quality framework based on weighted metrics. *Big Data and Cognitive Computing*, 6(4):153, 2022.
- [42] D. Engineers. Data architecture, 2018.
- [43] A. Erraissi and A. Belangour. Data sources and ingestion big data layers: meta-modeling of key concepts and features. *International Journal of Engineering and Technology*, 7(4):3607–3612, 2018.
- [44] A. Erraissi, B. Mouad, and A. Belangour. A big data visualization layer meta-model proposition. In *2019 8th International Conference on Modeling Simulation and Applied Optimization (ICMSAO)*, pages 1–5. IEEE, 2019.
- [45] G. Expectations. Great expectations, 2021.
- [46] G. Expectations. Great expectations, 2021.
- [47] H. Fadlallah, R. Kilany, H. Dhayne, R. e. Haddad, R. Haque, Y. Taher, and A. Jaber. Bigqa: Declarative big data quality assessment. *ACM Journal of Data and Information Quality*, 2023.
- [48] I. Gartner. How to improve your data quality, 2021.

- [49] A. Gillet, É. Leclercq, and N. Cullot. Lambda+, the renewal of the lambda architecture: Category theory to the rescue. In *International Conference on Advanced Information Systems Engineering*, pages 381–396. Springer, 2021.
- [50] M. Gribaudo, M. Iacono, and M. Kiran. A performance modeling framework for lambda architecture based applications. *Future Generation Computer Systems*, 86:1032–1041, 2018.
- [51] M. Guerriero, S. Tajfar, D. A. Tamburri, and E. Di Nitto. Towards a model-driven design tool for big data architectures. In *Proceedings of the 2nd international workshop on BIG data software engineering*, pages 37–43, 2016.
- [52] K. F. Hyde. Recognising deductive processes in qualitative research. *Qualitative market research: An international journal*, 3(2):82–90, 2000.
- [53] IBM. What is a data architecture?, 2018.
- [54] W. Inmon, D. Linstedt, and M. Levins. Chapter 1.1 - an introduction to data architecture. In W. Inmon, D. Linstedt, and M. Levins, editors, *Data Architecture (Second Edition)*, pages 1–5. Academic Press, second edition edition, 2019.
- [55] W. Inmon, D. Linstedt, and M. Levins. Chapter 1.6 - the life cycle of data: Understanding data over time. In W. Inmon, D. Linstedt, and M. Levins, editors, *Data Architecture (Second Edition)*, pages 33–37. Academic Press, second edition edition, 2019.
- [56] W. Inmon, D. Linstedt, and M. Levins. *Data Architecture: A Primer for the Data Scientist: A Primer for the Data Scientist*. Academic Press, 2019.
- [57] D. International. *DAMA-DMBOK: Data Management Body of Knowledge (2nd Edition)*. Technics Publications, LLC, Denville, NJ, USA, 2017.
- [58] ISO/IEC/IEEE. ISO/IEC/IEEE 42010:2022 Systems and software engineering – Architecture description, 2022.

- [59] S. Jesilevska. Data quality dimensions to ensure optimal data quality. *Romanian Economic Journal*, 20(63), 2017.
- [60] S. Ji, Q. Li, W. Cao, P. Zhang, and H. Muccini. Quality assurance technologies of big data applications: A systematic literature review. *Applied Sciences*, 10(22):8052, 2020.
- [61] S. John Walker. Big data: A revolution that will transform how we live, work, and think, 2014.
- [62] G. K. Kalipe and R. K. Behera. Big data architectures: A detailed and application oriented review. *Int. Journal Innov. Technol. Explor. Eng*, 8:2182–2190, 2019.
- [63] F. Kalna and A. Belangour. A meta-model for diverse data sources in business intelligence. *American Journal of Embedded Systems and Applications*, 7(1):1–8, 2019.
- [64] A. Karahasanoviæ, B. Anda, E. Arisholm, S. E. Hove, M. Jørgensen, D. I. Sjøberg, and R. Welland. Collecting feedback during software engineering experiments. *Empirical Software Engineering*, 10:113–147, 2005.
- [65] A. Karkouch, H. Mousannif, H. Al Moatassime, and T. Noel. Data quality in internet of things: A state-of-the-art survey. *Journal of Network and Computer Applications*, 73:57–81, 2016.
- [66] M. Kiran, P. Murphy, I. Monga, J. Dugan, and S. S. Baveja. Lambda architecture for cost-effective batch and speed big data processing. In *2015 IEEE International Conference on Big Data (Big Data)*, pages 2785–2792. IEEE, 2015.
- [67] M. Kleppmann. Designing data-intensive applications, 2019.
- [68] A. Labs. Deequ - unit tests for data, 2018.
- [69] V. A. Lambert and C. E. Lambert. Qualitative descriptive research: An acceptable design. *Pacific Rim international journal of nursing research*, 16(4):255–256, 2012.

- [70] L. Liu and L. Chi. Evolutional data quality: A theory-specific view. In *ICIQ*, pages 292–304, 2002.
- [71] L. E. Lwakatare, A. Raj, J. Bosch, H. H. Olsson, and I. Crnkovic. A taxonomy of software engineering challenges for machine learning systems: An empirical investigation. In *Agile Processes in Software Engineering and Extreme Programming: 20th International Conference, XP 2019, Montréal, QC, Canada, May 21–25, 2019, Proceedings 20*, pages 227–243. Springer International Publishing, 2019.
- [72] B. R. Maxim, M. Galster, I. Mistrik, and B. Tekinerdogan. *Data-intensive systems, knowledge management, and software engineering*. CRC Press Boca Raton, FL, 2021.
- [73] J. Maxwell, L. Bickman, and D. Rog. The sage handbook of applied social research methods. *SAGE Publications, Inc., Thousand Oaks*. doi, 10:9781483348858, 2009.
- [74] J. McKay and P. Marshall. The dual imperatives of action research. *Information Technology & People*, 14(1):46–59, 2001.
- [75] S. B. Merriam and R. S. Grenier. *Qualitative research in practice: Examples for discussion and analysis*. John Wiley & Sons, 2019.
- [76] S. Mishra and A. Misra. Structured and unstructured big data analytics. In *2017 International Conference on Current Trends in Computer, Electrical, Electronics and Communication (CTCEEC)*, pages 740–746. IEEE, 2017.
- [77] A. F. Mohammed, V. T. Humbe, and S. S. Chowhan. A review of big data environment and its related technologies. In *2016 International Conference on Information Communication and Embedded Systems (ICICES)*, pages 1–5. IEEE, 2016.
- [78] J. Moran-Ellis. Real world research: A resource for social scientists and practitioner-researchers. *Sociology*, 28(2):642–644, 1994.
- [79] H. Muccini and M. Sharaf. Caps: Architecture description of situational aware

- cyber physical systems. In *2017 IEEE International Conference on Software Architecture (ICSA)*, pages 211–220. IEEE, 2017.
- [80] A. R. Munappy, D. I. Mattos, J. Bosch, H. H. Olsson, and A. Dakkak. From ad-hoc data analytics to dataops. In *Proceedings of the International Conference on Software and System Processes*, pages 165–174, 2020.
- [81] P. Nesi, G. Pantaleo, M. Paolucci, and I. Zaza. Auditing and assessment of data traffic flows in an iot architecture. In *2018 IEEE 4th International Conference on Collaboration and Internet Computing (CIC)*, pages 388–391. IEEE, 2018.
- [82] S. Owen, P. Brereton, and D. Budgen. Protocol analysis: a neglected practice. *Communications of the ACM*, 49(2):117–122, 2006.
- [83] M. Pathirage. Kappa architecture-where every thing is a stream. *linea*/. Available: <http://milinda.pathirage.org/kappa-architecture.com/>. [Ultimo acceso: 12 Febrero 2019], 2020.
- [84] D. Perez-Palacin, J. Merseguer, J. I. Requeno, M. Guerriero, E. Di Nitto, and D. A. Tamburri. A uml profile for the design, quality assessment and deployment of data-intensive applications. *Software and Systems Modeling*, 18:3577–3614, 2019.
- [85] D. E. Perry, S. E. Sim, and S. Easterbrook. Case studies for software engineers. In *Proceedings of the 28th international conference on software engineering*, pages 1045–1046, 2006.
- [86] K. Petersen, C. Gencel, N. Asghari, D. Baca, and S. Betz. Action research as a model for industry-academia collaboration in the software engineering context. In *Proceedings of the 2014 international workshop on Long-term industrial collaboration on software engineering*, pages 55–62, 2014.
- [87] N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich. Data management challenges in production machine learning. In *Proceedings of the 2017 ACM International Conference on Management of Data*, pages 1723–1726, 2017.

- [88] N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich. Data lifecycle challenges in production machine learning: a survey. *ACM SIGMOD Record*, 47(2):17–28, 2018.
- [89] A. Raj, J. Bosch, H. H. Olsson, and T. J. Wang. Modelling data pipelines. In *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pages 13–20. IEEE, 2020.
- [90] P. Raj, T. Poongodi, B. Balusamy, and M. Khari. *The internet of things and big data analytics: integrated platforms and industry use cases*. CRC Press, 2020.
- [91] A. Ramasamy and S. Chowdhury. Big data quality dimensions: a systematic literature review. *JISTEM-Journal of Information Systems and Technology Management*, 17, 2020.
- [92] R. Rawat and R. Yadav. Big data: Big data analysis, issues and challenges and technologies. *IOP Conference Series: Materials Science and Engineering*, 1022(1):012014, jan 2021.
- [93] P. Runeson and M. Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical software engineering*, 14:131–164, 2009.
- [94] J. Saldaña. *The coding manual for qualitative researchers*. sage, 2021.
- [95] N. Sawant and H. Shah. Big data application architecture. In *Big data Application Architecture Q and A*, pages 9–28. Springer, 2013.
- [96] C. B. Seaman. Qualitative methods in empirical studies of software engineering. *IEEE Transactions on software engineering*, 25(4):557–572, 1999.
- [97] M. Sharaf, M. Abughazala, and H. Muccini. Arduino realization of caps iot architecture descriptions. In *Proceedings of the 12th European Conference on Software Architecture: Companion Proceedings*, pages 1–4, 2018.
- [98] M. Sharaf, M. Abughazala, H. Muccini, and M. Abusair. An architecture framework for modelling and simulation of situational-aware cyber-physical systems. In *European Conference on Software Architecture*, pages 95–111. Springer, 2017.

- [99] F. Sidi, P. H. Shariat Panahy, L. S. Affendey, M. A. Jabar, H. Ibrahim, and A. Mustapha. Data quality: A survey of data quality dimensions. In *2012 International Conference on Information Retrieval & Knowledge Management*, pages 300–304, 2012.
- [100] Y. Simmhan and S. Perera. Big data analytics platforms for real-time applications in iot. In *Big data analytics*, pages 115–135. Springer, 2016.
- [101] I. I. Sinan, J. Degila, V. Nwaocha, and S. A. Onashoga. Data architectures’ evolution and protection. In *2022 International Conference on Electrical, Computer and Energy Technologies (ICECET)*, pages 1–6. IEEE, 2022.
- [102] J. Singer, S. E. Sim, and T. C. Lethbridge. Software engineering data collection for field studies. In *Guide to Advanced Empirical Software Engineering*, pages 9–34. Springer, 2008.
- [103] S. Soni and A. Singh. Improving data quality using big data framework: A proposed approach. In *IOP Conference Series: Materials Science and Engineering*, volume 1022, page 012092. IOP Publishing, 2021.
- [104] A. Strauss and J. Corbin. Basics of qualitative research techniques. 1998.
- [105] J. Sutton and Z. Austin. Qualitative research: Data collection, analysis, and management. *The Canadian journal of hospital pharmacy*, 68(3):226, 2015.
- [106] E. Syriani, L. Luhunu, and H. Sahraoui. Systematic mapping study of template-based code generation. *Computer Languages, Systems & Structures*, 52:43–62, 2018.
- [107] I. Taleb, M. A. Serhani, C. Bouhaddioui, and R. Dssouli. Big data quality framework: a holistic approach to continuous quality management. *Journal of Big Data*, 8(1):1–41, 2021.
- [108] I. Taleb, M. A. Serhani, and R. Dssouli. Big data quality: A survey. In *2018 IEEE International Congress on Big Data (BigData Congress)*, pages 166–173, 2018.

- [109] Talend. How modern data architecture drives real business results, 2018.
- [110] Talend. What is a data architecture framework?, 2018.
- [111] techtarget. What is data architecture? a data management blueprint, 2018.
- [112] A. A. Tole. Big data challenges. *Database systems journal*, 4(3), 2013.
- [113] Towardsdatascience. Designing data products, 2018.
- [114] Twitter. Twitter seeing 6 billion api calls per day, 70k per second, 2010.
- [115] C. Verma and R. Pandey. Comparative analysis of gfs and hdfs: technology and architectural landscape. In *2018 10th International Conference on Computational Intelligence and Communication Networks (CICN)*, pages 54–58. IEEE, 2018.
- [116] J. M. Verner, J. Sampson, V. Tosic, N. A. Bakar, and B. A. Kitchenham. Guidelines for industrially-based multiple case studies in software engineering. In *2009 Third International Conference on Research Challenges in Information Science*, pages 313–324. IEEE, 2009.
- [117] M. Voelter. A catalog of patterns for program generation. In *EuroPLoP*, pages 285–320, 2003.
- [118] A. Von Mayrhauser and A. M. Vans. Identification of dynamic comprehension processes during large scale maintenance. *IEEE Transactions on Software Engineering*, 22(6):424–437, 1996.
- [119] J. Wang, Y. Liu, P. Li, Z. Lin, S. Sindakis, and S. Aggarwal. Overview of data quality: Examining the dimensions, antecedents, and impacts of data quality. *Journal of the Knowledge Economy*, pages 1–20, 2023.
- [120] J. Warren and N. Marz. *Big Data: Principles and best practices of scalable realtime data systems*. Simon and Schuster, 2015.

- [121] D. Wei, H. Ning, F. Shi, Y. Wan, J. Xu, S. Yang, and L. Zhu. Dataflow management in the internet of things: Sensing, control, and security. *Tsinghua Science and Technology*, 26(6):918–930, 2021.
- [122] L. P. Wong. Data analysis in qualitative research: A brief guide to using nvivo. *Malaysian family physician: the official journal of the Academy of Family Physicians of Malaysia*, 3(1):14, 2008.
- [123] X. Zheng, Q. Li, and L. Kong. A data storage architecture supporting multi-level customization for saas. In *2010 Seventh Web Information Systems and Applications Conference*, pages 106–109. IEEE, 2010.
- [124] T. Zschörnig, R. Wehlitz, and B. Franczyk. Iot analytics architectures: challenges, solution proposals and future research directions. In *International Conference on Research Challenges in Information Science*, pages 76–92. Springer, 2020.

Appendix A

Appendix

Table A.1: Mapper Table from QD to GE

Quality Dimensions	Great Expectations
Uniqueness	expect_column_values_to_be_unique
Completeness	expect_column_values_to_not_be_null
	expect_column_values_to_be_null
Validity	expect_column_values_to_be_of_type
	expect_column_values_to_be_in_type_list
	expect_column_values_to_be_in_set
	expect_column_values_to_not_be_in_set
	expect_column_values_to_be_between
	expect_column_values_to_be_increasing
	expect_column_values_to_be_decreasing
	expect_column_distinct_values_to_equal_set
	expect_column_distinct_values_to_contain_set

Table A.1 continued from previous page

Quality Dimensions	Great Expectations
	expect_column_mean_to_be_between
	expect_column_median_to_be_between
	expect_column_stdev_to_be_between
	expect_column_unique_value_count_to_be_between
	expect_column_most_common_value_to_be_in_set
	expect_column_sum_to_be_between
	expect_column_min_to_be_between
	expect_column_max_to_be_between
Consistency	expect_column_value_lengths_to_be_between
	expect_column_value_lengths_to_equal
	expect_column_values_to_match_regex
	expect_column_values_to_match_regex_list
	expect_column_values_to_match_like_pattern
	expect_column_values_to_not_match_like_pattern
	expect_column_values_to_match_like_pattern_list
	expect_column_values_to_not_match_like_pattern_list
	expect_column_values_to_match_strftime_format
	expect_column_values_to_be_dateutil_parseable
	expect_column_values_to_be_json_parseable
	expect_column_values_to_match_json_schema
	expect_column_values_to_not_be_null

Timeliness

Table A.1 continued from previous page

Quality Dimensions	Great Expectations
	expect_column_min_to_be_between
	expect_column_max_to_be_between
	expect_column_values_to_be_in_set
	expect_column_values_to_not_be_in_set
	expect_column_values_to_be_between
Accuracy	expect_column_values_to_not_be_null
	expect_column_values_to_be_null
	expect_column_values_to_be_of_type
	expect_column_values_to_be_in_type_list
	expect_column_values_to_be_in_set
	expect_column_values_to_not_be_in_set
	expect_column_values_to_be_between
	expect_column_values_to_be_increasing
	expect_column_values_to_be_decreasing
	expect_column_value_lengths_to_be_between
	expect_column_value_lengths_to_equal
	expect_column_values_to_match_regex
	expect_column_values_to_not_match_regex
	expect_column_values_to_match_regex_list
	expect_column_values_to_not_match_regex_list
	expect_column_values_to_match_like_pattern

Table A.1 continued from previous page

Quality Dimensions	Great Expectations
	expect_column_values_to_not_match_like_pattern
	expect_column_values_to_match_like_pattern_list
	expect_column_values_to_not_match_like_pattern_list
	expect_column_values_to_match_strftime_format
	expect_column_values_to_be_dateutil_parseable
	expect_column_values_to_be_json_parseable
	expect_column_values_to_match_json_schema
	expect_column_distinct_values_to_equal_set
	expect_column_distinct_values_to_contain_set
	expect_column_mean_to_be_between
	expect_column_median_to_be_between
	expect_column_stddev_to_be_between
	expect_column_unique_value_count_to_be_between
	expect_column_most_common_value_to_be_in_set
	expect_column_sum_to_be_between
	expect_column_min_to_be_between
	expect_column_max_to_be_between
	expect_column_kl_divergence_to_be_less_than